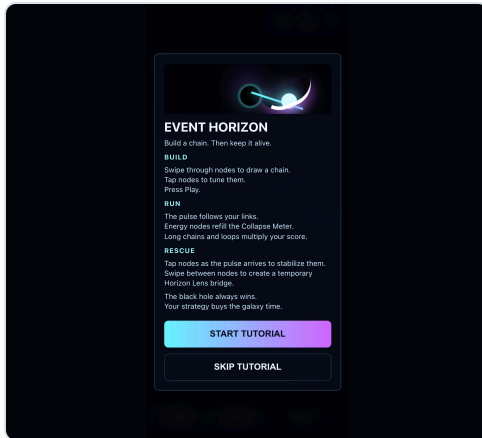


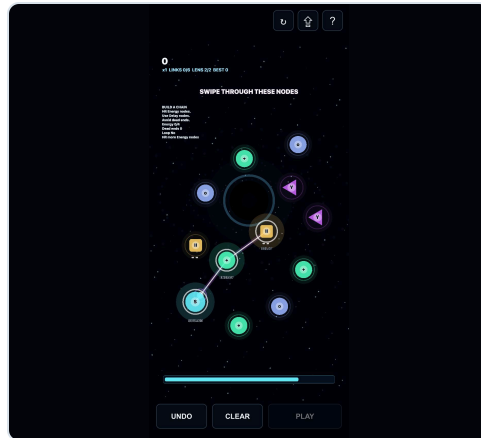
Event Horizon Iteration 04 Report

Playability, tutorial clarity, and tap/swipe/strategy grammar.

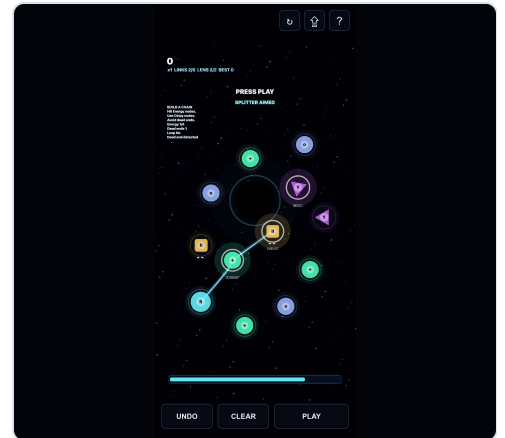
Screenshots



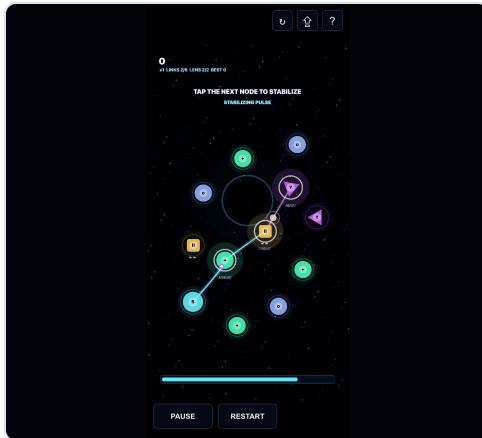
Help and tutorial entry



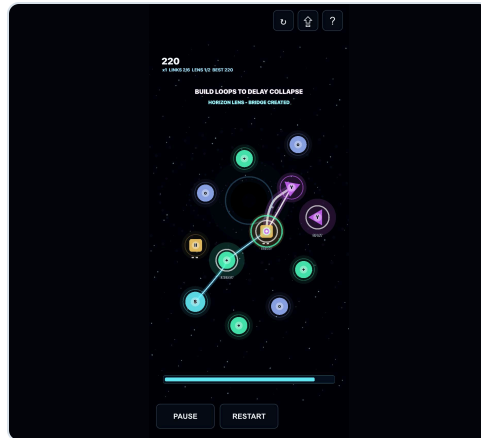
Swipe-chain tutorial



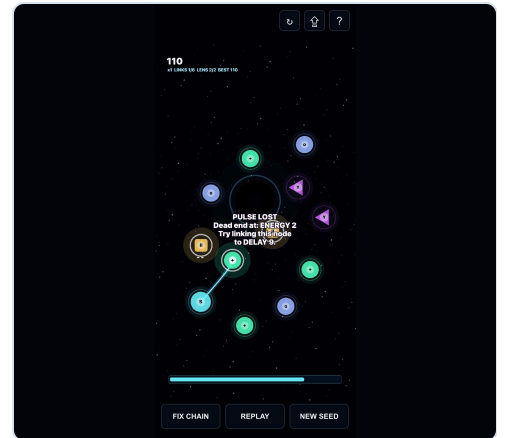
Node tap strategy



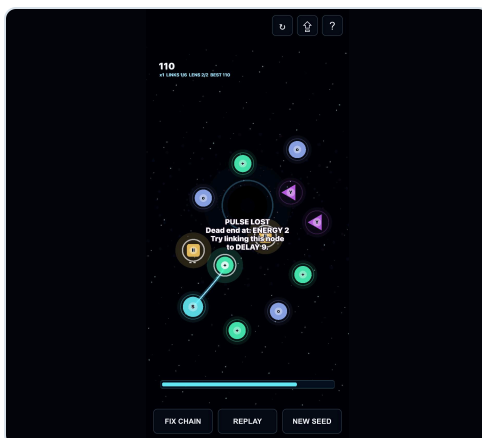
Pulse running



Horizon Lens rescue



Dead-end fix prompt



End screen

Changed Source Index

- README.md
- index.html
- package.json
- scripts/capture-iteration-04-artifacts.mjs
- scripts/generate-iteration-04-report.mjs
- scripts/test-score-submit.mjs
- src/game/pulse/PulseGeometry.ts
- src/game/pulse/PulseInputController.ts
- src/game/pulse/PulseLevelGenerator.ts
- src/game/pulse/PulseMode.ts
- src/game/pulse/PulseRenderer.ts
- src/game/pulse/PulseSimulation.ts
- src/game/pulse/PulseTypes.ts
- src/main.ts

- src/styles.css
 - tests/e2e/playable.spec.ts
- tests/pulse-simulation.test.ts
 - tests/score-submit.test.ts

Event Horizon Iteration 04 Report

Summary

Iteration 04 focuses on playability. The core grammar is now visible and teachable: swipe through nodes to draw a chain, tap nodes to tune strategy, press Play, tap the pulse's next node to stabilize it, and swipe a Horizon Lens bridge to rescue the run.

Diagnosis Of Iteration 03 Confusion

Iteration 03 introduced the Pulse Chain pivot, but it still asked players to infer too much. Link placement was possible, but the first move was not obvious, node types were mostly color-coded, tap had little strategic meaning, and failure did not explain what to fix. Iteration 04 turns those missing ideas into tutorial steps and direct feedback.

New Tap/Swipe/Strategy Grammar

- Build: swipe through several nodes to create a directional chain automatically.
- Build: tap one node, then another node to create a precise link.
- Build: tap selected special nodes to prime Energy, cycle Delay timing, or aim Splitters.
- Run: tap the next node shortly before pulse arrival to stabilize it for score and dark-energy gain.
- Rescue: swipe between two nodes during playback to create a temporary Horizon Lens bridge.
- Strategy: hit Energy nodes, use Delay timing, branch with Splitters, avoid dead ends, and build loops.

Exact Files Changed

- README.md
- index.html
- package.json
- scripts/capture-iteration-04-artifacts.mjs
- scripts/generate-iteration-04-report.mjs
- scripts/test-score-submit.mjs
- src/game/pulse/PulseGeometry.ts
- src/game/pulse/PulseInputController.ts
- src/game/pulse/PulseLevelGenerator.ts
- src/game/pulse/PulseMode.ts
- src/game/pulse/PulseRenderer.ts
- src/game/pulse/PulseSimulation.ts
- src/game/pulse/PulseTypes.ts
- src/main.ts
- src/styles.css
- tests/e2e/playable.spec.ts
- tests/pulse-simulation.test.ts
- tests/score-submit.test.ts
- docs/artifacts/iteration-04-dead-end-fix-mobile.jpg
- docs/artifacts/iteration-04-end-screen-mobile.jpg
- docs/artifacts/iteration-04-help-mobile.jpg
- docs/artifacts/iteration-04-horizon-lens-mobile.jpg
- docs/artifacts/iteration-04-node-tap-strategy-mobile.jpg
- docs/artifacts/iteration-04-pulse-running-mobile.jpg
- docs/artifacts/iteration-04-test-results.txt
- docs/artifacts/iteration-04-tutorial-swipe-chain-mobile.jpg

Diff Summary

README.md		31	+-
index.html		21	+-
package.json		4	+-
scripts/test-score-submit.mjs		18	+-
src/game/pulse/PulseGeometry.ts		53	+++
src/game/pulse/PulseInputController.ts		24	+-
src/game/pulse/PulseLevelGenerator.ts		28	+-
src/game/pulse/PulseMode.ts		52	+++
src/game/pulse/PulseRenderer.ts		197	+++++
src/game/pulse/PulseSimulation.ts		538	+++++
src/game/pulse/PulseTypes.ts		65	+++
src/main.ts		69	+++
src/styles.css		40	++
tests/e2e/playable.spec.ts		227	+++++
tests/pulse-simulation.test.ts		171	+++++
tests/score-submit.test.ts		18	+-
16 files changed, 1257 insertions(+), 299 deletions(-)			

Tests Run And Results

```
COMMAND: npm run build
> event-horizon@0.1.0 build
> tsc --noEmit && vite build
```

```
vite v8.0.14 building client environment for production...
[2K
transforming...✓ 724 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html 3.46 kB | gzip: 1.18 kB
dist/assets/index-CVGETPBR.css 4.50 kB | gzip: 1.58 kB
dist/assets/webworkerAll-u0-LKNcn.js 0.06 kB | gzip: 0.07 kB
dist/assets/WebGLRenderer-Dr5I3uql.js 0.07 kB | gzip: 0.07 kB
dist/assets/CanvasRenderer-C_VE2uLM.js 0.07 kB | gzip: 0.08 kB
dist/assets/WebGPURenderer-BqF0A8r4.js 0.07 kB | gzip: 0.08 kB
dist/assets/init-nWfKNZF3.js 0.16 kB | gzip: 0.16 kB | map: 0.57 kB
dist/assets/browserAll-Dj01SdvZ.js 0.29 kB | gzip: 0.23 kB | map: 1.51 kB
dist/assets/getTextureBatchBindGroup-Dw2p-Wfk.js 0.40 kB | gzip: 0.31 kB | map: 1.78 kB
dist/assets/CanvasPool-9Gf99FtZ.js 0.80 kB | gzip: 0.45 kB | map: 3.50 kB
dist/assets/canvasUtils-By6x9n4d.js 6.07 kB | gzip: 2.07 kB | map: 21.84 kB
dist/assets/BufferResource-CHWjzlhG.js 10.57 kB | gzip: 2.81 kB | map: 25.57 kB
dist/assets/WebGPURenderer-CaI5dhXK.js 37.78 kB | gzip: 10.56 kB | map: 121.24 kB
dist/assets/FilterSystem-DKkk6B7X.js 40.30 kB | gzip: 13.07 kB | map: 159.86 kB
dist/assets/FederatedEventTarget-BGQ8UmS3.js 42.56 kB | gzip: 11.09 kB | map: 155.08 kB
dist/assets/WebGLRenderer-4UnqJ0bN.js 67.75 kB | gzip: 18.59 kB | map: 214.44 kB
dist/assets/RenderTargetSystem-D4_io0eT.js 77.17 kB | gzip: 22.67 kB | map: 295.31 kB
dist/assets/CanvasRenderer-lu0yL7Nx.js 79.94 kB | gzip: 24.36 kB | map: 404.18 kB
dist/assets/Geometry-C1JBQvr3.js 101.60 kB | gzip: 31.13 kB | map: 498.88 kB
dist/assets/index-DE1jNjA0.js 127.88 kB | gzip: 38.76 kB | map: 515.44 kB

✓ built in 149ms

COMMAND: npm run lint

> event-horizon@0.1.0 lint
> eslint .

COMMAND: npm run test

> event-horizon@0.1.0 test
> vitest run

RUN v4.0.15 /Users/kevinhegg/Desktop/event-horizon
✓ tests/score-submit.test.ts (3 tests) 13ms
✓ tests/simulation.test.ts (7 tests) 6ms
✓ tests/pulse-simulation.test.ts (12 tests) 14ms

Test Files 3 passed (3)
Tests 22 passed (22)
Start at 16:10:49
Duration 150ms (transform 115ms, setup 0ms, import 148ms, tests 33ms, environment 0ms)

COMMAND: npm run test:e2e

> event-horizon@0.1.0 test:e2e
> playwright test

Running 6 tests using 1 worker

(node:66296) Warning: The 'NO_COLOR' env is ignored due to the 'FORCE_COLOR' env being set.
(Use 'node --trace-warnings ...' to show where the warning was created)
(node:66296) Warning: The 'NO_COLOR' env is ignored due to the 'FORCE_COLOR' env being set.
(Use 'node --trace-warnings ...' to show where the warning was created)
1 [mobile-chrome] › tests/e2e/playable.spec.ts:4:1 › first visit opens updated help and tutorial (349ms)
✓ 2 [mobile-chrome] › tests/e2e/playable.spec.ts:12:1 › tutorial Step 1 highlights nodes and swipe creates a chain (995ms)
✓ 3 [mobile-chrome] › tests/e2e/playable.spec.ts:26:1 › tapping splitter changes tutorial state and output priority (668ms)
✓ 4 [mobile-chrome] › tests/e2e/playable.spec.ts:36:1 › pressing Play launches visible pulse and node tap stabilizes it (3.1s)
✓ 5 [mobile-chrome] › tests/e2e/playable.spec.ts:53:1 › swiping Horizon Lens creates bridge and replay records grammar (3.4s)
✓ 6 [mobile-chrome] › tests/e2e/playable.spec.ts:73:1 › dead-end failure shows suggested fix and FIX CHAIN returns to build (1.4s)

6 passed (10.6s)

COMMAND: npm run score:test

> event-horizon@0.1.0 score:test
> node scripts/test-score-submit.mjs

[
  {
    "status": 200,
    "body": {
      "ok": true,
      "acceptedAt": "2026-05-30T20:11:00.819Z",
      "score": 1280,
      "survivalMs": 67421
    }
  },
  {
    "status": 200,
    "body": {
      "ok": true,
      "acceptedAt": "2026-05-30T20:11:00.820Z",
      "score": 620,
      "survivalMs": 12120
    }
  }
]

COMMAND: npm run capture:iteration-04

> event-horizon@0.1.0 capture:iteration-04
> node scripts/capture-iteration-04-artifacts.mjs

Captured docs/artifacts/iteration-04-help-mobile.jpg
Captured docs/artifacts/iteration-04-tutorial-swipe-chain-mobile.jpg
Captured docs/artifacts/iteration-04-node-tap-strategy-mobile.jpg
Captured docs/artifacts/iteration-04-pulse-running-mobile.jpg
Captured docs/artifacts/iteration-04-horizon-lens-mobile.jpg
Captured docs/artifacts/iteration-04-dead-end-fix-mobile.jpg
Captured docs/artifacts/iteration-04-end-screen-mobile.jpg

COMMAND: npm run report:iteration-04

> event-horizon@0.1.0 report:iteration-04
> node scripts/generate-iteration-04-report.mjs

Report generated after this log was written.
```

Screenshots

- docs/artifacts/iteration-04-help-mobile.jpg
- docs/artifacts/iteration-04-tutorial-swipe-chain-mobile.jpg
- docs/artifacts/iteration-04-node-tap-strategy-mobile.jpg
- docs/artifacts/iteration-04-pulse-running-mobile.jpg
- docs/artifacts/iteration-04-horizon-lens-mobile.jpg

- docs/artifacts/iteration-04-dead-end-fix-mobile.jpg
- docs/artifacts/iteration-04-end-screen-mobile.jpg

Known Limitations

- Playwright mobile simulation passed; no physical phone was manually tested in this run.
- Tutorial is intentionally hand-tuned for tutorial-001; future seeds still use simple generated layouts.
- Horizon Lens is still implemented as a temporary bridge, not freeform pulse deflection.
- Splitter aiming is useful and visible, but the strategy model can still be tuned further with player testing.
- Sound, haptics, and richer retry analytics remain future work.

Next Recommended Iteration

- Test on real iPhone and Android hardware and tune touch radii and tutorial timing.
- Add audio/haptic feedback for chain creation, stabilization, lens creation, and dead ends.
- Add a replay viewer that visually replays chain swipes, taps, and Horizon Lens inputs.
- Add a small “why this chain is good” animation after successful loops.

Full Diffs For Tracked Changes

```
diff --git a/README.md b/README.md
index fdd1cf..b5def0e 100644
--- a/README.md
+++ b/README.md
@@ -10,6 +10,9 @@ Event Horizon is a mobile-first cosmic chain-reaction game about delaying a gala
- Seeded 'mulberry32' RNG and replay payloads from seed + input timings
- Default Pulse Chain mode: connect Dark Energy Nodes, press Play, and watch a Stabilizing Pulse travel the network
- Tap-tap and drag link placement with a visible link budget, undo, clear, and reset
+ Swipe-through-node chain drawing, tap-tap link placement, and tap-based node tuning
+ Interactive `tutorial-001` first-run seed that teaches swipe chain, tap strategy, Play, stabilize taps, and Horizon Lens rescue
- Energy, Delay, Splitter, Conduit, and Source nodes with scoring and multiplier rules
+ Energy nodes can be primed, Delay nodes cycle timing, and Splitters cycle output priority
+ Pulse-phase taps stabilize arriving nodes for score and dark-energy gain
- Horizon Lens swipes during pulse playback create short-lived temporary bridges
- Path-based input recording with mobile Pointer Events and TouchEvent fallback
@@ -40,4 +43,6 @@ npm run score:test
npm run capture:iteration-03
npm run report:iteration-03
+npm run capture:iteration-04
+npm run report:iteration-04
npm run capture:iteration-02
npm run report:iteration-02
@@ -95,9 +100,19 @@ npx netlify dev
  "startedAt": 1780185600000,
  "buildInputs": [
-   { "t": 0, "kind": "link", "fromId": 1, "toId": 2 },
-   { "t": 300, "kind": "link", "fromId": 2, "toId": 3 },
+   {
+     "t": 0,
+     "kind": "chainSwipe",
+     "nodeIds": [1, 2, 3],
+     "path": [
+       { "x": 250, "y": 1388, "t": 0 },
+       { "x": 420, "y": 1165, "t": 90 },
+       { "x": 635, "y": 1010, "t": 180 }
+     ]
+   },
+   { "t": 300, "kind": "nodeTap", "nodeId": 4, "action": "splitter" },
+   { "t": 620, "kind": "play" }
  ],
  "liveInputs": [
+   { "t": 2200, "kind": "stabilize", "nodeId": 3, "rating": "stabilized", "success": true },
+   {
+     "t": 1600,
@@ -118,4 +133,6 @@ npx netlify dev
  "loopsCompleted": 1,
  "linksUsed": 5,
+  "bestChainLength": 5,
+  "energyNodesHit": 2,
+  "stabilized": false,
+  "collapsed": false
@@ -145,8 +162,8 @@ Legacy mode still uses:
```bash
-git switch -c feat/pulse-chain-pivot
+git switch -c feat/iteration-04-playability-tap-swipe-strategy
git add .
-git commit -m "Pivot Event Horizon to pulse chain gameplay"
-git push -u origin feat/pulse-chain-pivot
+gh pr create --base main --head feat/pulse-chain-pivot --title "Pivot Event Horizon to pulse chain gameplay" --body-file docs/iteration-03-report.md
+git commit -m "Improve Event Horizon playability tutorial and strategy"
+git push -u origin feat/iteration-04-playability-tap-swipe-strategy
+gh pr create --base main --head feat/iteration-04-playability-tap-swipe-strategy --title "Improve Event Horizon playability tutorial and strategy" --body-file docs/iteration-04-report.md
diff --git a/index.html b/index.html
index ba08306..f41f594 100644
--- a/index.html
+++ b/index.html
@@ -32,15 +32,14 @@
</div>
<h1 id="help-title">EVENT HORIZON</h1>
<p>Build a dark-energy chain. Then press Play.</p>

- Connect nodes with gravitational links.
- Press Play to launch the stabilizing pulse.
- Energy nodes refill the Collapse Meter.
- Long chains and loops build multipliers.
- During playback, swipe to create a temporary Horizon Lens bridge.
- Keep the galaxy alive as long as you can.
-
<p>The black hole always wins.
</p>
<button id="help-play-button" type="button">PLAY</button>
+ <p>Build a chain. Then keep it alive.</p>
+ <h2>BUILD</h2>
+ <p>Swipe through nodes to draw a chain.
</p>
+ <h2>RUN</h2>
+ <p>The pulse follows your links.
</p>
+ <h2>RESCUE</h2>
+ <p>Tap nodes as the pulse arrives to stabilize them.
</p>
+ <p>The black hole always wins.
</p>
+ <button id="help-play-button" type="button">START TUTORIAL</button>
+ <button id="help-skip-button" type="button">SKIP TUTORIAL</button>
</div>
</section>
diff --git a/package.json b/package.json
index dee2931..b19037e 100644
--- a/package.json
+++ b/package.json
```

```

@@ -15,7 +15,9 @@
"capture:iteration-02": "node scripts/capture-iteration-02-artifacts.mjs",
"capture:iteration-03": "node scripts/capture-iteration-03-artifacts.mjs",
+ "capture:iteration-04": "node scripts/capture-iteration-04-artifacts.mjs",
+ "report:pdf": "node scripts/generate-report-pdf.mjs",
- "report:iteration-02": "node scripts/generate-iteration-02-report.mjs",
+ "report:iteration-03": "node scripts/generate-iteration-03-report.mjs",
+ "report:iteration-04": "node scripts/generate-iteration-04-report.mjs"
},
"dependencies": {
diff --git a/scripts/test-score-submit.mjs b/scripts/test-score-submit.mjs
index 6d1bb1c..dd8bde1 100644
--- a/scripts/test-score-submit.mjs
+++ b/scripts/test-score-submit.mjs
@@ -17,12 +17,22 @@ const pulseReplay = {
 version: 1,
 modes: 'pulse-chain',
- seed: 'tutorial',
+ seed: 'tutorial-001',
 startedAt: 1780185600000,
 buildInputs: [
 { t: 0, kind: 'link', fromId: 1, toId: 2 },
 { t: 300, kind: 'link', fromId: 2, toId: 3 },
 {
 t: 0,
 kind: 'chainSwipe',
 nodeIds: [1, 2, 3],
 path: [
 { x: 250, y: 1380, t: 0 },
 { x: 420, y: 1165, t: 90 },
 { x: 635, y: 1010, t: 180 }
]
 },
 { t: 300, kind: 'nodeTap', nodeId: 4, action: 'splitter' },
 { t: 620, kind: 'play' }
],
 liveInputs: [
 { t: 2200, kind: 'stabilize', nodeId: 3, rating: 'stabilized', success: true },
 {
 t: 1600,
@@ -43,4 +53,6 @@ const pulseReplay = {
 loopsCompleted: 1,
 linksUsed: 5,
+ bestChainLength: 5,
+ energyNodesHit: 2,
+ stabilized: false,
+ collapsed: false
diff --git a/src/game/pulse/PulseGeometry.ts b/src/game/pulse/PulseGeometry.ts
index 9abafeb..3c5d7dc 100644
--- a/src/game/pulse/PulseGeometry.ts
+++ b/src/game/pulse/PulseGeometry.ts
@@ -117,4 +117,26 @@ export function pathCrossesNodeRadius(path: readonly WorldPoint[], node: PulseNo
}

+export function nodesCrossedByPath(
+ nodes: readonly PulseNode[],
+ path: readonly WorldPoint[],
+ radius = 76
+): PulseNode[] {
+ const crossed = nodes
+ .map((node) => {
+ const hit = firstPathHit(node, path, radius + node.radius * 0.35);
+ return hit === undefined ? undefined : { node, order: hit };
+ })
+ .filter((entry): entry is { node: PulseNode; order: number } => entry !== undefined)
+ .sort((a, b) => a.order - b.order)
+ .map((entry) => entry.node);
+ const result: PulseNode[] = [];
+ for (const node of crossed) {
+ if (result[result.length - 1]?.id !== node.id) {
+ result.push(node);
+ }
+ }
+ return result;
+}

+export function distanceNodeToPath(node: PulseNode, path: readonly WorldPoint[]): number {
+ if (path.length === 0) {
@@ -133,4 +155,35 @@ export function distanceNodeToPath(node: PulseNode, path: readonly WorldPoint[])
}

+function firstPathHit(node: PulseNode, path: readonly WorldPoint[], radius: number): number | undefined {
+ if (path.length === 0) {
+ return undefined;
+ }
+ let traveled = 0;
+ if (path.length === 1) {
+ return Math.sqrt(distanceSquared(node.x, node.y, path[0].x, path[0].y)) <= radius ? 0 : undefined;
+ }
+ for (let index = 1; index < path.length; index += 1) {
+ const previous = path[index - 1];
+ const point = path[index];
+ const segmentLength = Math.max(0.001, Math.hypot(point.x - previous.x, point.y - previous.y));
+ const distance = distancePointToSegment(node, previous, point);
+ if (distance <= radius) {
+ return traveled + segmentLength * projectionT(node, previous, point);
+ }
+ traveled += segmentLength;
+ }
+ return undefined;
+}

+function projectionT(point: WorldPoint, start: WorldPoint, end: WorldPoint): number {
+ const dx = end.x - start.x;
+ const dy = end.y - start.y;
+ const lengthSq = dx * dx + dy * dy;
+ if (lengthSq <= 0.000001) {
+ return 0;
+ }
+ return clamp(((point.x - start.x) * dx + (point.y - start.y) * dy) / lengthSq, 0, 1);
+}

+export function curvedLinkPath(from: WorldPoint, to: WorldPoint, bend = 0.12): WorldPoint[] {
+ const mid = midpointOf(from, to);
diff --git a/src/game/pulse/PulseInputController.ts b/src/game/pulse/PulseInputController.ts
index f779c1e..93d2185 100644
--- a/src/game/pulse/PulseInputController.ts
+++ b/src/game/pulse/PulseInputController.ts
@@ -76,4 +76,8 @@ export class PulseInputController {
 const snapshot = this.sim.getSnapshot();
 if (snapshot.phase !== 'build') {
 const pulseNode = this.nearestNode(point, 86);
 if (snapshot.phase === 'pulse' && pulseNode) {
 this.lastResult = this.sim.stabilizeNode(pulseNode.id);
 }
 return;
 }
}

@@ -81,5 +85,10 @@ export class PulseInputController {
 if (!node) {
 this.clearSelection();
 this.lastResult = { ok: false, kind: 'invalid', message: 'NO NODE' };
 this.lastResult = this.sim.removeLinkNear(point);
 }
}

```

```

+ return;
+ }
+ if (snapshot.tutorialStep === 'tap-splitter' && node.type === 'splitter') {
+ this.lastResult = this.sim.tapNode(node.id);
+ this.selectedNodeId = undefined;
+ return;
+ }
@@ -90,5 +99,8 @@ export class PulseInputController {
+ }
+ if (this.selectedNodeId === node.id) {
- this.clearSelection();
+ this.lastResult = this.sim.tapNode(node.id);
+ this.selectedNodeId = undefined;
+ this.previewFromId = undefined;
+ this.previewPoint = undefined;
+ return;
+ }
@@ -100,11 +112,5 @@ export class PulseInputController {
 const snapshot = this.sim.getSnapshot();
 if (snapshot.phase === 'build') {
- const startNode = this.nearestNode(gesture.start, 88);
- const endNode = this.nearestNode(gesture.end, 94);
- if (startNode && endNode) {
- this.lastResult = this.sim.addLink(startNode.id, endNode.id);
- } else {
- this.lastResult = { ok: false, kind: 'invalid', message: 'DRAG NODE TO NODE' };
- }
+ this.lastResult = this.sim.applyChainSwipe(gesture.points);
+ this.selectedNodeId = undefined;
+ } else if (snapshot.phase === 'pulse') {
diff --git a/src/game/pulse/PulseLevelGenerator.ts b/src/game/pulse/PulseLevelGenerator.ts
index 47c0878..681b643 100644
--- a/src/game/pulse/PulseLevelGenerator.ts
+++ b/src/game/pulse/PulseLevelGenerator.ts
@@ -12,5 +12,5 @@ export function getDailyPulseSeed(date = new Date()): string {
 export function generatePulseLevel(seed: string): PulseLevel {
 const rng = createSeededRandom('pulse-chain-${seed}');
- const firstSeed = seed === 'daily-2026-05-30' || seed === 'tutorial' || seed === 'eh-pulse-alpha';
+ const firstSeed = seed === 'tutorial-001' || seed === 'daily-2026-05-30' || seed === 'tutorial' || seed === 'eh-pulse-alpha';
+ const baseNodes = firstSeed ? tutorialNodes() : generatedNodes(rng);
 return {
@@ -26,15 +26,15 @@ export function generatePulseLevel(seed: string): PulseLevel {
 function tutorialNodes(): PulseNode[] {
 const specs: Array<[PulseNodeType, number, number, number, string]> = [
- ['source', 255, 1388, 0, 'SOURCE'],
- ['energy', 420, 1165, 1, 'ENERGY'],
- ['delay', 640, 1015, 1, 'DELAY'],
- ['splitter', 780, 760, 2, 'SPLIT'],
- ['energy', 525, 610, 2, 'ENERGY'],
- ['energy', 835, 1215, 2, 'ENERGY'],
- ['conduit', 310, 810, 2, 'CONDUIT'],
- ['conduit', 700, 1410, 2, 'CONDUIT'],
- ['delay', 250, 1080, 1, 'DELAY'],
+ ['source', 250, 1388, 0, 'SOURCE'],
+ ['energy', 420, 1165, 1, '+100'],
+ ['delay', 635, 1010, 1, 'DELAY'],
+ ['splitter', 770, 775, 2, 'SPLIT'],
+ ['energy', 515, 620, 2, '+100'],
+ ['energy', 835, 1215, 2, '+100'],
+ ['conduit', 305, 805, 2, 'CONDUIT'],
+ ['conduit', 705, 1412, 2, 'CONDUIT'],
+ ['delay', 252, 1085, 1, 'DELAY'],
+ ['splitter', 910, 935, 2, 'SPLIT'],
- ['energy', 485, 1515, 2, 'ENERGY'],
+ ['energy', 485, 1515, 2, '+100'],
+ ['conduit', 805, 545, 2, 'CONDUIT']
];
@@ -110,5 +110,9 @@ function makeNode(id: number, type: PulseNodeType, x: number, y: number, ring: n
 radius: type === 'source' ? 54 : type === 'splitter' ? 50 : 46,
 activationMs: 0,
 scoreCooldownMs: 0,
+ scoreCooldownMs: 0,
+ primed: false,
+ delayLevel: 1,
+ splitterPriority: 0,
+ stabilizedMs: 0
 };
}
diff --git a/src/game/pulse/PulseMode.ts b/src/game/pulse/PulseMode.ts
index 5966d5f..4d03748 100644
--- a/src/game/pulse/PulseMode.ts
+++ b/src/game/pulse/PulseMode.ts
@@ -132,4 +132,11 @@ export class PulseMode {
+ }
+ fixChain() {
+ this.scoreSubmitted = false;
+ this.input?.clearSelection();
+ this.loop.resetClock();
+ return this.sim.fixChain();
+ }
+
+ undo() {
+ return this.sim.undo();
@@ -144,4 +144,49 @@ export class PulseMode {
+ simulateChainSwipe(nodeIds: readonly number[]) {
+ const points: Array<WorldPoint & { t: number }> = [];
+ for (const id of nodeIds) {
+ const node = this.sim.getNodes().find((candidate) => candidate.id === id);
+ if (node) {
+ points.push({ x: node.x, y: node.y, t: points.length * 80 });
+ }
+ }
+ return this.sim.applyChainSwipe(points);
+ }
+
+ startTutorial(): void {
+ this.sim.startTutorial();
+ this.scoreSubmitted = false;
+ this.loop.resetClock();
+ }
+
+ skipTutorial(): void {
+ this.sim.skipTutorial();
+ }
+
+ getTutorialStep() {
+ return this.sim.getTutorialStep();
+ }
+
+ analyzeChain() {
+ return this.sim.analyzeChain();
+ }
+
+ getSuggestedFixes() {
+ return this.sim.getSuggestedFixes();
+ }
+
+ primeNode(id: number) {
+ return this.sim.primeNode(id);
+ }

```

```

+ cycleNode(id: number) {
+ return this.sim.cycleNode(id);
+ }
+
+ stabilizeNode(id: number) {
+ return this.sim.stabilizeNode(id);
+ }
+
+ forceBuildPhase(): void {
+ this.sim.forceBuildPhase();
+ }
diff --git a/src/game/pulse/PulseRenderer.ts b/src/game/pulse/PulseRenderer.ts
index f0f8e72..ad2c1a9 100644
--- a/src/game/pulse/PulseRenderer.ts
+++ b/src/game/pulse/PulseRenderer.ts
@@ -23,4 +23,5 @@ export class PulseRenderer {
 private readonly previewLayer = new Graphics();
 private readonly nodeLayer = new Graphics();
+ private readonly nodeTextLayer = new Container();
+ private readonly pulseLayer = new Graphics();
 private readonly blackHole = new Graphics();
@@ -45,4 +45,4 @@ export class PulseRenderer {
 });
+ private readonly messageText = new Text({
+ text: '',
+ style: new TextStyle({
+ align: 'center',
+ fill: '#9ffcff',
+ fontFamily: 'Inter, system-ui, sans-serif',
+ fontSize: 26,
+ fontWeight: '900',
+ stroke: { color: '#061120', width: 4 }
+ })
+ });
+ private readonly strategyText = new Text({
+ text: '',
+ style: new TextStyle({
+ fill: '#7fbfff',
+ fontFamily: 'Inter, system-ui, sans-serif',
+ fontSize: 18,
+ fontWeight: '800',
+ lineHeight: 23,
+ stroke: { color: '#061120', width: 4 }
+ })
+ });
 private readonly meter = new Graphics();
 private readonly debugText = new Text({
@@ -56,9 +56,10 @@ export class PulseRenderer {
 fill: 'ffffff',
 fontFamily: 'Inter, system-ui, sans-serif',
 fontSize: 48,
 fontStyle: 40,
+ fontWeight: '900',
 stroke: { color: '#12051c', width: 6 }
 });
+ private readonly nodeTexts = new Map<number, Text>();

 constructor(private readonly stage: Container) {
@@ -72,14 +72,14 @@ export class PulseRenderer {
 this.previewLayer,
 this.nodeLayer,
+ this.nodeTextLayer,
 this.pulseLayer,
 this.hintText,
 this.hud
);
 this.hud.addChild(this.meter, this.scoreText, this.metaText, this.debugText, this.endText);
+ this.hud.addChild(this.meter, this.scoreText, this.metaText, this.strategyText, this.messageText, this.debugText, this.endText);
 this.hintText.anchor.set(0.5);
 this.hintText.position.set(WORLD_WIDTH / 2, 260);
+ this.messageText.anchor.set(0.5);
+ this.messageText.position.set(WORLD_WIDTH / 2, 318);
 this.scoreText.position.set(68, 68);
 this.metaText.position.set(72, 130);
- this.debugText.position.set(72, 178);
+ this.strategyText.position.set(68, 330);
+ this.debugText.position.set(72, 520);
 this.endText.anchor.set(0.5);
 this.endText.position.set(WORLD_WIDTH / 2, WORLD_HEIGHT * 0.52);
@@ -138,9 +138,9 @@ export class PulseRenderer {
 const layer = link.temporary ? this.tempLinkLayer : this.linkLayer;
 const alpha = link.temporary ? clamp(1 - link.ageMs / link.expiresMs, 0, 1) : 1;
 const flash = clamp(link.flashMs / 520, 0, 1);
+ this.drawCurve(layer, curvedLinkPath(from, to, link.temporary ? -0.16 : 0.12), {
+ glowColor: link.temporary ? 0xd267ff : 0x4dcccff,
+ coreColor: link.temporary ? 0xffffffff : 0x9fe7fff,
+ alpha,
+ width: link.temporary ? 9 : 6
+ });
 this.drawFlowDots(layer, from, to, snapshot.timeMs, link.temporary, alpha);
@@ -181,4 +181,4 @@ export class PulseRenderer {
 private renderPreview(snapshot: PulseSnapshot, input: PulseInputViewState): void {
 this.previewLayer.clear();
 if (snapshot.tutorialGhostPath.length > 1) {
+ this.drawCurve(this.previewLayer, resamplePath(snapshot.tutorialGhostPath, 34), {
+ glowColor: 0xd267ff,
+ coreColor: 0xffffffff,
+ alpha: 0.62 + Math.sin(snapshot.timeMs * 0.006) * 0.18,
+ width: 6
+ });
+ const ghost = pointAlongPolyline(snapshot.tutorialGhostPath, (snapshot.timeMs * 0.00032) % 1);
+ this.previewLayer.circle(ghost.x, ghost.y, 18).fill({ color: 0xffffffff, alpha: 0.74 });
+ this.previewLayer.circle(ghost.x, ghost.y, 34).stroke({ color: 0xd267ff, alpha: 0.48, width: 5 });
+ }
 if (snapshot.phase === 'build' && input.previewFromId && input.previewPoint) {
 const from = findNode(snapshot.nodes, input.previewFromId);
@@ -192,4 +192,4 @@ export class PulseRenderer {
 }
+ if (snapshot.phase === 'build' && input.liveGesture.length > 1) {
+ this.drawCurve(this.previewLayer, resamplePath(input.liveGesture, 36), {
+ glowColor: 0xd267ff,
+ coreColor: 0xffffffff,
+ alpha: 0.82,
+ width: 7
+ });
+ }
+ if (snapshot.phase === 'pulse' && input.liveGesture.length > 1) {
+ this.drawCurve(this.previewLayer, resamplePath(input.liveGesture, 36), {
@@ -204,20 +204,20 @@ export class PulseRenderer {
 private renderNodes(snapshot: PulseSnapshot, input: PulseInputViewState): void {
 this.nodeLayer.clear();
 const visibleNodeIds = new Set<number>();
 for (const node of snapshot.nodes) {
 const selected = input.selectedNodeId === node.id;
 const nearest = input.nearestNodeId === node.id;
 const active = node.activationMs > 0;
+ const highlighted =
+ snapshot.tutorialHighlightNodeIds.includes(node.id) ||
+ snapshot.deadEndNodeId === node.id ||

```

```

+ snapshot.suggestedFixes.some((fix) => fix.fromId === node.id || fix.toId === node.id);
const color = NODE_COLORS[node.type];
- const halo = selected || nearest || active ? 0.56 : node.type === 'energy' || node.type === 'source' ? 0.34 : 0.22;
this.nodeLayer.circle(node.x, node.y, node.radius + 26 + (active ? 18 : 0)).fill({ color, alpha: halo * 0.23 });
- this.nodeLayer.circle(node.x, node.y, node.radius + 12).stroke({ color, alpha: halo, width: selected ? 7 : 4 });
+ const halo = selected || nearest || active || highlighted ? 0.68 : node.type === 'energy' || node.type === 'source' ? 0.34 : 0.22;
+ this.nodeLayer.circle(node.x, node.y, node.radius + 30 + (active || highlighted ? 20 : 0)).fill({ color, alpha: halo * 0.23 });
+ if (node.primed) {
+ this.nodeLayer.circle(node.x, node.y, node.radius + 26).stroke({ color: 0xffffffff, alpha: 0.84, width: 6 });
+ }
+ if (node.stabilizedMs > 0) {
+ this.nodeLayer.circle(node.x, node.y, node.radius + 38).stroke({ color: 0xd4ffbf, alpha: 0.76, width: 8 });
+ }
+ this.nodeLayer.circle(node.x, node.y, node.radius + 12).stroke({ color: highlighted ? 0xffffffff : color, alpha: halo, width: selected || highlighted ? 7 : 4 });
+ if (node.type === 'splitter') {
- this.nodeLayer.regularPoly(node.x, node.y, node.radius, 3, -Math.PI / 2).fill({ color, alpha: 0.86 });
+ this.nodeLayer.regularPoly(node.x, node.y, node.radius, 3, -Math.PI / 2 + node.splitterPriority * 0.7).fill({ color, alpha: 0.86 });
+ } else if (node.type === 'delay') {
- this.nodeLayer.roundRect(node.x - node.radius * 0.74, node.y - node.radius * 1.48, node.radius * 1.48, 12).fill({ color, alpha: 0.88 });
+ for (let tick = 0; tick <= node.delayLevel; tick += 1) {
+ this.nodeLayer.roundRect(node.x - 22 + tick * 22, node.y + node.radius + 16, 13, 7, 3).fill({ color: 0xffffffff, alpha: 0.78 });
+ }
+ } else {
+ this.nodeLayer.circle(node.x, node.y, node.radius).fill({ color, alpha: 0.88 });
+ }
+ this.nodeLayer.circle(node.x, node.y, node.radius * 0.45).fill({ color: 0xffffffff, alpha: active ? 0.82 : 0.36 });
+ const text = this.nodeText(node.id);
+ text.text = iconForNode(node);
+ text.position.set(node.x, node.y - 2);
+ text.visible = true;
+ visibleNodeIds.add(node.id);
+ if (snapshot.tutorialActive && highlighted) {
+ const label = this.nodeText(node.id + 1000);
+ label.text = node.type === 'energy' ? 'ENERGY' : node.label;
+ label.position.set(node.x, node.y + node.radius + 46);
+ label.visible = true;
+ visibleNodeIds.add(node.id + 1000);
+ }
+ }
+ for (const [id, text] of this.nodeTexts) {
+ if (!visibleNodeIds.has(id)) {
+ text.visible = false;
+ }
+ }
+ }
}
@@ -239,6 +318,19 @@ export class PulseRenderer {
 this.scoreText.text = String(snapshot.score);
 this.metaText.text = `x${snapshot.multiplier} LINKS ${snapshot.linksUsed}/${snapshot.linkBudget} LENS ${snapshot.lensCharges}/2 BEST ${Math.max(snapshot.score,
Number(localStorage.getItem('eventHorizon.bestScore') ?? 0))}`;
+ this.strategyText.visible = snapshot.phase === 'build';
+ this.strategyText.text = [
+ 'BUILD A CHAIN',
+ 'Hit Energy nodes,',
+ 'Use Delay nodes,',
+ 'Avoid dead ends.',
+ `Energy ${snapshot.chainAnalysis.reachableEnergyNodes}/${snapshot.chainAnalysis.totalEnergyNodes}`,
+ `Dead ends ${snapshot.chainAnalysis.deadEndNodeIds.length}`,
+ `Loop ${snapshot.chainAnalysis.hasLoop ? 'Yes' : 'No'}`,
+ snapshot.chainAnalysis.quality
+].join('\n');
 this.hintText.text = snapshot.tutorialHint;
 this.hintText.visible = snapshot.phase !== 'ended';
+ this.messageText.text = snapshot.lastInputResult.message;
+ this.messageText.visible = snapshot.phase !== 'ended' && snapshot.lastInputResult.message !== snapshot.tutorialHint;
 this.meter.clear();
 const meterWidth = WORLD_WIDTH - 156;
@@ -250,13 +342,21 @@ export class PulseRenderer {
 this.meter.roundRect(0, 0, 0, 0);
 this.endText.visible = snapshot.phase === 'ended';
- this.endText.text = snapshot.phase === 'ended'
- ? `${snapshot.stabilized ? 'SECTOR STABILIZED' : 'GALAXY COLLAPSED'}\n${snapshot.score} * ${formatTime(snapshot.timeMs)}\nSEED ${snapshot.seed}`
- : '';
+ if (snapshot.phase === 'ended') {
+ const fix = snapshot.suggestedFixes[0];
+ this.endText.text = snapshot.endReason === 'pulse-died'
+ ? `PULSE STAYED dead end at: ${nodeLabel(snapshot, snapshot.deadEndNodeId)}\n${fix ? `Try Linking this node\n${nodeLabel(snapshot, fix.toId)}.` : `Try adding one more outgoing link.`}`
+ : `${snapshot.stabilized ? 'SECTOR STABILIZED' : 'GALAXY COLLAPSED'}\n${snapshot.score} * ${formatTime(snapshot.timeMs)}\nEnergy nodes hit
+${snapshot.chainAnalysis.reachableEnergyNodes}/${snapshot.chainAnalysis.totalEnergyNodes}\nSEED ${snapshot.seed}`;
+ } else {
+ this.endText.text = '';
+ }
+ }
 this.debugText.visible = debug;
 if (debug) {
 this.debugText.text = [
 `phase: ${snapshot.phase}`,
 `tutorial: ${snapshot.tutorialStep}`,
 `selected: ${input.selectedNodeId ?? '—'}`,
 `nearest: ${input.nearestNodeId ?? '—'}`,
 `links: ${snapshot.linksUsed}/${snapshot.linkBudget} pulses: ${snapshot.pulses.length}`,
 `chain: ${snapshot.lastChainNodeIds.join('>') || '—'}`,
 `analysis: ${snapshot.chainAnalysis.quality}`,
 `last: ${snapshot.lastInputResult.message}`,
 `hash: ${snapshot.stepHash}`
];
 }
@@ -279,4 +379,25 @@ export class PulseRenderer {
}

+ private nodeText(id: number): Text {
+ let text = this.nodeTexts.get(id);
+ if (!text) {
+ text = new Text({
+ text: '',
+ style: new TextStyle({
+ align: 'center',
+ fill: '#061120',
+ fontFamily: 'Inter, system-ui, sans-serif',
+ fontSize: id >= 1000 ? 18 : 26,
+ fontWeight: '900',
+ stroke: { color: 'ffffff', width: id >= 1000 ? 2 : 3 }
+ })
+ });
+ text.anchor.set(0.5);
+ this.nodeTextLayer.addChild(text);
+ this.nodeTexts.set(id, text);
+ }
+ return text;
+ }

 private drawFlowDots(
 graphics: Graphics,
@@ -326,2 +447,56 @@ function pulsePoint(snapshot: PulseSnapshot, fromId: number, toId: number | undefined
 return pointOnQuadratic(path[0], path[1], path[2], progress);
 }

+function iconForNode(node: PulseNode): string {
+ if (node.type === 'source') {
+ return 'S';
+ }
+ if (node.type === 'energy') {
+ return '+';
+ }
+ if (node.type === 'delay') {
+ return 'II';
+ }
+ if (node.type === 'splitter') {

```



```

+ return 'Y';
+ }
+ return 'o';
+}
+function pointAlongPolyline(points: readonly WorldPoint[], t: number): WorldPoint {
+ if (points.length === 0) {
+ return { x: 0, y: 0 };
+ }
+ if (points.length === 1) {
+ return points[0];
+ }
+ const lengths: number[] = [];
+ let total = 0;
+ for (let index = 1; index < points.length; index += 1) {
+ const length = Math.hypot(points[index].x - points[index - 1].x, points[index].y - points[index - 1].y);
+ lengths.push(length);
+ total += length;
+ }
+ let target = clamp(t, 0, 1) * total;
+ for (let index = 1; index < points.length; index += 1) {
+ const length = lengths[index - 1];
+ if (target <= length) {
+ const local = length <= 0 ? 0 : target / length;
+ return {
+ x: points[index - 1].x + (points[index].x - points[index - 1].x) * local,
+ y: points[index - 1].y + (points[index].y - points[index - 1].y) * local
+ };
+ }
+ target -= length;
+ }
+ return points[points.length - 1];
+}
+
+function nodeLabel(snapshot: PulseSnapshot, nodeId: number | undefined): string {
+ const node = nodeId === undefined ? undefined : findNode(snapshot.nodes, nodeId);
+ if (!node) {
+ return 'UNKNOWN NODE';
+ }
+ const label = node.type === 'energy' ? 'ENERGY' : node.label;
+ return `${label} ${node.id}`;
+}
+
+diff --git a/src/game/pulse/PulseSimulation.ts b/src/game/pulse/PulseSimulation.ts
index bf9725f..f69a0dd 100644
--- a/src/game/pulse/PulseSimulation.ts
+++ b/src/game/pulse/PulseSimulation.ts
@@ -2,10 +2,12 @@ import { INITIAL_ENERGY, MAX_ENERGY } from '../constants';
import { clamp } from '../math';
import { quantizeGesturePath, type GesturePathPoint, type WorldPoint } from '../gestures';
-import { nearestTwoNodesToPath } from './PulseGeometry';
+import { distancePointToSegment, nearestTwoNodesToPath, nodesCrossedByPath } from './PulseGeometry';
import { generatePulseLevel } from './PulseLevelGenerator';
import type {
 BuildInput,
 ChainAnalysis,
 HorizonLens,
 LiveInput,
 NodeTapAction,
 PulseGamePhase,
 PulseInputResult,
@@ -16,15 +16,21 @@ import type {
 PulseResult,
 PulseSnapshot,
 PulseState,
+ SuggestedFix,
+ TutorialStep
} from './PulseTypes';

-const PULSE_SPEED = 460;
+const NORMAL_PULSE_SPEED = 310;
+const TUTORIAL_PULSE_SPEED = 220;
const LINK_TRAVERSAL_SCORE = 10;
const ENERGY_NODE_SCORE = 100;
const DELAY_NODE_SCORE = 25;
const SPLITTER_NODE_SCORE = 75;
-const LENS_DURATION_MS = 1200;
+const LENS_DURATION_MS = 1500;
+const TUTORIAL_LENS_DURATION_MS = 2100;
const LENS_MAX_CHARGES = 2;
-const ENERGY_DRAIN_PER_SECOND = 0.52;
+const ENERGY_DRAIN_PER_SECOND = 0.38;
+const STABILIZE_SCORE = 50;
+const DELAY_MS = [360, 700, 1080] as const;

export interface PulseSimulationOptions {
@@ -34,6 +34,6 @@ export interface PulseSimulationOptions {
export class PulseSimulation {
- readonly seed: string;
+ readonly startedAt: number;
+ private seedValue: string;
+ private level: PulseLevel;
+ private phase: PulseGamePhase = 'build';
@@ -49,5 +49,5 @@ export class PulseSimulation {
 private nextLensId = 1;
 private endReason: PulseSnapshot['endReason'];
+ private deadEndNodeId: number | undefined;
+ private collapsed = false;
+ private stabilized = false;
@@ -60,13 +60,13 @@ export class PulseSimulation {
 private stepHash = '00000000';
 private lensCharges = LENS_MAX_CHARGES;
+ private tutorialActive = false;
+ private tutorialStep: TutorialStep = 'skipped';
+ private lastChainNodeIds: number[] = [];
+ private lastTapAction: NodeTapAction = 'none';
+ private energyNodesHit = new Set<number>();
+ private slowDrainMs = 0;

 constructor(options: PulseSimulationOptions) {
- this.seed = options.seed;
+ this.seedValue = options.seed;
+ this.startedAt = options.startedAt;
+ this.level = generatePulseLevel(options.seed);
+ if (options.seed === 'tutorial-001') {
+ this.startTutorial();
+ }
+ this.updateHash();
 }

+ get seed(): string {
+ return this.seedValue;
+ }

+ reset(seed = this.seed): void {
+ this.seedValue = seed;
+ this.level = generatePulseLevel(seed);
+ this.phase = 'build';
@@ -82,5 +82,5 @@ export class PulseSimulation {
 this.nextLensId = 1;
 this.endReason = undefined;
 this.deadEndNodeId = undefined;

```

```

 this.collapsed = false;
 this.stabilized = false;
@@ -91,5 +115,11 @@ export class PulseSimulation {
 this.selectedNodeId = undefined;
 this.lensCharges = LENS_MAX_CHARGES;
 this.lastInputResult = { ok: true, kind: 'none', message: 'CONNECT TWO NODES' };
+ this.lastChainNodeIds = [];
+ this.lastTapAction = 'none';
+ this.energyNodesHit = new Set();
+ this.slowDrainMs = 0;
+ this.tutorialActive = seed === 'tutorial-001';
+ this.tutorialStep = this.tutorialActive ? 'swipe-chain' : 'skipped';
+ this.lastInputResult = { ok: true, kind: 'none', message: this.tutorialActive ? 'SWIPE THROUGH THESE NODES' : 'DRAW A CHAIN' };
 this.updateHash();
 }
@@ -107,5 +137,8 @@ export class PulseSimulation {
 if (this.phase === 'pulse') {
 this.darkEnergy = clamp(this.darkEnergy - (dtMs / 1000) * ENERGY_DRAIN_PER_SECOND, 0, MAX_ENERGY);
+ const tutorialGrace = this.tutorialActive && this.timeMs < 10000;
+ const drainScale = this.slowDrainMs > 0 || tutorialGrace ? 0.35 : 1;
+ this.darkEnergy = clamp(this.darkEnergy - (dtMs / 1000) * ENERGY_DRAIN_PER_SECOND * drainScale, 0, MAX_ENERGY);
+ this.slowDrainMs = Math.max(0, this.slowDrainMs - dtMs);
 this.updatePulses(dtMs);
 this.expireTemporaryLinks(dtMs);
@@ -146,5 +179,6 @@ export class PulseSimulation {
 temporary: false,
 ageMs: 0,
 expiresMs: 0
+ expiresMs: 0,
+ flashMs: 420
 };
 this.nextLinkId += 1;
@@ -159,4 +193,170 @@ export class PulseSimulation {
}

+ applyChainSwipe(points: readonly GesturePathPoint[], record = true): PulseInputResult {
+ if (this.phase !== 'build') {
+ this.lastInputResult = { ok: false, kind: 'invalid', message: 'CHAIN ONLY DURING BUILD' };
+ return this.lastInputResult;
+ }
+ const path = quantizeGesturePath(points, 24);
+ const crossed = nodesCrossedByPath(this.level.nodes, path, 78);
+ const nodeIds = crossed.map((node) => node.id);
+ this.lastChainNodeIds = nodeIds;
+ if (nodeIds.length < 2) {
+ this.lastInputResult = { ok: false, kind: 'chainSwipe', message: 'NO NODES CROSSED', nodeIds };
+ this.advanceTutorial('chain-miss');
+ return this.lastInputResult;
+ }
+
+ let created = 0;
+ const createdNodes: number[] = [nodeIds[0]];
+ for (let index = 1; index < nodeIds.length; index += 1) {
+ const fromId = nodeIds[index - 1];
+ const toId = nodeIds[index];
+ if (fromId === toId) {
+ continue;
+ }
+ const result = this.addLink(fromId, toId, false);
+ if (!result.ok) {
+ if (result.message === 'LINK EXISTS') {
+ createdNodes.push(toId);
+ continue;
+ }
+ if (result.message === 'LINK LIMIT') {
+ break;
+ }
+ continue;
+ }
+ created += 1;
+ createdNodes.push(toId);
+ }
+
+ if (created > 0 && record) {
+ this.buildInputs.push({ t: Math.round(this.timeMs), kind: 'chainSwipe', nodeIds: createdNodes, path });
+ }
+ this.selectedNodeId = undefined;
+ this.lastInputResult = created > 0
+ ? { ok: true, kind: 'chainSwipe', message: `CHAIN CREATED ${created} LINKS`, nodeIds: createdNodes }
+ : { ok: false, kind: 'chainSwipe', message: 'CHAIN BLOCKED', nodeIds };
+ this.advanceTutorial('chain');
+ this.updateHash();
+ return this.lastInputResult;
+ }
+
+ tapNode(nodeId: number, record = true): PulseInputResult {
+ const node = this.nodeById(nodeId);
+ if (!node) {
+ this.lastInputResult = { ok: false, kind: 'invalid', message: 'NO NODE', nodeId };
+ return this.lastInputResult;
+ }
+ if (this.phase === 'pulse') {
+ return this.stabilizeNode(nodeId, record);
+ }
+
+ let action: NodeTapAction = 'select';
+ if (node.type === 'energy') {
+ node.primed = !node.primed;
+ node.activationMs = 520;
+ action = 'prime';
+ this.lastInputResult = { ok: true, kind: 'nodeTap', message: node.primed ? 'ENERGY PRIMED' : 'ENERGY UNPRIMED', nodeId };
+ } else if (node.type === 'delay') {
+ node.delayLevel = ((node.delayLevel + 1) % 3) as 0 | 1 | 2;
+ node.activationMs = 520;
+ action = 'delay';
+ this.lastInputResult = { ok: true, kind: 'nodeTap', message: `DELAY ${node.delayLevel + 1}`, nodeId };
+ } else if (node.type === 'splitter') {
+ node.splitterPriority = (node.splitterPriority + 1) % 3;
+ node.activationMs = 520;
+ action = 'splitter';
+ this.lastInputResult = { ok: true, kind: 'nodeTap', message: 'SPLITTER AIMED', nodeId };
+ this.advanceTutorial('splitter');
+ } else {
+ this.selectedNodeId = nodeId;
+ this.lastInputResult = { ok: true, kind: 'select', message: `SELECTED ${node.label}`, nodeId };
+ }
+ this.lastTapAction = action;
+ if (record) {
+ this.buildInputs.push({ t: Math.round(this.timeMs), kind: 'nodeTap', nodeId, action });
+ }
+ this.updateHash();
+ return this.lastInputResult;
+ }
+
+ stabilizeNode(nodeId: number, record = true): PulseInputResult {
+ const node = this.nodeById(nodeId);
+ if (!node) {
+ this.lastInputResult = { ok: false, kind: 'invalid', message: 'NO NODE', nodeId };
+ return this.lastInputResult;
+ }
+ const arrival = this.arrivalReadiness(nodeId);
+ let rating: 'perfect' | 'stabilized' | 'early' | 'late' = 'early';

```

```

+ let success = false;
+ if (arrival === 'late') {
+ rating = 'late';
+ } else if (arrival === 'perfect') {
+ rating = 'perfect';
+ success = true;
+ } else if (arrival === 'soon') {
+ rating = 'stabilized';
+ success = true;
+ }
+
+ if (success) {
+ node.stabilizedMs = 760;
+ node.activationMs = 620;
+ this.addScore(rating === 'perfect' ? STABILIZE_SCORE + 25 : STABILIZE_SCORE, 2.2);
+ this.slowDrainMs = 1500;
+ this.lastInputResult = {
+ ok: true,
+ kind: 'stabilize',
+ message: rating === 'perfect' ? 'PERFECT TAP +75' : 'STABILIZED +50',
+ nodeId,
+ scoreDelta: rating === 'perfect' ? 75 : 50,
+ energyDelta: 2
+ };
+ } else {
+ this.lastInputResult = { ok: false, kind: 'stabilize', message: rating === 'late' ? 'LATE' : 'EARLY', nodeId };
+ }
+ this.lastTapAction = 'stabilize';
+ if (record) {
+ this.liveInputs.push({ t: Math.round(this.timeMs), kind: 'stabilize', nodeId, rating, success });
+ }
+ this.advanceTutorial('stabilize');
+ this.updateHash();
+ return this.lastInputResult;
+ }
+
+ removeLinkNear(point: WorldPoint): PulseInputResult {
+ if (this.phase !== 'build') {
+ this.lastInputResult = { ok: false, kind: 'invalid', message: 'LINKS LOCKED' };
+ return this.lastInputResult;
+ }
+ let bestIndex = -1;
+ let bestDistance = 54;
+ for (let index = 0; index < this.links.length; index += 1) {
+ const link = this.links[index];
+ if (link.temporary) {
+ continue;
+ }
+ const from = this.nodeById(link.fromId);
+ const to = this.nodeById(link.toId);
+ if (!from || !to) {
+ continue;
+ }
+ const distance = distancePointToSegment(point, from, to);
+ if (distance < bestDistance) {
+ bestDistance = distance;
+ bestIndex = index;
+ }
+ }
+ if (bestIndex === -1) {
+ this.lastInputResult = { ok: false, kind: 'invalid', message: 'NO LINK' };
+ return this.lastInputResult;
+ }
+ this.links.splice(bestIndex, 1);
+ this.lastInputResult = { ok: true, kind: 'clear', message: 'LINK REMOVED' };
+ this.updateHash();
+ return this.lastInputResult;
+ }
+
+ undo(): PulseInputResult {
+ const index = [...this.links].map((link, linkIndex) => ({ link, linkIndex })).reverse().find(entry => !entry.link.temporary);
+ @@ -198,4 +398,5 @@ export class PulseSimulation {
+ }
+ this.lastInputResult = { ok: true, kind: 'play', message: 'STABILIZING PULSE' };
+ this.advanceTutorial('play');
+ this.updateHash();
+ return this.lastInputResult;
+ @@ -218,5 +419,5 @@ export class PulseSimulation {
+ this.createLens(lensPath, false);
+ this.liveInputs.push({ t: Math.round(this.timeMs), kind: 'lens', path, success: false });
+ - this.lastInputResult = { ok: false, kind: 'lens', message: 'NO ANCHOR' };
+ + this.lastInputResult = { ok: false, kind: 'lens', message: 'NO ANCHOR - SWIPE BETWEEN TWO NODES' };
+ return this.lastInputResult;
+ }
+ @@ -227,5 +428,5 @@ export class PulseSimulation {
+ this.createLens(lensPath, false);
+ this.liveInputs.push({ t: Math.round(this.timeMs), kind: 'lens', path, fromId: from.id, toId: to.id, success: false });
+ - this.lastInputResult = { ok: false, kind: 'lens', message: 'NO ANCHOR', fromId: from.id, toId: to.id };
+ + this.lastInputResult = { ok: false, kind: 'lens', message: 'NO ANCHOR - SWIPE BETWEEN TWO NODES', fromId: from.id, toId: to.id };
+ return this.lastInputResult;
+ }
+ @@ -237,5 +438,6 @@ export class PulseSimulation {
+ temporary: true,
+ ageMs: 0,
+ expiresMs: LENS_DURATION_MS
+ + expiresMs: this.tutorialActive ? TUTORIAL_LENS_DURATION_MS : LENS_DURATION_MS,
+ flashMs: 520
+ });
+ this.nextLinkId += 1;
+ @@ -243,5 +445,6 @@ export class PulseSimulation {
+ this.createLens(lensPath, true, from.id, to.id);
+ this.liveInputs.push({ t: Math.round(this.timeMs), kind: 'lens', path, fromId: from.id, toId: to.id, success: true });
+ - this.lastInputResult = { ok: true, kind: 'lens', message: 'BRIDGE CREATED', fromId: from.id, toId: to.id };
+ + this.lastInputResult = { ok: true, kind: 'lens', message: 'HORIZON LENS - BRIDGE CREATED', fromId: from.id, toId: to.id };
+ this.advanceTutorial('lens');
+ this.updateHash();
+ return this.lastInputResult;
+ @@ -268,4 +471,82 @@ export class PulseSimulation {
+ }
+
+ startTutorial(): void {
+ this.seedValue = 'tutorial-001';
+ this.level = generatePulseLevel(this.seedValue);
+ this.tutorialActive = true;
+ this.tutorialStep = 'swipe-chain';
+ this.phase = 'build';
+ this.timeMs = 0;
+ this.score = 0;
+ this.multiplier = 1;
+ this.maxMultiplier = 1;
+ this.chainLength = 0;
+ this.loopsCompleted = 0;
+ this.nextLinkId = 1;
+ this.nextPulseId = 1;
+ this.nextLensId = 1;
+ this.buildInputs = [];
+ this.liveInputs = [];
+ this.links = [];
+ this.pulses = [];
+ this.lenses = [];
+ this.endReason = undefined;
+ this.deadEndNodeId = undefined;
+ this.collapsed = false;

```

```

+ this.stabilized = false;
+ this.selectedNodeId = undefined;
+ this.darkEnergy = INITIAL_ENERGY;
+ this.lensCharges = LENS_MAX_CHARGES;
+ this.lastChainNodeIds = [];
+ this.lastTapAction = 'none';
+ this.energyNodesHit = new Set();
+ this.slowDrainMs = 0;
+ this.lastInputResult = { ok: true, kind: 'none', message: 'SWIPE THROUGH THESE NODES', nodeIds: [1, 2, 3] };
+ this.updateHash();
+ }
+
+ skipTutorial(): void {
+ this.tutorialActive = false;
+ this.tutorialStep = 'skipped';
+ this.lastInputResult = { ok: true, kind: 'none', message: 'DRAW A CHAIN' };
+ this.updateHash();
+ }
+
+ getTutorialStep(): TutorialStep {
+ return this.tutorialStep;
+ }
+
+ analyzeChain(): ChainAnalysis {
+ return this.computeChainAnalysis();
+ }
+
+ getSuggestedFixes(): readonly SuggestedFix[] {
+ return this.computeSuggestedFixes();
+ }
+
+ primeNode(id: number): PulseInputResult {
+ const node = this.nodeById(id);
+ if (!node || node.type !== 'energy') {
+ this.lastInputResult = { ok: false, kind: 'invalid', message: 'NOT ENERGY', nodeId: id };
+ return this.lastInputResult;
+ }
+ node.primed = true;
+ node.activationMs = 520;
+ this.lastTapAction = 'prime';
+ this.lastInputResult = { ok: true, kind: 'nodeTap', message: 'ENERGY PRIMED', nodeId: id };
+ this.updateHash();
+ return this.lastInputResult;
+ }
+
+ cycleNode(id: number): PulseInputResult {
+ return this.tapNode(id);
+ }
+
+ fixChain(): PulseInputResult {
+ this.forceBuildPhase();
+ this.lastInputResult = { ok: true, kind: 'fix', message: 'FIX THE DEAD END', nodeId: this.deadEndNodeId };
+ return this.lastInputResult;
+ }
+
+ getNodes(): readonly PulseNode[] {
+ return this.level.nodes;
+ }
+
+ @@ -290,9 +571,14 @@ export class PulseSimulation {
+ seed: this.seed,
+ startedAt: this.startedAt,
+ buildInputs: this.buildInputs.map((input) => ({ ...input })),
+ liveInputs: this.liveInputs.map((input) => ({
+ ...input,
+ path: input.path.map((point) => ({ ...point })))
+ })),
+ buildInputs: this.buildInputs.map((input) =>
+ input.kind === 'chainSwipe'
+ ? { ...input, nodeIds: [...input.nodeIds], path: input.path.map((point) => ({ ...point }))) }
+ : { ...input }
+),
+ liveInputs: this.liveInputs.map((input) =>
+ input.kind === 'lens'
+ ? { ...input, path: input.path.map((point) => ({ ...point }))) }
+ : { ...input }
+),
+ result: this.getResult(),
+ stepHash: this.stepHash
+ }
+
+ @@ -328,4 +614,13 @@ export class PulseSimulation {
+ selectedNodeId: this.selectedNodeId,
+ tutorialHint: this.tutorialHint(),
+ tutorialActive: this.tutorialActive,
+ tutorialStep: this.tutorialStep,
+ tutorialHighlightNodeIds: this.tutorialHighlightNodeIds(),
+ tutorialGhostPath: this.tutorialGhostPath(),
+ chainAnalysis: this.computeChainAnalysis(),
+ suggestedFixes: this.computeSuggestedFixes(),
+ lastChainNodeIds: this.lastChainNodeIds,
+ lastTapAction: this.lastTapAction,
+ deadEndNodeId: this.deadEndNodeId,
+ lastInputResult: this.lastInputResult,
+ stepHash: this.stepHash
+ }
+
+ @@ -419,10 +714,13 @@ export class PulseSimulation {
+ if (node.type === 'energy') {
+ const fresh = node.scoreCooldownMs <= 0;
+ this.addScore(fresh ? ENERGY_NODE_SCORE : 25, fresh ? 6.2 : 1.4);
+ const primedBonus = node.primed ? 80 : 0;
+ this.addScore(fresh ? ENERGY_NODE_SCORE : 25) + primedBonus, fresh ? 6.2 + primedBonus / 40 : 1.4);
+ node.primed = false;
+ node.scoreCooldownMs = fresh ? 2600 : node.scoreCooldownMs;
+ this.energyNodesHit.add(node.id);
+ this.lensCharges = Math.min(LENS_MAX_CHARGES, this.lensCharges + 1);
+ } else if (node.type === 'delay') {
+ this.addScore(DELAY_NODE_SCORE, 1.1);
+ pulse.delayMs = 620;
+ pulse.delayMs = DELAY_MS[node.delayLevel];
+ return;
+ } else if (node.type === 'splitter') {
+ @@ -440,6 +738,11 @@ export class PulseSimulation {
+ }
+
+ if (this.nodeById(pulse.currentNodeId)?.type === 'splitter' && outgoing.length > 1) {
+ for (const link of outgoing) {
+ const currentNode = this.nodeById(pulse.currentNodeId);
+ const orderedOutgoing = currentNode?.type === 'splitter'
+ ? [...outgoing].sort((a, b) => splitterOrder(a, b, currentNode.splitterPriority))
+ : outgoing;
+
+ if (currentNode?.type === 'splitter' && orderedOutgoing.length > 1) {
+ for (const link of orderedOutgoing) {
+ this.spawnPulse(pulse.currentNodeId, link.toId, pulse.previousNodeId, pulse.comboChainLength, pulse.visitedNodeIds);
+ }
+ }
+ }
+
+ @@ -448,5 +751,5 @@ export class PulseSimulation {
+
+ const preferred = outgoing.find((link) => link.toId !== pulse.previousNodeId) ?? outgoing[0];
+ const preferred = orderedOutgoing.find((link) => link.toId !== pulse.previousNodeId) ?? orderedOutgoing[0];
+ pulse.nextNodeId = preferred.toId;
+ pulse.progress = 0;
+ }
+
+ @@ -476,5 +779,5 @@ export class PulseSimulation {
+ nextNodeId,
+ progress: 0,
+ speed: PULSE_SPEED,
+ speed: this.tutorialActive ? TUTORIAL_PULSE_SPEED : NORMAL_PULSE_SPEED,

```

```

 ageMs: 0,
 energy: 1,
@@ -489,4 +792,5 @@ export class PulseSimulation {
 private killPulse(pulse: PulseState): void {
 pulse.alive = false;
+ this.deadEndNodeId = pulse.currentNodeId;
+ this.darkEnergy = clamp(this.darkEnergy - 5.6, 0, MAX_ENERGY);
+ this.multiplier = 1;
@@ -511,4 +815,8 @@ export class PulseSimulation {
 node.activationMs = Math.max(0, node.activationMs - dtMs);
 node.scoreCooldownMs = Math.max(0, node.scoreCooldownMs - dtMs);
+ node.stabilizedMs = Math.max(0, node.stabilizedMs - dtMs);
+ }
+ for (const link of this.links) {
+ link.flashMs = Math.max(0, link.flashMs - dtMs);
+ }
+ for (let index = this.lenses.length - 1; index >= 0; index --= 1) {
@@ -570,4 +878,6 @@ export class PulseSimulation {
 loopsCompleted: this.loopsCompleted,
 linksUsed: this.links.filter((link) => !link.temporary).length,
 bestChainLength: this.chainLength,
+ energyNodesHit: this.energyNodesHit.size,
+ stabilized: this.stabilized,
+ collapsed: this.collapsed
@@ -576,7 +886,27 @@ export class PulseSimulation {
 private tutorialHint(): string {
+ if (this.tutorialActive) {
+ if (this.tutorialStep === 'swipe-chain') {
+ return 'SWIPE THROUGH THESE NODES';
+ }
+ if (this.tutorialStep === 'tap-splitter') {
+ return 'TAP THE SPLITTER TO AIM IT';
+ }
+ if (this.tutorialStep === 'press-play') {
+ return 'PRESS PLAY';
+ }
+ if (this.tutorialStep === 'stabilize') {
+ return 'TAP THE NEXT NODE TO STABILIZE';
+ }
+ if (this.tutorialStep === 'lens') {
+ return 'SWIPE BETWEEN NODES TO CREATE A HORIZON LENS';
+ }
+ if (this.tutorialStep === 'loops') {
+ return 'BUILD LOOPS TO DELAY COLLAPSE';
+ }
+ }
+ if (this.phase === 'build') {
+ if (this.links.filter((link) => !link.temporary).length === 0) {
- return 'CONNECT TWO NODES';
+ return 'SWIPE THROUGH NODES TO DRAW A CHAIN';
+ }
+ return 'PRESS PLAY';
@@ -588,4 +918,150 @@ export class PulseSimulation {
 }

+ private advanceTutorial(event: 'chain' | 'chain-miss' | 'splitter' | 'play' | 'stabilize' | 'lens'): void {
+ if (!this.tutorialActive) {
+ return;
+ }
+ if (this.tutorialStep === 'swipe-chain' && event === 'chain' && this.links.some((link) => link.fromId === 1 && link.toId === 2) && this.links.some((link) => link.fromId === 2 && link.toId
=== 3)) {
+ this.tutorialStep = 'tap-splitter';
+ this.lastInputResult = { ...this.lastInputResult, message: 'CHAIN CREATED. TAP THE SPLITTER' };
+ } else if (this.tutorialStep === 'tap-splitter' && event === 'splitter') {
+ this.tutorialStep = 'press-play';
+ } else if (this.tutorialStep === 'press-play' && event === 'play') {
+ this.tutorialStep = 'stabilize';
+ } else if (this.tutorialStep === 'stabilize' && event === 'stabilize') {
+ this.tutorialStep = 'lens';
+ } else if (this.tutorialStep === 'lens' && event === 'lens') {
+ this.tutorialStep = 'loops';
+ }
+ }
+
+ private tutorialHighlightNodeIds(): number[] {
+ if (!this.tutorialActive) {
+ return [];
+ }
+ if (this.tutorialStep === 'swipe-chain') {
+ return [1, 2, 3];
+ }
+ if (this.tutorialStep === 'tap-splitter') {
+ return [4];
+ }
+ if (this.tutorialStep === 'stabilize') {
+ return [3, 4];
+ }
+ if (this.tutorialStep === 'lens') {
+ return [3, 4];
+ }
+ return [];
+ }
+
+ private tutorialGhostPath(): WorldPoint[] {
+ const ids = this.tutorialHighlightNodeIds();
+ if (ids.length < 2) {
+ return [];
+ }
+ return ids
+ .map((id) => this.nodeById(id))
+ .filter((node): node is PulseNode => node !== undefined)
+ .map((node) => ({ x: node.x, y: node.y }));
+ }
+
+ private computeChainAnalysis(): ChainAnalysis {
+ const totalEnergyNodes = this.level.nodes.filter((node) => node.type === 'energy').length;
+ const reachable = this.reachableFromSource();
+ const reachableEnergyNodes = this.level.nodes.filter((node) => node.type === 'energy' && reachable.has(node.id)).length;
+ const deadEndNodeIds = [...reachable].filter((nodeId) => nodeId !== this.level.sourceId && this.outgoingLinks(nodeId).filter((link) => !link.temporary).length === 0);
+ const hasLoop = this.hasReachableLoop();
+ const linksUsed = this.links.filter((link) => !link.temporary).length;
+ let quality: ChainAnalysis['quality'] = 'Draw a chain';
+ if (linksUsed > 0) {
+ quality = 'Good start';
+ }
+ if (deadEndNodeIds.length > 0) {
+ quality = 'Dead end detected';
+ } else if (hasLoop && reachableEnergyNodes >= 2) {
+ quality = 'Great loop';
+ } else if (hasLoop) {
+ quality = 'Loop possible';
+ } else if (reachableEnergyNodes < Math.min(2, totalEnergyNodes)) {
+ quality = 'Hit more Energy nodes';
+ }
+ return { reachableEnergyNodes, totalEnergyNodes, deadEndNodeIds, hasLoop, linksUsed, quality };
+ }
+
+ private computeSuggestedFixes(): SuggestedFix[] {
+ const analysis = this.computeChainAnalysis();
+ const fromId = this.deadEndNodeId ?? analysis.deadEndNodeIds[0];
+ const from = fromId ? this.nodeById(fromId) : undefined;
+ if (!from) {

```

```

+ return [];
+ }
+ return this.level.nodes
+ .filter((node) => node.id !== from.id && !this.links.some((link) => link.fromId === from.id && link.toId === node.id))
+ .sort((a, b) => Math.hypot(a.x - from.x, a.y - from.y) - Math.hypot(b.x - from.x, b.y - from.y))
+ .slice(0, 2)
+ .map((node) => ({ fromId: from.id, toId: node.id, message: `Try linking ${from.label} to ${node.label}` }));
+ }
+
+ private reachableFromSource(): Set<number> {
+ const seen = new Set<number>();
+ const queue = [this.level.sourceId];
+ while (queue.length > 0) {
+ const nodeId = queue.shift();
+ if (nodeId === undefined || !seen.has(nodeId)) {
+ continue;
+ }
+ seen.add(nodeId);
+ for (const link of this.outgoingLinks(nodeId).filter((candidate) => !candidate.temporary)) {
+ queue.push(link.toId);
+ }
+ }
+ return seen;
+ }
+
+ private hasReachableLoop(): boolean {
+ const reachable = this.reachableFromSource();
+ const visiting = new Set<number>();
+ const visited = new Set<number>();
+ const visit = (nodeId: number): boolean => {
+ if (!reachable.has(nodeId)) {
+ return false;
+ }
+ if (visiting.has(nodeId)) {
+ return true;
+ }
+ if (visited.has(nodeId)) {
+ return false;
+ }
+ visiting.add(nodeId);
+ for (const link of this.outgoingLinks(nodeId).filter((candidate) => !candidate.temporary)) {
+ if (visit(link.toId)) {
+ return true;
+ }
+ }
+ visiting.delete(nodeId);
+ visited.add(nodeId);
+ return false;
+ };
+ return visit(this.level.sourceId);
+ }
+
+ private arrivalReadiness(nodeId: number): 'perfect' | 'soon' | 'early' | 'late' {
+ for (const pulse of this.pulses) {
+ if (pulse.nextNodeId === nodeId) {
+ if (pulse.progress >= 0.78) {
+ return 'perfect';
+ }
+ if (pulse.progress >= 0.42) {
+ return 'soon';
+ }
+ return 'early';
+ }
+ if (pulse.currentNodeId === nodeId) {
+ return 'late';
+ }
+ }
+ return 'early';
+ }
+
+ private updateHash(): void {
+ const data = [
@@ -595,6 +1071,8 @@ export class PulseSimulation {
 Math.round(this.darkEnergy * 10),
 this.multiplier,
 this.tutorialStep,
+ this.level.nodes.map((node) => `${node.id}:${node.primed ? 1 : 0}:${node.delayLevel}:${node.splitterPriority}`).join('|'),
+ this.links.map((link) => `${link.fromId}:${link.toId}:${link.temporary ? Math.round(link.expiresMs - link.ageMs) : 0}`).join('|'),
- this.pulses.map((pulse) => `${pulse.currentNodeId}>${pulse.nextNodeId ?? 0}:${Math.round(pulse.progress * 1000)}:${pulse.delayMs}`).join('|')
+ this.pulses.map((pulse) => `${pulse.currentNodeId}>${pulse.nextNodeId ?? 0}:${Math.round(pulse.progress * 1000)}:${Math.round(pulse.delayMs)}').join('|')
].join(';');
 let hash = 2166136261;
@@ -622,2 +1100,8 @@ function multiplierForChain(chainLength: number): number {
 return 1;
}

+function splitterOrder(a: PulseLink, b: PulseLink, priority: number): number {
+ const aValue = (a.toId + priority * 7) % 13;
+ const bValue = (b.toId + priority * 7) % 13;
+ return aValue - bValue;
+}
+}
diff --git a/src/game/pulse/PulseTypes.ts b/src/game/pulse/PulseTypes.ts
index 02d143d..e097207 100644
--- a/src/game/pulse/PulseTypes.ts
+++ b/src/game/pulse/PulseTypes.ts
@@ -4,4 +4,6 @@ export type PulseGamePhase = 'build' | 'pulse' | 'ended';
 export type PulseEndReason = 'collapsed' | 'stabilized' | 'pulse-died' | 'manual';
 export type PulseNodeType = 'source' | 'conduit' | 'energy' | 'delay' | 'splitter';
+export type TutorialStep = 'swipe-chain' | 'tap-splitter' | 'press-play' | 'stabilize' | 'lens' | 'loops' | 'complete' | 'skipped';
+export type NodeTapAction = 'select' | 'prime' | 'delay' | 'splitter' | 'stabilize' | 'none';

 export interface PulseNode extends WorldPoint {
@@ -13,4 +15,8 @@ export interface PulseNode extends WorldPoint {
 activationMs: number;
 scoreCooldownMs: number;
+ primed: boolean;
+ delayLevel: 0 | 1 | 2;
+ splitterPriority: number;
+ stabilizedMs: number;
}

@@ -22,4 +28,5 @@ export interface PulseLink {
 ageMs: number;
 expiresMs: number;
+ flashMs: number;
}

@@ -50,4 +57,19 @@ export interface HorizonLens {
}

+export interface ChainAnalysis {
+ reachableEnergyNodes: number;
+ totalEnergyNodes: number;
+ deadEndNodeIds: number[];
+ hasLoop: boolean;
+ linksUsed: number;
+ quality: 'Draw a chain' | 'Good start' | 'Hit more Energy nodes' | 'Dead end detected' | 'Loop possible' | 'Great loop';
+}
+
+export interface SuggestedFix {
+ fromId: number;
+ toId: number;
+ message: string;
}

```

```

+}
+
+export interface PulseLevel {
+ seed: string;
+}
+@ -61,16 +83,26 @@ export interface PulseLevel {
+export type BuildInput =
+ | { t: number; kind: 'link'; fromId: number; toId: number }
+ | { t: number; kind: 'chainSwipe'; nodeIds: number[]; path: { x: number; y: number; t: number }[] }
+ | { t: number; kind: 'nodeTap'; nodeId: number; action: NodeTapAction }
+ | { t: number; kind: 'undo' }
+ | { t: number; kind: 'clear' }
+ | { t: number; kind: 'play' };
+
+export interface LiveInput {
+ t: number;
+ kind: 'lens';
+ path: { x: number; y: number; t: number }[];
+ fromId?: number;
+ toId?: number;
+ success: boolean;
+}
+export type LiveInput =
+ | {
+ t: number;
+ kind: 'lens';
+ path: { x: number; y: number; t: number }[];
+ fromId?: number;
+ toId?: number;
+ success: boolean;
+ }
+ | {
+ t: number;
+ kind: 'stabilize';
+ nodeId: number;
+ rating: 'perfect' | 'stabilized' | 'early' | 'late';
+ success: boolean;
+ };
+
+export interface PulseResult {
+}
+@ -80,4 +112,6 @@ export interface PulseResult {
+ loopsCompleted: number;
+ linksUsed: number;
+ bestChainLength: number;
+ energyNodesHit: number;
+ stabilized: boolean;
+ collapsed: boolean;
+}
+@ -97,8 +131,12 @@ export interface PulseReplayPayload {
+export interface PulseInputResult {
+ ok: boolean;
+ kind: 'select' | 'link' | 'undo' | 'clear' | 'play' | 'lens' | 'invalid' | 'none';
+ kind: 'select' | 'link' | 'chainSwipe' | 'nodeTap' | 'stabilize' | 'undo' | 'clear' | 'play' | 'lens' | 'fix' | 'invalid' | 'none';
+ message: string;
+ fromId?: number;
+ toId?: number;
+ nodeId?: number;
+ nodeIds?: number[];
+ scoreDelta?: number;
+ energyDelta?: number;
+}
+
+@ -130,4 +168,13 @@ export interface PulseSnapshot {
+ selectedNodeId?: number;
+ tutorialHint: string;
+ tutorialActive: boolean;
+ tutorialStep: TutorialStep;
+ tutorialHighlightNodeIds: readonly number[];
+ tutorialGhostPath: readonly WorldPoint[];
+ chainAnalysis: ChainAnalysis;
+ suggestedFixes: readonly SuggestedFix[];
+ lastChainNodeIds: readonly number[];
+ lastTapAction: NodeTapAction;
+ deadEndNodeId?: number;
+ lastInputResult: PulseInputResult;
+ stepHash: string;
+diff --git a/src/main.ts b/src/main.ts
+index c0c1ef1..f5e9832 100644
+--- a/src/main.ts
++++ b/src/main.ts
+@ -2,5 +2,4 @@ import './styles.css';
+import { EventHorizonGame } from './game/EventHorizonGame';
+import { PulseMode } from './game/pulse/PulseMode';
+-import { getDailyPulseSeed } from './game/pulse/PulseLevelGenerator';
+
+interface EventHorizonRuntime {
+}
+@ -22,4 +21,5 @@ const helpButton = document.querySelector<HTMLButtonElement>('#help-button');
+const helpOverlay = document.querySelector<HTMLElement>('#help-overlay');
+const helpPlayButton = document.querySelector<HTMLButtonElement>('#help-play-button');
++const helpSkipButton = document.querySelector<HTMLButtonElement>('#help-skip-button');
+const posterLink = document.querySelector<HTMLAnchorElement>('#poster-link');
+const pulseControls = document.querySelector<HTMLElement>('#pulse-controls');
+@ -35,4 +35,5 @@ if (
+ !helpOverlay ||
+ !helpPlayButton ||
+ !helpSkipButton ||
+ !posterLink ||
+ !pulseControls ||
+@ -47,5 +48,5 @@ const params = new URLSearchParams(window.location.search);
+const mode = params.get('mode') === 'legacy' ? 'legacy' : 'pulse-chain';
+const debugInput = params.get('debugInput') === '1';
+-const seed = params.get('seed') ?? getDailyPulseSeed();
++const seed = params.get('seed') ?? 'tutorial-001';
+
+const game: EventHorizonRuntime =
+@ -65,5 +66,5 @@ pulseControls.hidden = mode === 'legacy';
+let pulsePaused = false;
+
+-const helpKey = mode === 'legacy' ? 'eventHorizon.helpSeen' : 'eventHorizon.pulseHelpSeen';
++const helpKey = mode === 'legacy' ? 'eventHorizon.helpSeen' : 'eventHorizon.iteration04HelpSeen';
+
+const hasSeenHelp = (): boolean => {
+@ -85,12 +86,21 @@ const markHelpSeen = (): void => {
+ const openHelp = (): void => {
+ helpOverlay.hidden = false;
+ helpPlayButton.textContent = 'PLAY';
+ helpPlayButton.textContent = mode === 'legacy' ? 'PLAY' : 'START TUTORIAL';
+ helpSkipButton.hidden = mode === 'legacy';
+ game.setPaused(true);
+ };
+
+-const closeHelp = (): void => {
++const closeHelp = (startTutorial: boolean): void => {
+ helpOverlay.hidden = true;
+ markHelpSeen();
+ if (game instanceof PulseMode) {
+ if (startTutorial) {
+ game.startTutorial();
+ } else {
+ game.skipTutorial();
+ }
+ }
+ game.setPaused(false);
+ updatePulseControls();
+};

```

```

@@ -107,5 +117,6 @@ restartButton.addEventListener('click', () => {
 helpButton.addEventListener('click', openHelp);
 -helpPlayButton.addEventListener('click', closeHelp);
 +helpPlayButton.addEventListener('click', () => closeHelp(true));
 +helpSkipButton.addEventListener('click', () => closeHelp(false));

 shareButton.addEventListener('click', async () => {
 @@ -122,4 +133,7 @@ if (game instanceof PulseMode) {
 pulsePaused = !pulsePaused;
 game.setPaused(pulsePaused);
 + } else if (snapshot.endReason === 'pulse-died') {
 + pulsePaused = false;
 + game.fixChain();
 + } else {
 pulsePaused = false;
 @@ -143,4 +157,7 @@ if (game instanceof PulseMode) {
 pulsePaused = false;
 updatePulseControls();
 + } else if (game.getSnapshot().phase === 'ended') {
 + const nextSeed = `seed-${Date.now().toString(36)}`;
 + window.location.href = `${window.location.pathname}?seed=${encodeURIComponent(nextSeed)}`;
 + }
 });
 @@ -185,8 +202,9 @@ function updatePulseControls(): void {
 undoButton.hidden = false;
 clearButton.hidden = false;
 - playButton.hidden = true;
 - undoButton.textContent = 'Replay';
 - clearButton.textContent = 'Restart';
 - playButton.disabled = true;
 + playButton.hidden = false;
 + undoButton.textContent = snapshot.endReason === 'pulse-died' ? 'Fix Chain' : 'Replay';
 + clearButton.textContent = 'Replay';
 + playButton.textContent = 'New Seed';
 + playButton.disabled = false;
 }

 @@ -215,4 +233,13 @@ declare global {
 getSnapshot: () => unknown;
 playPulse: () => unknown;
 startTutorial: () => void;
 + skipTutorial: () => void;
 + getTutorialStep: () => unknown;
 + simulateChainSwipe: (nodeIds: number[]) => unknown;
 + analyzeChain: () => unknown;
 + getSuggestedFixes: () => unknown;
 + primeNode: (id: number) => unknown;
 + cycleNode: (id: number) => unknown;
 + stabilizeNode: (id: number) => unknown;
 + setInputDebug: (enabled: boolean) => void;
 + simulateLens: (points: { x: number; y: number }[]) => unknown;
 @@ -240,5 +267,5 @@ window.__EVENT_HORIZON_DEBUG__ =
 openHelp();
 + } else {
 - closeHelp();
 + closeHelp(false);
 + }
 },
 @@ -252,4 +279,13 @@ window.__EVENT_HORIZON_DEBUG__ =
 getSnapshot: () => game.getSnapshot(),
 playPulse: () => game.playPulse(),
 + startTutorial: () => game.startTutorial(),
 + skipTutorial: () => game.skipTutorial(),
 + getTutorialStep: () => game.getTutorialStep(),
 + simulateChainSwipe: (nodeIds) => game.simulateChainSwipe(nodeIds),
 + analyzeChain: () => game.analyzeChain(),
 + getSuggestedFixes: () => game.getSuggestedFixes(),
 + primeNode: (id) => game.primeNode(id),
 + cycleNode: (id) => game.cycleNode(id),
 + stabilizeNode: (id) => game.stabilizeNode(id),
 + setInputDebug: (enabled) => game.setInputDebug(enabled),
 + simulateLens: (points) => game.simulateLens(points)
 @@ -264,5 +300,5 @@ window.__EVENT_HORIZON_DEBUG__ =
 openHelp();
 + } else {
 - closeHelp();
 + closeHelp(false);
 + }
 },
 @@ -276,4 +312,13 @@ window.__EVENT_HORIZON_DEBUG__ =
 getSnapshot: () => game.getSnapshot(),
 playPulse: () => undefined,
 + startTutorial: () => undefined,
 + skipTutorial: () => undefined,
 + getTutorialStep: () => undefined,
 + simulateChainSwipe: () => undefined,
 + analyzeChain: () => undefined,
 + getSuggestedFixes: () => [],
 + primeNode: () => undefined,
 + cycleNode: () => undefined,
 + stabilizeNode: () => undefined,
 + setInputDebug: (enabled) => game.setInputDebug(enabled),
 + simulateLens: () => undefined

diff --git a/src/styles.css b/src/styles.css
index 7413c0f..bbdb8c 100644
--- a/src/styles.css
+++ b/src/styles.css
@@ -95,5 +95,5 @@ a {
 #pulse-controls[data-phase="pulse"],
 #pulse-controls[data-phase="ended"] {
 - grid-template-columns: 1fr 1fr;
 + grid-template-columns: repeat(3, 1fr);
 }

 @@ -132,4 +132,8 @@ a {
 }

 +#pulse-controls button:disabled {
 + opacity: 0.45;
 + }
 +
 + #pulse-controls button[hidden] {
 + display: none;
 + }
 @@ -164,7 +168,7 @@ a {
 #help-panel {
 width: min(92vw, 440px);
 - max-height: min(88vh, 760px);
 + max-height: min(92vh, 760px);
 overflow: auto;
 - padding: 22px;
 + padding: 18px;
 border: 1px solid rgba(139, 222, 255, 0.34);
 border-radius: 8px;
 @@ -175,14 +179,21 @@ a {

 #help-panel h1 {
 - margin: 12px 0 8px;
 - font-size: 28px;
 + margin: 10px 0 6px;
 + font-size: 25px;
 letter-spacing: 0;
 }

```



```

}

+help-panel h2 {
+ margin: 10px 0 3px;
+ color: #9ffcff;
+ font-size: 13px;
+ letter-spacing: 0.08em;
+}
+
+help-panel p {
- margin: 10px 0;
+ margin: 7px 0;
+ color: #cfefff;
- font-size: 16px;
- line-height: 1.45;
+ font-size: 14px;
+ line-height: 1.36;
}

@@ -199,5 +210,6 @@ a {
}

-#help-play-button {
+#help-play-button,
+#help-skip-button {
+ width: 100%;
+ min-height: 52px;
@@ -213,8 +225,14 @@ a {
}

+#help-skip-button {
+ border: 1px solid rgba(179, 226, 255, 0.24);
+ background: rgba(7, 12, 25, 0.84);
+ color: #f7fbff;
+}
+
+.help-example {
+ position: relative;
- height: 126px;
- margin: 2px 0 10px;
+ height: 96px;
+ margin: 2px 0 8px;
+ overflow: hidden;
+ border-radius: 8px;
diff --git a/tests/e2e/playable.spec.ts b/tests/e2e/playable.spec.ts
index ed3061a..4c7c6ad 100644
--- a/tests/e2e/playable.spec.ts
+++ b/tests/e2e/playable.spec.ts
@@ -4,98 +4,80 @@ const WORLD_WIDTH = 1080;
const WORLD_HEIGHT = 1920;

-test('help opens on first visit', async ({ page }) => {
+test('first visit opens updated help and tutorial', async ({ page }) => {
+ await openGame(page);
+ await expect(page.locator('#help-overlay')).toBeVisible();
- await expect(page.locator('#help-title')).toHaveText('EVENT HORIZON');
- await expect(page.locator('#help-overlay')).toContainText('Build a dark-energy chain');
+ await expect(page.locator('#help-overlay')).toContainText('Build a chain. Then keep it alive.');
```

```

+ expect(snapshot.lastInputResult.kind).toBe('stabilize');
+ expect(snapshot.lastInputResult.ok).toBe(true);
+ expect(snapshot.tutorialStep).toBe('lens');
});

-test('swipe during pulse phase creates Horizon Lens and records replay inputs', async ({ page }) => {
- await openGameAndPlay(page);
- await buildTutorialChain(page);
- await page.locator('#pulse-play-button').click();
- await page.waitForFunction(() => (window.__EVENT_HORIZON__?.getSnapshot() as { phase?: string })?.phase === 'pulse');
- const nodes = await getNodes(page);
- const a = nodes.find((node) => node.id === 6) ?? nodes[4];
- const b = nodes.find((node) => node.id === 8) ?? nodes[5];
- const start = await worldToScreen(page, a.x, a.y);
- const end = await worldToScreen(page, b.x, b.y);
- await page.mouse.move(start.x, start.y);
- await page.mouse.down();
- await page.mouse.move((start.x + end.x) / 2, (start.y + end.y) / 2 - 30, { steps: 5 });
- await page.mouse.move(end.x, end.y, { steps: 5 });
- await page.mouse.up();
- await page.waitForTimeout(120);
- const result = await page.evaluate(() => window.__EVENT_HORIZON_DEBUG__?.getLastInputResult()) as { kind: string; message: string };
+test('swiping Horizon Lens creates bridge and replay records grammar', async ({ page }) => {
+ await tutorialPulseReadyForLens(page);
+ await swipeThroughNodes(page, [3, 4]);
+ await page.waitForTimeout(160);
+ const snapshot = await getSnapshot(page);
+ const replay = await page.evaluate(() => window.__EVENT_HORIZON__?.getReplayPayload()) as {
- buildInputs: unknown[];
- liveInputs: Array<{ kind: string; success: boolean }>;
+ buildInputs: Array<{ kind: string }>;
+ liveInputs: Array<{ kind: string; success?: boolean }>;
 };
- expect(result.kind).toBe('lens');
- expect(['BRIDGE CREATED', 'NO ANCHOR']).toContain(result.message);
- expect(replay.buildInputs.some((input) => input.as { kind: string })?.kind === 'play')).toBe(true);
+ expect(snapshot.lastInputResult.kind).toBe('lens');
+ expect(snapshot.lastInputResult.ok).toBe(true);
+ expect(snapshot.links.some((link) => link.temporary && link.fromId === 3 && link.toId === 4)).toBe(true);
+ expect(replay.buildInputs.some((input) => input.kind === 'chainSwipe')).toBe(true);
+ expect(replay.buildInputs.some((input) => input.kind === 'nodeTap')).toBe(true);
+ expect(replay.liveInputs.some((input) => input.kind === 'play')).toBe(true);
+ expect(replay.liveInputs.some((input) => input.kind === 'stabilize')).toBe(true);
+ expect(replay.liveInputs.some((input) => input.kind === 'lens')).toBe(true);
});

-test('collapse or stabilized end state is reachable', async ({ page }) => {
- await openGameAndPlay(page);
- await page.evaluate(() => window.__EVENT_HORIZON_DEBUG__?.forceCollapse());
- await page.waitForTimeout(150);
- const snapshot = await page.evaluate(() => window.__EVENT_HORIZON__?.getSnapshot()) as {
- phase: string;
- collapsed: boolean;
- stabilized: boolean;
- };
- expect(snapshot.phase).toBe('ended');
- expect(snapshot.collapsed || snapshot.stabilized).toBe(true);
+test('dead-end failure shows suggested fix and FIX CHAIN returns to build', async ({ page }) => {
+ await startTutorial(page);
+ await page.evaluate(() => {
+ window.__EVENT_HORIZON_DEBUG__?.skipTutorial();
+ window.__EVENT_HORIZON_DEBUG__?.clearLinks();
+ window.__EVENT_HORIZON_DEBUG__?.addLink(1, 2);
+ window.__EVENT_HORIZON_DEBUG__?.playPulse();
+ });
+ await page.waitForFunction(() => (window.__EVENT_HORIZON__?.getSnapshot() as { phase?: string } | undefined)?.phase === 'ended', null, { timeout: 9000 });
+ const ended = await getSnapshot(page);
+ expect(ended.endReason).toBe('pulse-died');
+ expect(ended.suggestedFixes.length).toBeGreaterThan(0);
+ await page.locator('#pulse-undo-button').click();
+ await page.waitForTimeout(120);
+ expect((await getSnapshot(page)).phase).toBe('build');
});

@@ -104,9 +86,9 @@ async function openGame(page: Page): Promise<void> {
 window.localStorage.clear();
});
- await page.goto('..?seed=tutorial&debugInput=1');
+ await page.goto('..?seed=tutorial-001&debugInput=1');
 await page.locator('canvas').waitFor({ state: 'visible' });
}

-async function openGameAndPlay(page: Page): Promise<void> {
+async function startTutorial(page: Page): Promise<void> {
 await openGame(page);
 await page.locator('#help-play-button').click();
@@ -114,37 +96,70 @@ async function openGameAndPlay(page: Page): Promise<void> {
}

-async function buildTutorialChain(page: Page): Promise<void> {
- const nodes = await getNodes(page);
- const byId = new Map(nodes.map((node) => [node.id, node]));
- for (const [fromId, toId] of [
- [1, 2],
- [2, 3],
- [3, 4],
- [4, 5],
- [4, 6]
-]) {
- const from = byId.get(fromId);
- const to = byId.get(toId);
- expect(from).toBeTruthy();
- expect(to).toBeTruthy();
- await tapWorld(page, from!.x, from!.y);
- await tapWorld(page, to!.x, to!.y);
- }
+async function tutorialChainReady(page: Page): Promise<void> {
+ await startTutorial(page);
+ await swipeThroughNodes(page, [1, 2, 3]);
+ await page.waitForFunction(() => (window.__EVENT_HORIZON__?.getSnapshot() as { tutorialStep?: string } | undefined)?.tutorialStep === 'tap-splitter');
}

-async function getNodes(page: Page) {
- return (await page.evaluate(() => window.__EVENT_HORIZON_DEBUG__?.getNodes())) as Array<{
- id: number;
- type: string;
- x: number;
- y: number;
- }>;
+async function tutorialReadyToPlay(page: Page): Promise<void> {
+ await tutorialChainReady(page);
+ await tapNode(page, 4);
+ await page.waitForFunction(() => (window.__EVENT_HORIZON__?.getSnapshot() as { tutorialStep?: string } | undefined)?.tutorialStep === 'press-play');
}

-async function tapWorld(page: Page, x: number, y: number): Promise<void> {
- const point = await worldToScreen(page, x, y);
+async function tutorialPulseReadyForLens(page: Page): Promise<void> {
+ await tutorialReadyToPlay(page);
+ await page.locator('#pulse-play-button').click();
+ await page.waitForFunction(() => (window.__EVENT_HORIZON__?.getSnapshot() as { phase?: string } | undefined)?.phase === 'pulse');
+ await page.waitForTimeout(2200);
+ await tapNode(page, 3);

```

```

+ await page.waitForFunction(() => (window.__EVENT_HORIZON__.getSnapshot() as { tutorialStep?: string } | undefined)?.tutorialStep === 'lens');
+}
+
+async function getSnapshot(page: Page) {
+ return (await page.evaluate(() => window.__EVENT_HORIZON__.getSnapshot())) as {
+ phase: string;
+ tutorialStep: string;
+ tutorialHighlightNodeIds: number[];
+ linksUsed: number;
+ lastChainNodeIds: number[];
+ lastInputResult: { kind: string; ok: boolean; message: string };
+ links: Array<{ fromId: number; toId: number; temporary: boolean }>;
+ suggestedFixes: unknown[];
+ endReason?: string;
+ };
+}
+
+async function nodeIdBy(page: Page, id: number) {
+ const node = await page.evaluate((nodeId) => {
+ const nodes = window.__EVENT_HORIZON_DEBUG__.getNodes() as Array<{ id: number; x: number; y: number; splitterPriority: number }> | undefined;
+ return nodes?.find((candidate) => candidate.id === nodeId);
+ }, id);
+ if (!node) {
+ throw new Error('Missing node ${id}');
+ }
+ return node as { id: number; x: number; y: number; splitterPriority: number };
+}
+
+async function tapNode(page: Page, id: number): Promise<void> {
+ const node = await nodeIdBy(page, id);
+ const point = await worldToScreen(page, node.x, node.y);
+ await page.mouse.click(point.x, point.y);
+}
+
+async function swipeThroughNodes(page: Page, ids: number[]): Promise<void> {
+ const points = [];
+ for (const id of ids) {
+ const node = await nodeIdBy(page, id);
+ points.push(await worldToScreen(page, node.x, node.y));
+ }
+ await page.mouse.move(points[0].x, points[0].y);
+ await page.mouse.down();
+ for (const point of points.slice(1)) {
+ await page.mouse.move(point.x, point.y, { steps: 8 });
+ }
+ await page.mouse.up();
+}
+
+async function worldToScreen(page: Page, x: number, y: number): Promise<{ x: number; y: number }> {
+ const box = await page.locator('canvas').boundingBox();
+ diff --git a/tests/pulse-simulation.test.ts b/tests/pulse-simulation.test.ts
+index 011b2cd..499d262 100644
+--- a/tests/pulse-simulation.test.ts
++++ b/tests/pulse-simulation.test.ts
+@@ -4,106 +4,116 @@ import { PulseSimulation } from '../src/game/pulse/PulseSimulation';
+
+ const options = {
+ - seed: 'tutorial',
+ + seed: 'tutorial-001',
+ startedAt: 1780185600000
+ };
+
+ describe('pulse-chain mode', () => {
+ - it('seeded level generation is deterministic', () => {
+ + it('tutorial level is deterministic and hand-tuned', () => {
+ const first = generatePulseLevel('tutorial-001');
+ const second = generatePulseLevel('tutorial-001');
+ expect(second).toEqual(first);
+ expect(first.nodes.slice(0, 4).map((node) => node.type)).toEqual(['source', 'energy', 'delay', 'splitter']);
+ });
+
+ it('seeded level generation is deterministic for normal seeds', () => {
+ expect(generatePulseLevel('abc')).toEqual(generatePulseLevel('abc'));
+ expect(generatePulseLevel('abc').nodes).not.toEqual(generatePulseLevel('xyz').nodes);
+ });
+
+ - it('same seed generates same nodes', () => {
+ - const first = new PulseSimulation(options).getNodes();
+ - const second = new PulseSimulation(options).getNodes();
+ - expect(second).toEqual(first);
+ + it('swipe crossing 3 nodes creates 2 links and records chainSwipe', () => {
+ + const sim = new PulseSimulation(options);
+ + const result = sim.applyChainSwipe(pathFor(sim, [1, 2, 3]));
+ + expect(result.ok).toBe(true);
+ + expect(sim.getLinks().map((link) => [link.fromId, link.toId])).toEqual([
+ + [1, 2],
+ + [2, 3]
+ +]);
+ + expect(sim.getReplayPayload().buildInputs.some((input) => input.kind === 'chainSwipe')).toBe(true);
+ });
+
+ - it('link placement respects budget and rejects duplicate/self links', () => {
+ + it('chain swipe ignores duplicate adjacent nodes and avoids self-links', () => {
+ const sim = new PulseSimulation(options);
+ expect(sim.addLink(1, 1).ok).toBe(false);
+ expect(sim.addLink(1, 2).ok).toBe(true);
+ expect(sim.addLink(1, 2).ok).toBe(false);
+ expect(sim.addLink(1, 3).ok).toBe(true);
+ expect(sim.addLink(2, 3).ok).toBe(true);
+ expect(sim.addLink(3, 4).ok).toBe(true);
+ expect(sim.addLink(4, 5).ok).toBe(true);
+ expect(sim.addLink(4, 6).ok).toBe(true);
+ expect(sim.addLink(6, 8).ok).toBe(false);
+ expect(sim.getSnapshot().linksUsed).toBe(6);
+ sim.applyChainSwipe(pathFor(sim, [1, 2, 2, 3]));
+ expect(sim.getLinks().map((link) => [link.fromId, link.toId])).toEqual([
+ [1, 2],
+ [2, 3]
+]);
+ });
+
+ - it('pulse travels from source to connected energy node and increases score/energy', () => {
+ + it('chain swipe respects link budget', () => {
+ const sim = new PulseSimulation(options);
+ sim.addLink(1, 2);
+ const beforeEnergy = sim.getSnapshot().darkEnergy;
+ sim.playPulse();
+ step(sim, 1500);
+ const snapshot = sim.getSnapshot();
+ expect(snapshot.score).toBeGreaterThan(0);
+ expect(snapshot.darkEnergy).toBeGreaterThan(beforeEnergy - 1);
+ sim.applyChainSwipe(pathFor(sim, [1, 2, 3, 4, 5, 6, 8, 11]));
+ expect(sim.getSnapshot().linksUsed).toBeLessThanOrEqual(sim.getSnapshot().linkBudget);
+ });
+
+ - it('delay node pauses pulse briefly', () => {
+ + it('chain analysis detects reachable energy nodes, dead ends, and loops', () => {
+ const sim = new PulseSimulation(options);
+ sim.addLink(1, 2);
+ sim.addLink(2, 3);
+ sim.playPulse();
+ step(sim, 1300);
+ const pulse = sim.getPulses()[0];

```

```

- expect(pulse?.currentNodeId).toBe(3);
- expect(pulse?.delayMs).toBeGreaterThan(0);
+ sim.applyChainSwipe(pathFor(sim, [1, 2, 3]));
+ expect(sim.analyzeChain()).toMatchObject({ reachableEnergyNodes: 1, deadEndNodeIds: [3], hasLoop: false });
+ sim.addLink(3, 4);
+ sim.addLink(4, 7);
+ sim.addLink(7, 9);
+ sim.addLink(9, 2);
+ const analysis = sim.analyzeChain();
+ expect(analysis.hasLoop).toBe(true);
+ expect(analysis.deadEndNodeIds).toHaveLength(0);
});

- it('splitter creates child pulses', () => {
+ it('tapping Energy primes it, Delay cycles delay, and Splitter cycles priority', () => {
 const sim = new PulseSimulation(options);
- sim.addLink(1, 2);
- sim.addLink(2, 3);
- sim.addLink(3, 4);
+ expect(sim.primeNode(2).ok).toBe(true);
+ expect(sim.getNodes().find((node) => node.id === 2)?.primed).toBe(true);
+ const delayBefore = sim.getNodes().find((node) => node.id === 3)?.delayLevel;
+ sim.cycleNode(3);
+ expect(sim.getNodes().find((node) => node.id === 3)?.delayLevel).not.toBe(delayBefore);
+ const splitterBefore = sim.getNodes().find((node) => node.id === 4)?.splitterPriority;
+ sim.cycleNode(4);
+ expect(sim.getNodes().find((node) => node.id === 4)?.splitterPriority).not.toBe(splitterBefore);
+ });

+ it('pulse travels, delay pauses, splitter branches, and energy scores', () => {
+ const sim = new PulseSimulation(options);
+ sim.applyChainSwipe(pathFor(sim, [1, 2, 3, 4]));
+ sim.addLink(4, 5);
+ sim.addLink(4, 6);
+ sim.playPulse();
+ step(sim, 2700);
+ expect(sim.getPulses().length).toBeGreaterThanOrEqual(2);
+ step(sim, 6500);
+ const snapshot = sim.getSnapshot();
+ expect(snapshot.score).toBeGreaterThan(100);
+ expect(snapshot.maxMultiplier).toBeGreaterThanOrEqual(1);
+ });

- it('long loop increases multiplier', () => {
+ it('pulse-phase tap stabilizes a node shortly before arrival', () => {
 const sim = new PulseSimulation(options);
- sim.addLink(1, 2);
- sim.addLink(2, 3);
- sim.addLink(3, 4);
- sim.addLink(4, 7);
- sim.addLink(7, 9);
- sim.addLink(9, 2);
+ sim.applyChainSwipe(pathFor(sim, [1, 2, 3]));
+ sim.playPulse();
+ step(sim, 9000);
+ const snapshot = sim.getSnapshot();
+ expect(snapshot.loopsCompleted).toBeGreaterThanOrEqual(1);
+ expect(snapshot.maxMultiplier).toBeGreaterThan(1);
+ step(sim, 2200);
+ const result = sim.stabilizeNode(3);
+ expect(result.ok).toBe(true);
+ expect(sim.getReplayPayload().liveInputs.some((input) => input.kind === 'stabilize' && input.nodeId === 3)).toBe(true);
+ });

- it('dead end kills pulse', () => {
+ it('Horizon Lens creates a temporary bridge', () => {
 const sim = new PulseSimulation(options);
- sim.addLink(1, 2);
- sim.applyChainSwipe(pathFor(sim, [1, 2, 3]));
- sim.playPulse();
- step(sim, 3600);
- expect(sim.getSnapshot().phase).toBe('ended');
- expect(sim.getSnapshot().endReason).toBe('pulse-died');
+ step(sim, 2600);
+ const result = sim.applyLens(pathFor(sim, [3, 4]));
+ expect(result.ok).toBe(true);
+ expect(sim.getLinks().some((link) => link.temporal && link.fromId === 3 && link.toId === 4)).toBe(true);
+ expect(sim.getReplayPayload().liveInputs.some((input) => input.kind === 'lens')).toBe(true);
+ });

- it('Horizon Lens creates a temporary bridge', () => {
+ it('dead end kills pulse and suggests fixes', () => {
 const sim = new PulseSimulation(options);
- sim.addLink(1, 2);
- sim.playPulse();
- const result = sim.applyLens([
- { x: 825, y: 1215, t: 0 },
- { x: 700, y: 1410, t: 90 }
-]);
- expect(result.ok).toBe(true);
- expect(sim.getLinks().some((link) => link.temporal)).toBe(true);
- expect(sim.getReplayPayload().liveInputs).toHaveLength(1);
+ step(sim, 6000);
+ expect(sim.getSnapshot().phase).toBe('ended');
+ expect(sim.getSnapshot().endReason).toBe('pulse-died');
+ expect(sim.getSuggestedFixes().length).toBeGreaterThan(0);
+ });

- it('replay with same seed and inputs reproduces result and stepHash', () => {
+ it('replay with chainSwipe, taps, and lens reproduces result and stepHash', () => {
 const first = runScripted();
 const second = runScripted();
@@ -120,20 +130,27 @@ function step(sim: PulseSimulation, ms: number): void {
}

+function pathFor(sim: PulseSimulation, nodeIds: number[]) {
+ return nodeIds.flatMap((nodeId, index) => {
+ const node = sim.getNodes().find((candidate) => candidate.id === nodeId);
+ if (!node) {
+ throw new Error(`Missing node ${nodeId}`);
+ }
+ return [
+ { x: node.x - 8, y: node.y - 8, t: index * 90 },
+ { x: node.x, y: node.y, t: index * 90 + 35 },
+ { x: node.x + 8, y: node.y + 8, t: index * 90 + 70 }
+];
+ });
+}
+
+function runScripted(): PulseSimulation {
+ const sim = new PulseSimulation(options);
+ for (const [from, to] of [
+ [1, 2],
+ [2, 3],
+ [3, 4],
+ [4, 5],
+ [4, 6]
+]) {
+ sim.addLink(from, to);
+ }
+ sim.applyChainSwipe(pathFor(sim, [1, 2, 3]));
+ sim.cycleNode(4);
+ sim.playPulse();

```

```

- sim.applyLens([
- { x: 835, y: 1215, t: 0 },
- { x: 700, y: 1410, t: 120 }
-]);
+ step(sim, 2200);
+ sim.stabilizeNode(3);
+ step(sim, 3000);
+ sim.applyLens(pathFor(sim, [3, 4]));
+ step(sim, 6500);
+ return sim;
diff --git a/tests/score-submit.test.ts b/tests/score-submit.test.ts
index c527df6..83a271a 100644
--- a/tests/score-submit.test.ts
+++ b/tests/score-submit.test.ts
@@ -18,12 +18,22 @@ const pulseReplay = {
 version: 1,
 mode: 'pulse-chain',
- seed: 'tutorial',
+ seed: 'tutorial-001',
 startedAt: 1780185600000,
 buildInputs: [
- { t: 0, kind: 'link', fromId: 1, toId: 2 },
- { t: 220, kind: 'link', fromId: 2, toId: 3 },
+ {
+ t: 0,
+ kind: 'chainSwipe',
+ nodeIds: [1, 2, 3],
+ path: [
+ { x: 250, y: 1380, t: 0 },
+ { x: 420, y: 1165, t: 90 },
+ { x: 635, y: 1010, t: 180 }
+]
+ },
+ { t: 220, kind: 'nodeTap', nodeId: 4, action: 'splitter' },
+ { t: 520, kind: 'play' }
],
 liveInputs: [
+ { t: 2200, kind: 'stabilize', nodeId: 3, rating: 'stabilized', success: true },
+ {
+ t: 1420,
@@ -44,4 +54,6 @@ const pulseReplay = {
 loopsCompleted: 1,
 linksUsed: 5,
+ bestChainLength: 5,
+ energyNodesHit: 2,
+ stabilized: false,
+ collapsed: false

```

## Full Source Code For Changed Text Files

### README.md

```

Event Horizon

Event Horizon is a mobile-first cosmic chain-reaction game about delaying a galaxy's collapse into a black hole. The current slice is a plain Vite + PixiJS v8 site with deterministic simulation state kept outside the renderer.

Current Slice

- Logical portrait playfield: `1080 x 1920`
- PixiJS canvas renderer with WebGL preference
- Fixed `60 Hz` simulation step decoupled from render frames
- Seeded `mulberry32` RNG and replay payloads from seed + input timings
- Default Pulse Chain mode: connect Dark Energy Nodes, press Play, and watch a Stabilizing Pulse travel the network
- Swipe-through-node chain drawing, tap-tap link placement, and tap-based node tuning
- Interactive `tutorial-001` first-run seed that teaches swipe chain, tap strategy, Play, stabilize taps, and Horizon Lens rescue
- Energy, Delay, Splitter, Conduit, and Source nodes with scoring and multiplier rules
- Energy nodes can be primed, Delay nodes cycle timing, and Splitters cycle output priority
- Pulse-phase taps stabilize arriving nodes for score and dark-energy gain
- Horizon Lens swipes during pulse playback create short-lived temporary bridges
- Path-based input recording with mobile Pointer Events and TouchEvent fallback
- First-run help overlay, tutorial hints, visible pulse/lens feedback, and debug hooks
- Bottom Collapse Meter, score/multiplier HUD, and stabilized/collapsed end states
- Share poster export from three captured gameplay frames
- Minimal Netlify score submit function that accepts legacy and Pulse Chain replay payloads
- Previous tether survival prototype remains available with `?mode=legacy`

Assumptions

- The initial repo was empty, so this is a greenfield Vite scaffold.
- Custom deterministic motion is enough for the Pulse Chain slice; no Matter.js or Planck.js dependency is needed.
- GitHub Pages is the public static target and uses `/event-horizon/` as the base path.
- Netlify deploys should set `VITE_BASE_PATH=/` through the included `netlify.toml`.
- The score function validates and acknowledges payloads only; durable storage is a backlog item.

Commands

```bash
npm install
npm run dev
npm run build
npm run lint
npm run test
npm run test:e2e
npm run score:test
npm run capture:iteration-03
npm run report:iteration-03
npm run capture:iteration-04
npm run report:iteration-04
npm run capture:iteration-02
npm run report:iteration-02
npm run report:pdf
```

Local Vite serves the game at:

```text
http://127.0.0.1:5173/event-horizon/
```

Deployment

GitHub Pages

The default Vite base path is `/event-horizon/`, so `npm run build` produces static assets compatible with:

```text
https://kevinhegg.github.io/event-horizon/
```

Netlify

`netlify.toml` uses:

```toml
[build]
command = "VITE_BASE_PATH=/ npm run build"
publish = "dist"

```

```
[functions]
  directory = "netlify/functions"
  '\'\`

The score endpoint is available at:

'\'\`text
./netlify/functions/score-submit
'\'\`

For local Netlify function routing, run Netlify CLI from the repo root:

'\'\`bash
npx netlify dev
'\'\`

## Replay Payload Shape

'\'\`json
{
  "version": 1,
  "mode": "pulse-chain",
  "seed": "tutorial",
  "startedAt": 1780185600000,
  "buildInputs": [
    {
      "t": 0,
      "kind": "chainSwipe",
      "nodeIds": [1, 2, 3],
      "path": [
        { "x": 250, "y": 1388, "t": 0 },
        { "x": 420, "y": 1165, "t": 90 },
        { "x": 635, "y": 1010, "t": 180 }
      ]
    },
    { "t": 300, "kind": "nodeTap", "nodeId": 4, "action": "splitter" },
    { "t": 620, "kind": "play" }
  ],
  "liveInputs": [
    { "t": 2200, "kind": "stabilize", "nodeId": 3, "rating": "stabilized", "success": true },
    {
      "t": 1600,
      "kind": "lens",
      "path": [
        { "x": 835, "y": 1215, "t": 0 },
        { "x": 700, "y": 1410, "t": 120 }
      ],
      "fromId": 6,
      "toId": 8,
      "success": true
    }
  ],
  "result": {
    "score": 620,
    "survivalMs": 12120,
    "maxMultiplier": 2,
    "loopsCompleted": 1,
    "linksUsed": 5,
    "bestChainLength": 5,
    "energyNodesHit": 2,
    "stabilized": false,
    "collapsed": false
  },
  "stepHash": "04ce9b1a"
}
'\'\`

Legacy mode still uses:

'\'\`json
{
  "version": 1,
  "seed": "eh-2026-05-29-alpha",
  "startedAt": 1780051200000,
  "survivalMs": 67421,
  "score": 1280,
  "energyCaptured": 87,
  "maxStreak": 24,
  "tapEvents": [],
  "swipeEvents": [],
  "phaseTransitions": []
}
'\'\`

## Current Branch Workflow

'\'\`bash
git switch -c feat/iteration-04-playability-tap-swipe-strategy
git add .
git commit -m "Improve Event Horizon playability tutorial and strategy"
git push -u origin feat/iteration-04-playability-tap-swipe-strategy
gh pr create --base main --head feat/iteration-04-playability-tap-swipe-strategy --title "Improve Event Horizon playability tutorial and strategy" --body-file docs/iteration-04-report.md
'\'\`
```

index.html

```
<!doctype html>
<html lang="en-US">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover" />
    <meta name="theme-color" content="#03040a" />
    <meta name="description" content="Event Horizon, a one-thumb galaxy survival prototype." />
    <title>Event Horizon</title>
  </head>
  <body>
    <main id="app" aria-label="Event Horizon game">
      <div id="game-shell">
        <div id="game-root" aria-label="Playable canvas"></div>
        <div id="status-layer" aria-live="polite">
          <button id="restart-button" type="button" aria-label="Restart run" title="Restart run"></button>
          <button id="share-button" type="button" aria-label="Create share poster" title="Create share poster"></button>
          <button id="help-button" type="button" aria-label="How to play" title="How to play"></button>
          <a id="poster-link" download="event-horizon-poster.png" aria-label="Download share poster"></a>
        </div>
        <div id="pulse-controls" aria-live="polite">
          <button id="pulse-undo-button" type="button">Undo</button>
          <button id="pulse-clear-button" type="button">Clear</button>
          <button id="pulse-play-button" type="button">Play</button>
        </div>
        <section id="help-overlay" aria-modal="true" role="dialog" aria-labelledby="help-title" hidden>
          <div id="help-panel">
            <div class="help-example" aria-hidden="true">
              <span class="help-hole"></span>
              <span class="help-tether"></span>
              <span class="help-orb"></span>
              <span class="help-swipe"></span>
            </div>
            <h1 id="help-title">EVENT HORIZON</h1>
            <p>Build a chain. Then keep it alive.</p>
          </div>
        </section>
      </div>
    </main>
  </body>
</html>
```

```

<h2>BUILD</h2>
<p>Swipe through nodes to draw a chain.<br />Tap nodes to tune them.<br />Press Play.</p>
<h2>RUN</h2>
<p>The pulse follows your links.<br />Energy nodes refill the Collapse Meter.<br />Long chains and loops multiply your score.</p>
<h2>RESCUE</h2>
<p>Tap nodes as the pulse arrives to stabilize them.<br />Swipe between nodes to create a temporary Horizon Lens bridge.</p>
<p>The black hole always wins.<br />Your strategy buys the galaxy time.</p>
<button id="help-play-button" type="button">START TUTORIAL</button>
<button id="help-skip-button" type="button">SKIP TUTORIAL</button>
</div>
</section>
</div>
</main>
<script type="module" src="/src/main.ts"></script>
</body>
</html>

```

package.json

```

{
  "name": "event-horizon",
  "version": "0.1.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite --host 127.0.0.1",
    "build": "tsc --noEmit 66 vite build",
    "preview": "vite preview --host 127.0.0.1",
    "test": "vitest run",
    "test:e2e": "playwright test",
    "lint": "eslint .",
    "score:test": "node scripts/test-score-submit.mjs",
    "capture:artifacts": "node scripts/capture-artifacts.mjs",
    "capture:iteration-02": "node scripts/capture-iteration-02-artifacts.mjs",
    "capture:iteration-03": "node scripts/capture-iteration-03-artifacts.mjs",
    "capture:iteration-04": "node scripts/capture-iteration-04-artifacts.mjs",
    "report:pdf": "node scripts/generate-report-pdf.mjs",
    "report:iteration-02r": "node scripts/generate-iteration-02-report.mjs",
    "report:iteration-03r": "node scripts/generate-iteration-03-report.mjs",
    "report:iteration-04r": "node scripts/generate-iteration-04-report.mjs"
  },
  "dependencies": {
    "pixi.js": "8.18.1"
  },
  "devDependencies": {
    "@eslint/js": "9.39.1",
    "@playwright/test": "1.57.0",
    "@types/node": "24.12.4",
    "eslint": "9.39.1",
    "typescript": "5.9.3",
    "typescript-eslint": "8.49.0",
    "vite": "8.0.14",
    "vitest": "4.0.15"
  }
}

```

scripts/capture-iteration-04-artifacts.mjs

```

import { mkdir } from 'node:fs/promises';
import { chromium } from '@playwright/test';

const WORLD_WIDTH = 1080;
const WORLD_HEIGHT = 1920;
const outputDir = new URL('../docs/artifacts/', import.meta.url);

await mkdir(outputDir, { recursive: true });

const browser = await chromium.launch({ channel: 'chrome' });
const page = await browser.newPage({
  viewport: { width: 390, height: 844 },
  deviceScaleFactor: 2,
  isMobile: true,
  hasTouch: true
});

await page.addInitScript(() => window.localStorage.clear());
await page.goto('http://127.0.0.1:5173/event-horizon/?seed=tutorial-001', { waitUntil: 'networkidle' });
await page.locator('canvas').waitFor({ state: 'visible', timeout: 10000 });
await screenshot('iteration-04-help-mobile.jpg', 78);

await page.locator('#help-play-button').click();
await page.waitForFunction(() => window.__EVENT_HORIZON__?.getSnapshot()?.tutorialStep === 'swipe-chain');
await page.waitForTimeout(220);
await screenshot('iteration-04-tutorial-swipe-chain-mobile.jpg', 78);

await swipeNodes([1, 2, 3]);
await page.waitForFunction(() => window.__EVENT_HORIZON__?.getSnapshot()?.tutorialStep === 'tap-splitter');
await tapNode(4);
await page.waitForFunction(() => window.__EVENT_HORIZON__?.getSnapshot()?.tutorialStep === 'press-play');
await page.waitForTimeout(520);
await screenshot('iteration-04-node-tap-strategy-mobile.jpg', 80);

await page.locator('#pulse-play-button').click();
await page.waitForFunction(() => window.__EVENT_HORIZON__?.getSnapshot()?.phase === 'pulse');
await page.waitForTimeout(920);
await screenshot('iteration-04-pulse-running-mobile.jpg', 80);

await page.waitForTimeout(1300);
await tapNode(3);
await page.waitForFunction(() => window.__EVENT_HORIZON__?.getSnapshot()?.tutorialStep === 'lens');
await swipeNodes([3, 4]);
await page.waitForTimeout(180);
await screenshot('iteration-04-horizon-lens-mobile.jpg', 80);

await resetForDeadEnd();
await page.waitForFunction(() => window.__EVENT_HORIZON__?.getSnapshot()?.phase === 'ended', null, { timeout: 9000 });
await page.waitForTimeout(180);
await screenshot('iteration-04-dead-end-fix-mobile.jpg', 76);

await page.evaluate(() => window.__EVENT_HORIZON_DEBUG__?.forceCollapse());
await page.waitForTimeout(160);
await screenshot('iteration-04-end-screen-mobile.jpg', 74);

await browser.close();

for (const file of [
  'iteration-04-help-mobile.jpg',
  'iteration-04-tutorial-swipe-chain-mobile.jpg',
  'iteration-04-node-tap-strategy-mobile.jpg',
  'iteration-04-pulse-running-mobile.jpg',
  'iteration-04-horizon-lens-mobile.jpg',
  'iteration-04-dead-end-fix-mobile.jpg',
  'iteration-04-end-screen-mobile.jpg'
]) {
  console.log(`Captured docs/artifacts/${file}`);
}

```

```

async function resetForDeadEnd() {
  await page.evaluate(() => {
    window.__EVENT_HORIZON__?.restart();
    window.__EVENT_HORIZON_DEBUG__?.skipTutorial();
    window.__EVENT_HORIZON_DEBUG__?.clearLinks();
    window.__EVENT_HORIZON_DEBUG__?.addLink(1, 2);
    window.__EVENT_HORIZON_DEBUG__?.playPulse();
  });
}

async function tapNode(id) {
  const node = await nodeById(id);
  const point = await worldToScreen(node.x, node.y);
  await page.mouse.click(point.x, point.y);
}

async function swipeNodes(ids) {
  const points = [];
  for (const id of ids) {
    const node = await nodeById(id);
    points.push(await worldToScreen(node.x, node.y));
  }
  await page.mouse.move(points[0].x, points[0].y);
  await page.mouse.down();
  for (const point of points.slice(1)) {
    await page.mouse.move(point.x, point.y, { steps: 8 });
  }
  await page.mouse.up();
}

async function nodeById(id) {
  const node = await page.evaluate((nodeId) => window.__EVENT_HORIZON_DEBUG__?.getNodes().find((candidate) => candidate.id === nodeId), id);
  if (!node) {
    throw new Error(`Node ${id} not found.`);
  }
  return node;
}

async function worldToScreen(x, y) {
  const box = await page.locator('canvas').boundingBox();
  if (!box) {
    throw new Error('Canvas box unavailable.');
```

```

  }
  const scale = Math.min(box.width / WORLD_WIDTH, box.height / WORLD_HEIGHT);
  const offsetX = (box.width - WORLD_WIDTH * scale) / 2;
  const offsetY = (box.height - WORLD_HEIGHT * scale) / 2;
  return {
    x: box.x + offsetX + x * scale,
    y: box.y + offsetY + y * scale
  };
}

async function screenshot(file, quality) {
  await page.screenshot({
    path: new URL(file, outputDir).pathname,
    type: 'jpeg',
    quality,
    fullPage: false
  });
}

```

scripts/generate-iteration-04-report.mjs

```

import { execFileSync } from 'node:child_process';
import { existsSync } from 'node:fs';
import { mkdir, readFile, stat, writeFile } from 'node:fs/promises';
import { extname } from 'node:path';
import { chromium } from '@playwright/test';

const repoRoot = new URL('./', import.meta.url);
const docsDir = new URL('./docs/', import.meta.url);
const artifactsDir = new URL('artifacts/', docsDir);
const reportPath = new URL('iteration-04-report.md', docsDir);
const pdfPath = new URL('iteration-04-report.pdf', docsDir);
const testResultsPath = new URL('artifacts/iteration-04-test-results.txt', docsDir);
const baseRef = process.env.REPORT_BASE_REF ?? 'origin/main';
const screenshots = [
  ['Help and tutorial entry', 'iteration-04-help-mobile.jpg'],
  ['Swipe-chain tutorial', 'iteration-04-tutorial-swipe-chain-mobile.jpg'],
  ['Node tap strategy', 'iteration-04-node-tap-strategy-mobile.jpg'],
  ['Pulse running', 'iteration-04-pulse-running-mobile.jpg'],
  ['Horizon Lens rescue', 'iteration-04-horizon-lens-mobile.jpg'],
  ['Dead-end fix prompt', 'iteration-04-dead-end-fix-mobile.jpg'],
  ['End screen', 'iteration-04-end-screen-mobile.jpg']
];

await mkdir(docsDir, { recursive: true });

const changedFiles = await collectChangedFiles();
const sourceFiles = changedFiles.filter((file) => isTextSource(file));
const binaryFiles = changedFiles.filter((file) => !isTextSource(file));
const testResults = existsSync(testResultsPath)
  ? await readFile(testResultsPath, 'utf8')
  : 'Test result log was not present when this report was generated.';
const diffStat = git(['diff', '--stat', `${baseRef}...HEAD`] || git(['diff', '--stat']));
const trackedDiff = [git(['diff', '--cached', '--no-ext-diff', '--unified=2']), git(['diff', '--no-ext-diff', '--unified=2'])]
  .filter(Boolean)
  .join('\n');

const markdown = await buildMarkdown(sourceFiles, binaryFiles, testResults, diffStat, trackedDiff);
await writeFile(reportPath, markdown);

const browser = await chromium.launch({ channel: 'chrome' });
const page = await browser.newPage();
await page.setContent(await buildHtml(markdown, sourceFiles), { waitUntil: 'load' });
await page.pdf({
  path: pdfPath.pathname,
  format: 'Letter',
  printBackground: true,
  preferCSSPageSize: true,
  margin: { top: '0.42in', right: '0.38in', bottom: '0.42in', left: '0.38in' }
});
await browser.close();

const size = await stat(pdfPath);
console.log('Wrote docs/iteration-04-report.md');
console.log('Wrote docs/iteration-04-report.pdf (${Math.round(size.size / 1024)} KiB)');

async function collectChangedFiles() {
  const names = new Set([
    ...lines(git(['diff', `${baseRef}...HEAD`, '--name-only'])),
    ...lines(git(['diff', '--cached', '--name-only'])),
    ...lines(git(['diff', '--name-only'])),
    ...lines(git(['ls-files', '--others', '--exclude-standard']))
  ]);
  names.delete('docs/iteration-04-report.md');
  names.delete('docs/iteration-04-report.pdf');
  return [...names].sort();
}

```



```

async function buildMarkdown(sourceFiles, binaryFiles, testResults, diffStat, trackedDiff) {
  const sourceBlocks = [];
  for (const file of sourceFiles) {
    const content = await readFile(new URL(file, repoRoot), 'utf8');
    sourceBlocks.push(
      `### ${file}\n\n\`${languageFor(file)}\n${content.replaceAll('\`', '\\`')}\n\``);
  }

  return `# Event Horizon Iteration 04 Report

## Summary

Iteration 04 focuses on playability. The core grammar is now visible and teachable: swipe through nodes to draw a chain, tap nodes to tune strategy, press Play, tap the pulse's next node to stabilize it, and swipe a Horizon Lens bridge to rescue the run.

## Diagnosis Of Iteration 03 Confusion

Iteration 03 introduced the Pulse Chain pivot, but it still asked players to infer too much. Link placement was possible, but the first move was not obvious, node types were mostly color-coded, tap had little strategic meaning, and failure did not explain what to fix. Iteration 04 turns those missing ideas into tutorial steps and direct feedback.

## New Tap/Swipe/Strategy Grammar

- Build: swipe through several nodes to create a directional chain automatically.
- Build: tap one node, then another node to create a precise link.
- Build: tap selected special nodes to prime Energy, cycle Delay timing, or aim Splitters.
- Run: tap the next node shortly before pulse arrival to stabilize it for score and dark-energy gain.
- Rescue: swipe between two nodes during playback to create a temporary Horizon Lens bridge.
- Strategy: hit Energy nodes, use Delay timing, branch with Splitters, avoid dead ends, and build loops.

## Exact Files Changed

${[...sourceFiles, ...binaryFiles].map((file) => `~ ${file}`).join('\n')}

## Diff Summary

\`\`\`text
${(diffStat || 'No diff stat was available.').trim()}
\`\`\`

## Tests Run And Results

\`\`\`text
${testResults.trim()}
\`\`\`

## Screenshots

${screenshots.map(([, file]) => `~ docs/artifacts/${file}`).join('\n')}

## Known Limitations

- Playwright mobile simulation passed; no physical phone was manually tested in this run.
- Tutorial is intentionally hand-tuned for \`${tutorial-001}\`; future seeds still use simple generated layouts.
- Horizon Lens is still implemented as a temporary bridge, not freeform pulse deflection.
- Splitter aiming is useful and visible, but the strategy model can still be tuned further with player testing.
- Sound, haptics, and richer retry analytics remain future work.

## Next Recommended Iteration

- Test on real iPhone and Android hardware and tune touch radii and tutorial timing.
- Add audio/haptic feedback for chain creation, stabilization, lens creation, and dead ends.
- Add a replay viewer that visually replays chain swipes, taps, and Horizon Lens inputs.
- Add a small "why this chain is good" animation after successful loops.

## Full Diffs For Tracked Changes

\`\`\`diff
${trackedDiff.trim() || 'No tracked working-tree diff was present. Untracked files are included in full-source sections below.'}
\`\`\`

## Full Source Code For Changed Text Files

${sourceBlocks.join('\n\n')}
`;
}

async function buildHtml(markdown, sourceFiles) {
  const images = await Promise.all(screenshots.map(async ([label, file]) => {
    const url = new URL(file, artifactsDir);
    if (!existsSync(url)) {
      return '';
    }
    const bytes = await readFile(url);
    return `<figure><figcaption>${escapeHtml(label)}</figcaption></figure>`;
  }));

  return `<!doctype html>
<html lang="en-US">
<head>
  <meta charset="utf-8" />
  <title>Event Horizon Iteration 04 Report</title>
  <style>
    @page { size: Letter; margin: 0.42in 0.38in; }
    * { box-sizing: border-box; }
    body { margin: 0; color: #15202b; font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif; font-size: 10px; line-height: 1.34; }
    h1, h2, h3 { color: #07111f; line-height: 1.12; margin: 0.58rem 0 0.28rem; break-after: avoid; page-break-after: avoid; }
    h1 { font-size: 21px; border-bottom: 2px solid #2f7eaf; padding-bottom: 7px; }
    h2 { font-size: 14.5px; }
    h3 { font-size: 11px; }
    p { margin: 0 0 0.4rem; }
    ul, ol { margin: 0 0 0.5rem 1.08rem; padding: 0; }
    li { margin: 0.08rem 0; }
    code { font-family: "SFMono-Regular", Menlo, Consolas, monospace; font-size: 7.8px; }
    pre { margin: 0.32rem 0 0.62rem; padding: 7px; white-space: pre-wrap; overflow-wrap: anywhere; word-break: break-word; background: #f3f6f9; border: 1px solid #d9e2ea; border-radius: 5px; font-family: "SFMono-Regular", Menlo, Consolas, monospace; font-size: 6.2px; line-height: 1.18; break-inside: auto; page-break-inside: auto; }
    .cover { background: #07111f; color: #eef9ff; border-radius: 8px; padding: 17px; margin-bottom: 11px; break-inside: avoid; }
    .cover h1 { color: #fff; border-color: #80e3ff; margin-top: 0; }
    .cover p { color: #cbeaf4; margin-bottom: 0; }
    .gallery { display: grid; grid-template-columns: repeat(3, 1fr); gap: 8px; margin: 8px 0 12px; break-inside: avoid; }
    .figure { margin: 0; break-inside: avoid; }
    img { display: block; width: 100%; max-height: 220px; object-fit: contain; border: 1px solid #d9e2ea; border-radius: 6px; background: #03040a; }
    figcaption { margin-top: 3px; color: #526171; font-size: 8.4px; text-align: center; }
    .source-index { columns: 2; column-gap: 22px; }
  </style>
</head>
<body>
  <section class="cover"><h1>Event Horizon Iteration 04 Report</h1><p>Playability, tutorial clarity, and tap/swipe/strategy grammar.</p></section>
  <section><h2>Screenshots</h2><div class="gallery">${images.join('')}</div></section>
  <section><h2>Changed Source Index</h2><ul class="source-index">${sourceFiles.map((file) => `<li>${escapeHtml(file)}</li>`).join('')}</ul></section>
  ${markdownToHtml(markdown)}
</body>
</html>`;
}

function markdownToHtml(markdown) {
  const rows = markdown.split('\n');
  const html = [];
  let inCode = false;
  let codeLines = [];
  let listOpen = false;
  for (const row of rows) {
    if (row.startsWith('\`\`\`')) {
      if (inCode) {
        html.push(`<pre>${escapeHtml(codeLines.join('\n'))}</pre>`);

```

```

    codeLines = [];
  }
  inCode = !inCode;
  continue;
}
if (inCode) {
  codeLines.push(row);
  continue;
}
if (row.startsWith('- ')) {
  if (!listOpen) {
    html.push('<ul>');
    listOpen = true;
  }
  html.push('<li>${inlineMarkdown(row.slice(2))}</li>');
  continue;
}
if (listOpen) {
  html.push('</ul>');
  listOpen = false;
}
if (row.startsWith('# ')) {
  html.push('<h1>${inlineMarkdown(row.slice(2))}</h1>');
} else if (row.startsWith('## ')) {
  html.push('<h2>${inlineMarkdown(row.slice(3))}</h2>');
} else if (row.startsWith('### ')) {
  html.push('<h3>${inlineMarkdown(row.slice(4))}</h3>');
} else if (row.trim()) {
  html.push('<p>${inlineMarkdown(row)}</p>');
}
}
if (listOpen) {
  html.push('</ul>');
}
return html.join('\n');
}

function inlineMarkdown(value) {
  return escapeHtml(value)
    .replace(/`([^\`]+)`/g, '<code>$1</code>')
    .replace(/\[([^\]]+)\]\([([^\]]+)\)/g, '<a href="$2">$1</a>');
}

function escapeHtml(value) {
  return String(value)
    .replaceAll('&', '&amp;')
    .replaceAll('<', '&lt;')
    .replaceAll('>', '&gt;')
    .replaceAll('"', '&quot;')
    .replaceAll("'", '&#39;');
}

function git(args) {
  try {
    return execFileSync('git', args, { cwd: repoRoot, encoding: 'utf8', maxBuffer: 1024 * 1024 * 32 });
  } catch {
    return '';
  }
}

function lines(value) {
  return value.split('\n').map((line) => line.trim()).filter(Boolean);
}

function isTextSource(file) {
  if (file.startsWith('docs/artifacts/') || file.endsWith('.pdf')) {
    return false;
  }
  return ['.css', '.html', '.js', '.json', '.md', '.mjs', '.ts', '.tsx', '.txt', '.yaml'].includes(extername(file));
}

function languageFor(file) {
  const extension = extername(file);
  if (extension === '.ts' || extension === '.tsx') {
    return 'ts';
  }
  if (extension === '.mjs' || extension === '.js') {
    return 'js';
  }
  if (extension === '.css') {
    return 'css';
  }
  if (extension === '.html') {
    return 'html';
  }
  if (extension === '.json') {
    return 'json';
  }
  if (extension === '.yaml') {
    return 'yaml';
  }
  if (extension === '.md') {
    return 'md';
  }
  return 'text';
}

```

scripts/test-score-submit.mjs

```

import scoreSubmit from '../netlify/functions/score-submit.mjs';

const replay = {
  version: 1,
  seed: 'eh-2026-05-29-alpha',
  startedAt: 1780051200000,
  survivalMs: 67421,
  score: 1280,
  energyCaptured: 87,
  maxStreak: 24,
  tapEvents: [],
  swipeEvents: [],
  phaseTransitions: []
};

const pulseReplay = {
  version: 1,
  mode: 'pulse-chain',
  seed: 'tutorial-001',
  startedAt: 1780185600000,
  buildInputs: [
    {
      t: 0,
      kind: 'chainSwipe',
      nodeIds: [1, 2, 3],
      path: [
        { x: 250, y: 1388, t: 0 },
        { x: 420, y: 1165, t: 90 },
        { x: 635, y: 1010, t: 180 }
      ]
    }
  ],
},

```

```

    { t: 300, kind: 'nodeTap', nodeId: 4, action: 'splitter' },
    { t: 620, kind: 'play' }
  ],
  liveInputs: [
    { t: 2200, kind: 'stabilize', nodeId: 3, rating: 'stabilized', success: true },
    {
      t: 1600,
      kind: 'lens',
      path: [
        { x: 835, y: 1215, t: 0 },
        { x: 700, y: 1410, t: 120 }
      ],
      fromId: 6,
      toId: 8,
      success: true
    }
  ],
  result: {
    score: 620,
    survivalMs: 12120,
    maxMultiplier: 2,
    loopsCompleted: 1,
    linksUsed: 5,
    bestChainLength: 5,
    energyNodesHit: 2,
    stabilized: false,
    collapsed: false
  },
  stepHash: '04ce9b1a'
};

const results = [];
for (const payload of [replay, pulseReplay]) {
  const response = await scoreSubmit(
    new Request('https://event-horizon.test/.netlify/functions/score-submit', {
      method: 'POST',
      headers: { 'content-type': 'application/json' },
      body: JSON.stringify(payload)
    })
  );
  const body = await response.json();
  const expectedScore = payload.mode === 'pulse-chain' ? payload.result.score : payload.score;
  if (response.status !== 200 || !body.ok || body.score !== expectedScore) {
    console.error(JSON.stringify({ status: response.status, body }, null, 2));
    process.exit(1);
  }
  results.push({ status: response.status, body });
}

console.log(JSON.stringify(results, null, 2));

```

src/game/pulse/PulseGeometry.ts

```

import { clamp, distanceSquared, distanceToSegmentSquared } from '../math';
import type { PulseNode } from '../PulseTypes';
import type { WorldPoint } from '../gestures';

export function simplifyPath<T> extends WorldPoint<(points: readonly T[], maxPoints = 24): T[] {
  if (points.length <= maxPoints) {
    return points.map((point) => point);
  }
  const sampled: T[] = [];
  for (let index = 0; index < maxPoints; index += 1) {
    sampled.push(points[Math.round((index / (maxPoints - 1)) * (points.length - 1))]);
  }
  return sampled;
}

export function resamplePath(points: readonly WorldPoint[], spacing = 38): WorldPoint[] {
  if (points.length <= 1) {
    return points.map((point) => ({ ...point }));
  }
  const result: WorldPoint[] = [{ ...points[0] }];
  let carry = 0;
  for (let index = 1; index < points.length; index += 1) {
    const previous = points[index - 1];
    const point = points[index];
    const segmentLength = Math.max(0.001, Math.hypot(point.x - previous.x, point.y - previous.y));
    let distance = spacing - carry;
    while (distance <= segmentLength) {
      const t = distance / segmentLength;
      result.push({
        x: previous.x + (point.x - previous.x) * t,
        y: previous.y + (point.y - previous.y) * t
      });
      distance += spacing;
    }
    carry = segmentLength - (distance - spacing);
  }
  const last = points[points.length - 1];
  result.push({ ...last });
  return result;
}

export function smoothPathQuadratic(points: readonly WorldPoint[]): string {
  if (points.length === 0) {
    return '';
  }
  if (points.length === 1) {
    return `M ${points[0].x} ${points[0].y}`;
  }
  const commands = ['M ${points[0].x} ${points[0].y}`];
  for (let index = 1; index < points.length - 1; index += 1) {
    const point = points[index];
    const next = points[index + 1];
    const midpoint = midpointOf(point, next);
    commands.push(`Q ${point.x} ${point.y} ${midpoint.x} ${midpoint.y}`);
  }
  const last = points[points.length - 1];
  commands.push(`L ${last.x} ${last.y}`);
  return commands.join(' ');
}

export function distancePointToSegment(point: WorldPoint, start: WorldPoint, end: WorldPoint): number {
  return Math.sqrt(distanceToSegmentSquared(point.x, point.y, start.x, start.y, end.x, end.y));
}

export function distanceSegmentToSegment(a: WorldPoint, b: WorldPoint, c: WorldPoint, d: WorldPoint): number {
  if (segmentsIntersect(a, b, c, d)) {
    return 0;
  }
  return Math.sqrt(
    Math.min(
      distanceToSegmentSquared(a.x, a.y, c.x, c.y, d.x, d.y),
      distanceToSegmentSquared(b.x, b.y, c.x, c.y, d.x, d.y),
      distanceToSegmentSquared(c.x, c.y, a.x, a.y, b.x, b.y),
      distanceToSegmentSquared(d.x, d.y, a.x, a.y, b.x, b.y)
    )
  );
}

```

```

export function nearestNodeToPath(
  nodes: readonly PulseNode[],
  path: readonly WorldPoint[],
  radius = 118,
  excludedId?: number
): PulseNode | undefined {
  let best: PulseNode | undefined;
  let bestDistance = radius;
  for (const node of nodes) {
    if (node.id === excludedId) {
      continue;
    }
    const distance = distanceNodeToPath(node, path);
    if (distance <= bestDistance) {
      best = node;
      bestDistance = distance;
    }
  }
  return best;
}

export function nearestTwoNodesToPath(
  nodes: readonly PulseNode[],
  path: readonly WorldPoint[],
  radius = 126
): [PulseNode, PulseNode] | undefined {
  const ranked = nodes
    .map((node) => ({ node, distance: distanceNodeToPath(node, path) }))
    .filter((entry) => entry.distance <= radius)
    .sort((a, b) => a.distance - b.distance);
  if (ranked.length < 2) {
    return undefined;
  }
  return [ranked[0].node, ranked[1].node];
}

export function pathCrossesNodeRadius(path: readonly WorldPoint[], node: PulseNode, radius = node.radius + 44): boolean {
  return distanceNodeToPath(node, path) <= radius;
}

export function nodesCrossedByPath(
  nodes: readonly PulseNode[],
  path: readonly WorldPoint[],
  radius = 76
): PulseNode[] {
  const crossed = nodes
    .map((node) => {
      const hit = firstPathHit(node, path, radius + node.radius * 0.35);
      return hit === undefined ? undefined : { node, order: hit };
    })
    .filter((entry): entry is { node: PulseNode; order: number } => entry !== undefined)
    .sort((a, b) => a.order - b.order)
    .map((entry) => entry.node);
  const result: PulseNode[] = [];
  for (const node of crossed) {
    if (result[result.length - 1]?.id !== node.id) {
      result.push(node);
    }
  }
  return result;
}

export function distanceNodeToPath(node: PulseNode, path: readonly WorldPoint[]): number {
  if (path.length === 0) {
    return Number.POSITIVE_INFINITY;
  }
  if (path.length === 1) {
    return Math.sqrt(distanceSquared(node.x, node.y, path[0].x, path[0].y));
  }
  let best = Number.POSITIVE_INFINITY;
  for (let index = 1; index < path.length; index += 1) {
    const previous = path[index - 1];
    const point = path[index];
    best = Math.min(best, distancePointToSegment(node, previous, point));
  }
  return best;
}

function firstPathHit(node: PulseNode, path: readonly WorldPoint[], radius: number): number | undefined {
  if (path.length === 0) {
    return undefined;
  }
  let traveled = 0;
  if (path.length === 1) {
    return Math.sqrt(distanceSquared(node.x, node.y, path[0].x, path[0].y)) <= radius ? 0 : undefined;
  }
  for (let index = 1; index < path.length; index += 1) {
    const previous = path[index - 1];
    const point = path[index];
    const segmentLength = Math.max(0.001, Math.hypot(point.x - previous.x, point.y - previous.y));
    const distance = distancePointToSegment(node, previous, point);
    if (distance <= radius) {
      return traveled + segmentLength * projectionT(node, previous, point);
    }
    traveled += segmentLength;
  }
  return undefined;
}

function projectionT(point: WorldPoint, start: WorldPoint, end: WorldPoint): number {
  const dx = end.x - start.x;
  const dy = end.y - start.y;
  const lengthSq = dx * dx + dy * dy;
  if (lengthSq <= 0.000001) {
    return 0;
  }
  return clamp(((point.x - start.x) * dx + (point.y - start.y) * dy) / lengthSq, 0, 1);
}

export function curvedLinkPath(from: WorldPoint, to: WorldPoint, bend = 0.12): WorldPoint[] {
  const mid = midpointOf(from, to);
  const dx = to.x - from.x;
  const dy = to.y - from.y;
  const length = Math.max(1, Math.hypot(dx, dy));
  const blackHolePull = 0.18;
  const control = {
    x: mid.x - (dy / length) * length * bend + (540 - mid.x) * blackHolePull,
    y: mid.y + (dx / length) * length * bend + (845 - mid.y) * blackHolePull
  };
  return [from, control, to];
}

export function pointOnQuadratic(a: WorldPoint, b: WorldPoint, c: WorldPoint, t: number): WorldPoint {
  const clamped = clamp(t, 0, 1);
  const inv = 1 - clamped;
  return {
    x: inv * inv * a.x + 2 * inv * clamped * b.x + clamped * clamped * c.x,
    y: inv * inv * a.y + 2 * inv * clamped * b.y + clamped * clamped * c.y
  };
}

function midpointOf(a: WorldPoint, b: WorldPoint): WorldPoint {

```

```

    return {
      x: (a.x + b.x) / 2,
      y: (a.y + b.y) / 2
    };
  }

function segmentsIntersect(a: WorldPoint, b: WorldPoint, c: WorldPoint, d: WorldPoint): boolean {
  const o1 = orientation(a, b, c);
  const o2 = orientation(a, b, d);
  const o3 = orientation(c, d, a);
  const o4 = orientation(c, d, b);
  if (o1 !== o2 && o3 !== o4) {
    return true;
  }
  return false;
}

function orientation(a: WorldPoint, b: WorldPoint, c: WorldPoint): -1 | 0 | 1 {
  const value = (b.y - a.y) * (c.x - b.x) - (b.x - a.x) * (c.y - b.y);
  if (Math.abs(value) < 0.000001) {
    return 0;
  }
  return value > 0 ? 1 : -1;
}

```

src/game/pulse/PulseInputController.ts

```

import { distanceSquared } from '../math';
import { InputHandler, type GesturePoint, type SwipeGesture, type WorldPoint } from '../InputHandler';
import type { PulseSimulation } from './PulseSimulation';
import type { PulseInputResult, PulseNode } from './PulseTypes';

export interface PulseInputViewState {
  selectedNodeId?: number;
  nearestNodeId?: number;
  previewFromId?: number;
  previewPoint?: WorldPoint;
  liveGesture: readonly WorldPoint[];
  lastResult: PulseInputResult;
}

export class PulseInputController {
  private input?: InputHandler;
  private selectedNodeId: number | undefined;
  private previewFromId: number | undefined;
  private previewPoint: WorldPoint | undefined;
  private nearestNodeId: number | undefined;
  private liveGesture: WorldPoint[] = [];
  private lastResult: PulseInputResult = { ok: true, kind: 'none', message: 'CONNECT TWO NODES' };
  private debug = false;

  constructor(
    private readonly target: HTMLElement,
    private readonly sim: PulseSimulation,
    private readonly screenToWorld: (clientX: number, clientY: number) => WorldPoint
  ) {}

  start(): void {
    this.input = new InputHandler(this.target, {
      screenToWorld: this.screenToWorld,
      onTap: (point) => this.handleTap(point),
      onSwipe: (gesture) => this.handleSwipe(gesture),
      onGesturePreview: (points) => this.handlePreview(points),
      onGestureEnd: () => this.handleGestureEnd()
    });
    this.input.setDebug(this.debug);
  }

  destroy(): void {
    this.input?.destroy();
  }

  setDebug(enabled: boolean): void {
    this.debug = enabled;
    this.input?.setDebug(enabled);
  }

  getViewState(): PulseInputViewState {
    return {
      selectedNodeId: this.selectedNodeId,
      nearestNodeId: this.nearestNodeId,
      previewFromId: this.previewFromId,
      previewPoint: this.previewPoint ? { ...this.previewPoint } : undefined,
      liveGesture: this.liveGesture.map((point) => ({ ...point })),
      lastResult: this.lastResult
    };
  }

  getDebugInfo() {
    return this.input?.getDebugInfo();
  }

  clearSelection(): void {
    this.selectedNodeId = undefined;
    this.previewFromId = undefined;
    this.previewPoint = undefined;
    this.nearestNodeId = undefined;
    this.liveGesture = [];
    this.sim.selectNode(undefined);
  }

  private handleTap(point: GesturePoint): void {
    const snapshot = this.sim.getSnapshot();
    if (snapshot.phase !== 'build') {
      const pulseNode = this.nearestNode(point, 86);
      if (snapshot.phase === 'pulse' && pulseNode) {
        this.lastResult = this.sim.stabilizeNode(pulseNode.id);
      }
      return;
    }
    const node = this.nearestNode(point, 76);
    if (!node) {
      this.clearSelection();
      this.lastResult = this.sim.removeLinkNear(point);
      return;
    }
    if (snapshot.tutorialStep === 'tap-splitter' && node.type === 'splitter') {
      this.lastResult = this.sim.tapNode(node.id);
      this.selectedNodeId = undefined;
      return;
    }
    if (this.selectedNodeId === undefined) {
      this.selectedNodeId = node.id;
      this.lastResult = this.sim.selectNode(node.id);
      return;
    }
    if (this.selectedNodeId === node.id) {
      this.lastResult = this.sim.tapNode(node.id);
      this.selectedNodeId = undefined;
      this.previewFromId = undefined;
    }
  }

```

```

    this.previewPoint = undefined;
    return;
  }
  this.lastResult = this.sim.addLink(this.selectedNodeId, node.id);
  this.selectedNodeId = undefined;
}

private handleSwipe(gesture: SwipeGesture): void {
  const snapshot = this.sim.getSnapshot();
  if (snapshot.phase === 'build') {
    this.lastResult = this.sim.applyChainSwipe(gesture.points);
    this.selectedNodeId = undefined;
  } else if (snapshot.phase === 'pulse') {
    this.lastResult = this.sim.applyLens(gesture.points);
  }
  this.liveGesture = [];
  this.previewFromId = undefined;
  this.previewPoint = undefined;
}

private handlePreview(points: readonly GesturePoint[]): void {
  const latest = points[points.length - 1];
  if (!latest) {
    return;
  }
  this.liveGesture = points.map((point) => ({ x: point.x, y: point.y }));
  this.nearestNodeId = this.nearestNode(latest, 92)?.id;
  const first = points[0];
  if (this.sim.getSnapshot().phase === 'build' && first) {
    this.previewFromId = this.nearestNode(first, 88)?.id;
    this.previewPoint = { x: latest.x, y: latest.y };
  }
}

private handleGestureEnd(): void {
  this.liveGesture = [];
  this.previewFromId = undefined;
  this.previewPoint = undefined;
}

private nearestNode(point: WorldPoint, maxDistance: number): PulseNode | undefined {
  let best: PulseNode | undefined;
  let bestDistance = maxDistance * maxDistance;
  for (const node of this.sim.getNodes()) {
    const distance = distanceSquared(point.x, point.y, node.x, node.y);
    if (distance <= bestDistance) {
      best = node;
      bestDistance = distance;
    }
  }
  return best;
}
}

```

src/game/pulse/PulseLevelGenerator.ts

```

import { BLACK_HOLE_X, BLACK_HOLE_Y, WORLD_HEIGHT, WORLD_WIDTH } from '../constants';
import { createSeededRandom } from '../rng';
import type { PulseLevel, PulseNode, PulseNodeType } from './PulseTypes';

export function getDailyPulseSeed(date = new Date()): string {
  const year = date.getUTCFullYear();
  const month = String(date.getUTCMonth() + 1).padStart(2, '0');
  const day = String(date.getUTCDate()).padStart(2, '0');
  return `daily-${year}-${month}-${day}`;
}

export function generatePulseLevel(seed: string): PulseLevel {
  const rng = createSeededRandom(`pulse-chain-${seed}`);
  const firstSeed = seed === 'tutorial-001' || seed === 'daily-2026-05-30' || seed === 'tutorial' || seed === 'eh-pulse-alpha';
  const baseNodes = firstSeed ? tutorialNodes() : generatedNodes(rng);
  return {
    seed,
    nodes: baseNodes,
    sourceId: 1,
    linkBudget: 6,
    targetScore: 1800,
    targetSurvivalMs: 45000
  };
}

function tutorialNodes(): PulseNode[] {
  const specs: Array<[PulseNodeType, number, number, number, string]> = [
    ['source', 250, 1300, 0, 'SOURCE'],
    ['energy', 420, 1165, 1, '+100'],
    ['delay', 635, 1010, 1, 'DELAY'],
    ['splitter', 770, 775, 2, 'SPLIT'],
    ['energy', 515, 620, 2, '+100'],
    ['energy', 835, 1215, 2, '+100'],
    ['conduit', 305, 805, 2, 'CONDUIT'],
    ['conduit', 705, 1412, 2, 'CONDUIT'],
    ['delay', 252, 1085, 1, 'DELAY'],
    ['splitter', 910, 935, 2, 'SPLIT'],
    ['energy', 485, 1515, 2, '+100'],
    ['conduit', 805, 545, 2, 'CONDUIT']
  ];
  return specs.map(([type, x, y, ring, label], index) => makeNode(index + 1, type, x, y, ring, label));
}

function generatedNodes(rng: () => number): PulseNode[] {
  const nodeTypes: PulseNodeType[] = [
    'source',
    'energy',
    'delay',
    'splitter',
    'energy',
    'conduit',
    'conduit',
    'energy',
    'conduit',
    'delay',
    'splitter',
    'energy',
    'conduit'
  ];
  const nodes: PulseNode[] = [];
  for (let index = 0; index < nodeTypes.length; index += 1) {
    const type = nodeTypes[index];
    if (type === 'source') {
      nodes.push(makeNode(1, 'source', 230 + rng() * 130, 1320 + rng() * 190, 0, 'SOURCE'));
      continue;
    }
    const ring = index < 6 ? 1 : 2;
    const angle = -2.55 + index * 0.53 + (rng() - 0.5) * 0.22;
    const radiusX = ring === 1 ? 280 + rng() * 48 : 420 + rng() * 78;
    const radiusY = ring === 1 ? 410 + rng() * 46 : 570 + rng() * 84;
    const x = clampToPlayfield(BLACK_HOLE_X + Math.cos(angle) * radiusX);
    const y = clampToPlayfieldY(BLACK_HOLE_Y + Math.sin(angle) * radiusY + 120);
    nodes.push(makeNode(index + 1, type, x, y, ring, labelFor(type)));
  }
}

```

```

    }
    return separateNodes(nodes);
}

function separateNodes(nodes: PulseNode[]): PulseNode[] {
    for (let pass = 0; pass < 5; pass += 1) {
        for (let a = 0; a < nodes.length; a += 1) {
            for (let b = a + 1; b < nodes.length; b += 1) {
                const left = nodes[a];
                const right = nodes[b];
                const dx = right.x - left.x;
                const dy = right.y - left.y;
                const distance = Math.max(1, Math.hypot(dx, dy));
                if (distance >= 132) {
                    continue;
                }
                const push = (132 - distance) / 2;
                left.x = clampToPlayfield(left.x - (dx / distance) * push);
                left.y = clampToPlayfieldY(left.y - (dy / distance) * push);
                right.x = clampToPlayfield(right.x + (dx / distance) * push);
                right.y = clampToPlayfieldY(right.y + (dy / distance) * push);
            }
        }
    }
    return nodes;
}

function makeNode(id: number, type: PulseNodeType, x: number, y: number, ring: number, label: string): PulseNode {
    return {
        id,
        type,
        x,
        y,
        ring,
        label,
        radius: type === 'source' ? 54 : type === 'splitter' ? 50 : 46,
        activationMs: 0,
        scoreCooldownMs: 0,
        primed: false,
        delayLevel: 1,
        splitterPriority: 0,
        stabilizedMs: 0
    };
}

function labelFor(type: PulseNodeType): string {
    if (type === 'energy') {
        return 'ENERGY';
    }
    if (type === 'delay') {
        return 'DELAY';
    }
    if (type === 'splitter') {
        return 'SPLIT';
    }
    return 'CONDUIT';
}

function clampToPlayfield(x: number): number {
    return Math.max(126, Math.min(WORLD_WIDTH - 126, x));
}

function clampToPlayfieldY(y: number): number {
    return Math.max(260, Math.min(WORLD_HEIGHT - 330, y));
}

```

src/game/pulse/PulseMode.ts

```

import { Application, type Renderer } from 'pixi.js';
import { WORLD_HEIGHT, WORLD_WIDTH } from '../constants';
import { FixedStepLoop } from '../FixedStepLoop';
import type { WorldPoint } from '../gestures';
import { clamp } from '../math';
import { createSharePoster, type PosterFrame } from '../posterizer';
import { submitScore } from '../scoreClient';
import { PulseInputController } from './PulseInputController';
import { getDailyPulseSeed } from './PulseLevelGenerator';
import { PulseRenderer } from './PulseRenderer';
import { PulseSimulation } from './PulseSimulation';
import type { PulseReplayPayload, PulseSnapshot } from './PulseTypes';

export interface PulseModeOptions {
    seed?: string;
    startedAt: number;
}

export class PulseMode {
    private readonly app = new Application<Renderer>();
    private readonly sim: PulseSimulation;
    private readonly loop: FixedStepLoop;
    private renderer?: PulseRenderer;
    private input?: PulseInputController;
    private scale = 1;
    private offsetX = 0;
    private offsetY = 0;
    private paused = false;
    private debug = false;
    private scoreSubmitted = false;
    private sampleCooldownMs = 0;
    private frameSamples: PosterFrame[] = [];

    constructor(
        private readonly root: HTMLElement,
        options: PulseModeOptions
    ) {
        const seed = options.seed ?? getDailyPulseSeed();
        this.sim = new PulseSimulation({ seed, startedAt: options.startedAt });
        this.loop = new FixedStepLoop(
            (dtMs) => this.step(dtMs),
            () => this.render()
        );
    }

    async start(): Promise<void> {
        await this.app.init({
            autoDensity: true,
            autoStart: false,
            backgroundAlpha: 0,
            clearBeforeRender: true,
            hello: false,
            preference: 'webgl',
            preserveDrawingBuffer: true,
            powerPreference: 'high-performance',
            resizeTo: this.root,
            resolution: Math.min(window.devicePixelRatio || 1, 2)
        });
        this.root.appendChild(this.app.canvas);
        this.renderer = new PulseRenderer(this.app.stage);
        this.input = new PulseInputController(this.app.canvas, this.sim, (clientX, clientY) => this.screenToWorld(clientX, clientY));
        this.input.setDebug(this.debug);
    }

```

```

    this.input.start();
    window.addEventListener('resize', this.resize);
    this.resize();
    this.loop.start();
}

destroy(): void {
    this.loop.stop();
    this.input7.destroy();
    this.renderer?.destroy();
    window.removeEventListener('resize', this.resize);
    this.app.destroy(true, { children: true, texture: true });
}

restart(): void {
    this.sim.reset();
    this.scoreSubmitted = false;
    this.frameSamples = [];
    this.sampleCooldownMs = 0;
    this.input7.clearSelection();
    this.loop.resetClock();
}

setPaused(paused: boolean): void {
    this.paused = paused;
    if (!paused) {
        this.loop.resetClock();
    }
}

setInputDebug(enabled: boolean): void {
    this.debug = enabled;
    this.input7.setDebug(enabled);
}

getMode(): 'pulse-chain' {
    return 'pulse-chain';
}

getSnapshot(): PulseSnapshot {
    return this.sim.getSnapshot();
}

getReplayPayload(): PulseReplayPayload {
    return this.sim.getReplayPayload();
}

getNodes() {
    return this.sim.getNodes();
}

getLinks() {
    return this.sim.getLinks();
}

getPulses() {
    return this.sim.getPulses();
}

getLastInputResult() {
    return this.sim.getLastInputResult();
}

addLink(fromId: number, toId: number) {
    return this.sim.addLink(fromId, toId);
}

clearLinks() {
    return this.sim.clearLinks();
}

fixChain() {
    this.scoreSubmitted = false;
    this.input7.clearSelection();
    this.loop.resetClock();
    return this.sim.fixChain();
}

undo() {
    return this.sim.undo();
}

playPulse() {
    return this.sim.playPulse();
}

simulateLens(points: readonly WorldPoint[]) {
    return this.sim.applyLens(points.map((point, index) => ({ ...point, t: index * 16 })));
}

simulateChainSwipe(nodeIds: readonly number[]) {
    const points: Array<WorldPoint & { t: number }> = [];
    for (const id of nodeIds) {
        const node = this.sim.getNodes().find((candidate) => candidate.id === id);
        if (node) {
            points.push({ x: node.x, y: node.y, t: points.length * 80 });
        }
    }
    return this.sim.applyChainSwipe(points);
}

startTutorial(): void {
    this.sim.startTutorial();
    this.scoreSubmitted = false;
    this.loop.resetClock();
}

skipTutorial(): void {
    this.sim.skipTutorial();
}

getTutorialStep() {
    return this.sim.getTutorialStep();
}

analyzeChain() {
    return this.sim.analyzeChain();
}

getSuggestedFixes() {
    return this.sim.getSuggestedFixes();
}

primeNode(id: number) {
    return this.sim.primeNode(id);
}

cycleNode(id: number) {
    return this.sim.cycleNode(id);
}

stabilizeNode(id: number) {
    return this.sim.stabilizeNode(id);
}

```



```

}

forceBuildPhase(): void {
  this.sim.forceBuildPhase();
}

forcePulsePhase(): void {
  this.sim.forcePulsePhase();
}

forceCollapse(): void {
  this.sim.forceCollapse();
}

forceEnd(): void {
  this.forceCollapse();
}

async exportPoster(): Promise<string> {
  this.sampleFrame('current');
  const snapshot = this.sim.getSnapshot();
  const frames = this.frameSamples.length >= 3 ? this.frameSamples : this.makeFallbackFrames();
  return createSharePoster(frames, {
    score: snapshot.score,
    survivalMs: snapshot.timeMs,
    seed: snapshot.seed,
    phase: 1
  });
}

private step(dtMs: number): void {
  if (this.paused) {
    return;
  }
  const wasEnded = this.sim.getSnapshot().ended;
  this.sim.step(dtMs);
  const snapshot = this.sim.getSnapshot();
  this.sampleCooldownMs -= dtMs;
  if (this.sampleCooldownMs <= 0 && !snapshot.ended) {
    this.sampleFrame('ts{snapshot.timeMs}');
    this.sampleCooldownMs = 1300;
  }
  if (!wasEnded && snapshot.ended && !this.scoreSubmitted) {
    this.scoreSubmitted = true;
    this.saveBest(snapshot);
    void submitScore(this.sim.getReplayPayload());
  }
}

private render(): void {
  this.renderer?.render(this.sim.getSnapshot(), this.input?.getViewState() ?? {
    liveGesture: [],
    lastResult: this.sim.getLastInputResult()
  }, {
    scale: this.scale,
    offsetX: this.offsetX,
    offsetY: this.offsetY,
    debug: this.debug
  });
  this.app.render();
}

private readonly resize = (): void => {
  const rect = this.root.getBoundingClientRect();
  this.scale = Math.min(rect.width / WORLD_WIDTH, rect.height / WORLD_HEIGHT);
  const viewWidth = WORLD_WIDTH * this.scale;
  const viewHeight = WORLD_HEIGHT * this.scale;
  this.offsetX = (rect.width - viewWidth) / 2;
  this.offsetY = (rect.height - viewHeight) / 2;
};

private screenToWorld(clientX: number, clientY: number): WorldPoint {
  const rect = this.app.canvas.getBoundingClientRect();
  return {
    x: clamp((clientX - rect.left - this.offsetX) / this.scale, 0, WORLD_WIDTH),
    y: clamp((clientY - rect.top - this.offsetY) / this.scale, 0, WORLD_HEIGHT)
  };
}

private sampleFrame(label: string): void {
  try {
    this.frameSamples.push({ dataUrl: this.app.canvas.toDataURL('image/png', 0.78), label });
    if (this.frameSamples.length > 3) {
      this.frameSamples.shift();
    }
  } catch {
    this.frameSamples = this.makeFallbackFrames();
  }
}

private makeFallbackFrames(): PosterFrame[] {
  const snapshot = this.sim.getSnapshot();
  return [0, 1, 2].map((index) => ({
    dataUrl: this.makeMockFrame(snapshot, index),
    label: `pulse-${index}`
  }));
}

private makeMockFrame(snapshot: PulseSnapshot, index: number): string {
  const canvas = document.createElement('canvas');
  canvas.width = 540;
  canvas.height = 960;
  const context = canvas.getContext('2d');
  if (!context) {
    return '';
  }
  context.fillStyle = '#03040a';
  context.fillRect(0, 0, canvas.width, canvas.height);
  context.fillStyle = index === 0 ? '#16315c' : index === 1 ? '#45246d' : '#0f5b69';
  context.beginPath();
  context.arc(270, 410, 150, 0, Math.PI * 2);
  context.fill();
  context.fillStyle = '#000';
  context.beginPath();
  context.arc(270, 410, 72, 0, Math.PI * 2);
  context.fill();
  context.fillStyle = '#f7fbff';
  context.font = '800 52px system-ui';
  context.fillText(String(snapshot.score), 54, 824);
  return canvas.toDataURL('image/png', 0.78);
}

private saveBest(snapshot: PulseSnapshot): void {
  try {
    const bestScore = Number(localStorage.getItem('eventHorizon.bestScore')) ?? 0;
    const bestSurvival = Number(localStorage.getItem('eventHorizon.bestSurvivalMs')) ?? 0;
    if (snapshot.score > bestScore) {
      localStorage.setItem('eventHorizon.bestScore', String(snapshot.score));
    }
    if (snapshot.timeMs > bestSurvival) {
      localStorage.setItem('eventHorizon.bestSurvivalMs', String(snapshot.timeMs));
    }
  } catch {
  }
}

```

```

    // Ignore storage failures in private or embedded browsers.
  }
}
}

```

src/game/pulse/PulseRenderer.ts

```

import { Container, Graphics, Text, TextStyle } from 'pixi.js';
import { BLACK_HOLE_X, BLACK_HOLE_Y, MAX_ENERGY, WORLD_HEIGHT, WORLD_WIDTH } from '../constants';
import { clamp, formatTime } from '../math';
import { curvedLinkPath, pointOnQuadratic, resamplePath } from './PulseGeometry';
import type { PulseInputViewState } from './PulseInputController';
import type { PulseNode, PulseSnapshot } from './PulseTypes';
import type { WorldPoint } from '../gestures';

const NODE_COLORS = {
  source: 0x67f4ff,
  conduit: 0x9bb6ff,
  energy: 0x4dffbf,
  delay: 0xffd166,
  splitter: 0xd267ff
} as const;

export class PulseRenderer {
  private readonly world = new Container();
  private readonly background = new Graphics();
  private readonly linkLayer = new Graphics();
  private readonly templinkLayer = new Graphics();
  private readonly lensLayer = new Graphics();
  private readonly previewLayer = new Graphics();
  private readonly nodeLayer = new Graphics();
  private readonly nodeTextLayer = new Container();
  private readonly pulseLayer = new Graphics();
  private readonly blackHole = new Graphics();
  private readonly hud = new Container();
  private readonly scoreText = new Text({
    text: '0',
    style: new TextStyle({ fill: '#f7fbff', fontFamily: 'Inter, system-ui, sans-serif', fontSize: 50, fontWeight: '900' })
  });
  private readonly metaText = new Text({
    text: '',
    style: new TextStyle({ fill: '#9fe7ff', fontFamily: 'Inter, system-ui, sans-serif', fontSize: 24, fontWeight: '800' })
  });
  private readonly hintText = new Text({
    text: '',
    style: new TextStyle({
      align: 'center',
      fill: 'ffffff',
      fontFamily: 'Inter, system-ui, sans-serif',
      fontSize: 34,
      fontWeight: '900',
      stroke: { color: '#180923', width: 5 }
    })
  });
  private readonly messageText = new Text({
    text: '',
    style: new TextStyle({
      align: 'center',
      fill: '#9ffcff',
      fontFamily: 'Inter, system-ui, sans-serif',
      fontSize: 26,
      fontWeight: '900',
      stroke: { color: '#061120', width: 4 }
    })
  });
  private readonly strategyText = new Text({
    text: '',
    style: new TextStyle({
      fill: '#f7fbff',
      fontFamily: 'Inter, system-ui, sans-serif',
      fontSize: 18,
      fontWeight: '900',
      lineHeight: 23,
      stroke: { color: '#061120', width: 4 }
    })
  });
  private readonly meter = new Graphics();
  private readonly debugText = new Text({
    text: '',
    style: new TextStyle({ fill: '#dff8ff', fontFamily: 'SFMono-Regular, Menlo, monospace', fontSize: 17, fontWeight: '600' })
  });
  private readonly endText = new Text({
    text: '',
    style: new TextStyle({
      align: 'center',
      fill: 'ffffff',
      fontFamily: 'Inter, system-ui, sans-serif',
      fontSize: 40,
      fontWeight: '900',
      stroke: { color: '#12051c', width: 6 }
    })
  });
  private readonly nodeTexts = new Map<number, Text>();

  constructor(private readonly stage: Container) {
    this.stage.addChild(this.world);
    this.world.addChild(
      this.background,
      this.linkLayer,
      this.templinkLayer,
      this.blackHole,
      this.lensLayer,
      this.previewLayer,
      this.nodeLayer,
      this.nodeTextLayer,
      this.pulseLayer,
      this.hintText,
      this.hud
    );
    this.hud.addChild(this.meter, this.scoreText, this.metaText, this.strategyText, this.messageText, this.debugText, this.endText);
    this.hintText.anchor.set(0.5);
    this.hintText.position.set(WORLD_WIDTH / 2, 260);
    this.messageText.anchor.set(0.5);
    this.messageText.position.set(WORLD_WIDTH / 2, 318);
    this.scoreText.position.set(68, 68);
    this.metaText.position.set(72, 130);
    this.strategyText.position.set(68, 330);
    this.debugText.position.set(72, 520);
    this.endText.anchor.set(0.5);
    this.endText.position.set(WORLD_WIDTH / 2, WORLD_HEIGHT * 0.52);
    this.drawBackground();
  }

  render(
    snapshot: PulseSnapshot,
    input: PulseInputViewState,
    options: { scale: number; offsetX: number; offsetY: number; debug: boolean }
  ): void {
    this.world.position.set(options.offsetX, options.offsetY);
  }

```

```

this.world.scale.set(options.scale);
this.renderLinks(snapshot);
this.renderBlackHole(snapshot);
this.renderLenses(snapshot);
this.renderPreview(snapshot, input);
this.renderNodes(snapshot, input);
this.renderPulses(snapshot);
this.renderHud(snapshot, input, options.debug);
}

destroy(): void {
  this.stage.removeChild(this.world);
  this.world.destroy({ children: true });
}

private drawBackground(): void {
  this.background.clear();
  this.background.rect(0, 0, WORLD_WIDTH, WORLD_HEIGHT).fill(0x03040a);
  for (let index = 0; index < 260; index += 1) {
    const x = (index * 197.63) % WORLD_WIDTH;
    const y = (index * 311.19) % WORLD_HEIGHT;
    const depth = (index % 13) / 12;
    this.background.circle(x, y, 1.1 + depth * 2.3).fill({ color: 0xc8efff, alpha: 0.17 + depth * 0.35 });
  }
  for (let index = 0; index < 170; index += 1) {
    const t = index / 169;
    const angle = t * Math.PI * 8.6;
    const radius = 80 + t * 690;
    const x = BLACK_HOLE_X + Math.cos(angle) * radius;
    const y = BLACK_HOLE_Y + Math.sin(angle) * radius * 0.56;
    this.background.circle(x, y, 3 + t * 6).fill({ color: index % 2 === 0 ? 0x264e9a : 0x743a96, alpha: 0.05 + (1 - t) * 0.08 });
  }
}

private renderLinks(snapshot: PulseSnapshot): void {
  this.linkLayer.clear();
  this.tempLinkLayer.clear();
  for (const link of snapshot.links) {
    const from = findNode(snapshot.nodes, link.fromId);
    const to = findNode(snapshot.nodes, link.toId);
    if (!from || !to) {
      continue;
    }
    const layer = link.temporary ? this.tempLinkLayer : this.linkLayer;
    const alpha = link.temporary ? clamp(1 - link.ageMs / link.expiresMs, 0, 1) : 1;
    const flash = clamp(link.flashMs / 520, 0, 1);
    this.drawCurve(layer, curvedLinkPath(from, to, link.temporary ? -0.16 : 0.12), {
      glowColor: link.temporary ? 0xd267ff : 0x4dccff,
      coreColor: link.temporary ? 0xffffffff : 0x9fe7ff,
      alpha,
      width: link.temporary ? 9 + flash * 5 : 6 + flash * 4
    });
    this.drawFlowDots(layer, from, to, snapshot.timeMs, link.temporary, alpha);
  }
}

private renderBlackHole(snapshot: PulseSnapshot): void {
  this.blackHole.clear();
  const pulse = Math.sin(snapshot.timeMs * 0.004) * 0.5 + 0.5;
  const collapse = 1 - snapshot.darkEnergy / MAX_ENERGY;
  const radius = 92 + collapse * 58 + (snapshot.phase === 'ended' && snapshot.collapsed ? 130 : 0);
  this.blackHole.circle(BLACK_HOLE_X, BLACK_HOLE_Y, radius * 2.2).fill({ color: 0x0b1426, alpha: 0.12 + collapse * 0.18 });
  this.blackHole.circle(BLACK_HOLE_X, BLACK_HOLE_Y, radius * 1.28).stroke({
    color: snapshot.phase === 'ended' && snapshot.stabilized ? 0x4dfbf : 0x6bcfff,
    alpha: 0.22 + collapse * 0.28,
    width: 7 + pulse * 6
  });
  this.blackHole.circle(BLACK_HOLE_X, BLACK_HOLE_Y, radius).fill({ color: 0x000000, alpha: 0.86 });
  this.blackHole.circle(BLACK_HOLE_X, BLACK_HOLE_Y, radius * 0.38).fill({ color: 0x03040a, alpha: 1 });
}

private renderLenses(snapshot: PulseSnapshot): void {
  this.lensLayer.clear();
  for (const lens of snapshot.lenses) {
    const alpha = clamp(1 - lens.ageMs / lens.durationMs, 0, 1);
    const smooth = resamplePath(lens.path, 44);
    this.drawCurve(this.lensLayer, smooth, {
      glowColor: lens.success ? 0xd267ff : 0xff6a83,
      coreColor: lens.success ? 0xffffffff : 0xfbf0c0,
      alpha,
      width: 8
    });
    const end = smooth[smooth.length - 1];
    this.lensLayer.circle(end.x, end.y, 15).stroke({ color: lens.success ? 0xffffffff : 0xff6a83, alpha, width: 4 });
  }
}

private renderPreview(snapshot: PulseSnapshot, input: PulseInputViewState): void {
  this.previewLayer.clear();
  if (snapshot.tutorialGhostPath.length > 1) {
    this.drawCurve(this.previewLayer, resamplePath(snapshot.tutorialGhostPath, 34), {
      glowColor: 0xd267ff,
      coreColor: 0xffffffff,
      alpha: 0.62 + Math.sin(snapshot.timeMs * 0.006) * 0.18,
      width: 6
    });
    const ghost = pointAlongPolyline(snapshot.tutorialGhostPath, (snapshot.timeMs * 0.00032) % 1);
    this.previewLayer.circle(ghost.x, ghost.y, 18).fill({ color: 0xffffffff, alpha: 0.74 });
    this.previewLayer.circle(ghost.x, ghost.y, 34).stroke({ color: 0xd267ff, alpha: 0.48, width: 5 });
  }
  if (snapshot.phase === 'build' && input.previewFromId && input.previewPoint) {
    const from = findNode(snapshot.nodes, input.previewFromId);
    if (from) {
      this.drawCurve(this.previewLayer, curvedLinkPath(from, input.previewPoint, 0.08), {
        glowColor: 0xd267ff,
        coreColor: 0xffffffff,
        alpha: 0.58,
        width: 5
      });
    }
  }
  if (snapshot.phase === 'build' && input.liveGesture.length > 1) {
    this.drawCurve(this.previewLayer, resamplePath(input.liveGesture, 36), {
      glowColor: 0xd267ff,
      coreColor: 0xffffffff,
      alpha: 0.82,
      width: 7
    });
  }
  if (snapshot.phase === 'pulse' && input.liveGesture.length > 1) {
    this.drawCurve(this.previewLayer, resamplePath(input.liveGesture, 36), {
      glowColor: 0xd267ff,
      coreColor: 0xffffffff,
      alpha: 0.85,
      width: 7
    });
  }
}

private renderNodes(snapshot: PulseSnapshot, input: PulseInputViewState): void {
  this.nodeLayer.clear();
  const visibleNodeIds = new Set<number>();
  for (const node of snapshot.nodes) {

```

```

const selected = input.selectedNodeId === node.id;
const nearest = input.nearestNodeId === node.id;
const active = node.activationMs > 0;
const highlighted =
  snapshot.tutorialHighlightNodeIds.includes(node.id) ||
  snapshot.deadEndNodeId === node.id ||
  snapshot.suggestedFixes.some((fix) => fix.fromId === node.id || fix.toId === node.id);
const color = NODE_COLORS[node.type];
const halo = selected || nearest || active || highlighted ? 0.68 : node.type === 'energy' || node.type === 'source' ? 0.34 : 0.22;
this.nodeLayer.circle(node.x, node.y, node.radius + 30 + (active || highlighted ? 20 : 0)).fill({ color, alpha: halo * 0.23 });
if (node.primed) {
  this.nodeLayer.circle(node.x, node.y, node.radius + 26).stroke({ color: 0xffffff, alpha: 0.84, width: 6 });
}
if (node.stabilizedMs > 0) {
  this.nodeLayer.circle(node.x, node.y, node.radius + 38).stroke({ color: 0x4dffbf, alpha: 0.76, width: 8 });
}
this.nodeLayer.circle(node.x, node.y, node.radius + 12).stroke({ color: highlighted ? 0xffffffff : color, alpha: halo, width: selected || highlighted ? 7 : 4 });
if (node.type === 'splitter') {
  this.nodeLayer.regularPoly(node.x, node.y, node.radius, 3, -Math.PI / 2 + node.splitterPriority * 0.7).fill({ color, alpha: 0.86 });
} else if (node.type === 'delay') {
  this.nodeLayer.roundRect(node.x - node.radius * 0.74, node.y - node.radius * 0.74, node.radius * 1.48, node.radius * 1.48, 12).fill({ color, alpha: 0.88 });
  for (let tick = 0; tick <= node.delayLevel; tick += 1) {
    this.nodeLayer.roundRect(node.x - 22 + tick * 22, node.y + node.radius + 16, 13, 7, 3).fill({ color: 0xffffffff, alpha: 0.78 });
  }
} else {
  this.nodeLayer.circle(node.x, node.y, node.radius).fill({ color, alpha: 0.88 });
}
this.nodeLayer.circle(node.x, node.y, node.radius * 0.45).fill({ color: 0xffffffff, alpha: active ? 0.82 : 0.36 });
const text = this.nodeText(node.id);
text.text = iconForNode(node);
text.position.set(node.x, node.y - 2);
text.visible = true;
visibleNodeIds.add(node.id);
if (snapshot.tutorialActive && highlighted) {
  const label = this.nodeText(node.id + 1000);
  label.text = node.type === 'energy' ? 'ENERGY' : node.label;
  label.position.set(node.x, node.y + node.radius + 46);
  label.visible = true;
  visibleNodeIds.add(node.id + 1000);
}
for (const [id, text] of this.nodeTexts) {
  if (!visibleNodeIds.has(id)) {
    text.visible = false;
  }
}
}

private renderPulses(snapshot: PulseSnapshot): void {
  this.pulseLayer.clear();
  for (const pulse of snapshot.pulses) {
    const point = pulsePoint(snapshot, pulse.currentNodeId, pulse.nextNodeId, pulse.progress);
    if (!point) {
      continue;
    }
    this.pulseLayer.circle(point.x, point.y, 30).fill({ color: 0x67f4ff, alpha: 0.18 });
    this.pulseLayer.circle(point.x, point.y, 15).fill({ color: 0xd267ff, alpha: 0.72 });
    this.pulseLayer.circle(point.x, point.y, 7).fill({ color: 0xffffffff, alpha: 0.95 });
  }
}

private renderHud(snapshot: PulseSnapshot, input: PulseInputViewState, debug: boolean): void {
  this.scoreText.text = String(snapshot.score);
  this.metaText.text = `x${snapshot.multiplier} LINKS ${snapshot.linksUsed}/${snapshot.linkBudget} LENS ${snapshot.lensCharges}/2 BEST ${Math.max(snapshot.score, Number(localStorage.getItem('eventHorizon.bestScore') ?? 0))}`;
  this.strategyText.visible = snapshot.phase === 'build';
  this.strategyText.text = [
    'BUILD A CHAIN',
    'Hit Energy nodes.',
    'Use Delay nodes.',
    'Avoid dead ends.',
    `Energy ${snapshot.chainAnalysis.reachableEnergyNodes}/${snapshot.chainAnalysis.totalEnergyNodes}`,
    `Dead ends ${snapshot.chainAnalysis.deadEndNodeIds.length}`,
    `Loop ${snapshot.chainAnalysis.hasLoop ? 'Yes' : 'No'}`,
    snapshot.chainAnalysis.quality
  ].join('\n');
  this.hintText.text = snapshot.tutorialHint;
  this.hintText.visible = snapshot.phase !== 'ended';
  this.messageText.text = snapshot.lastInputResult.message;
  this.messageText.visible = snapshot.phase !== 'ended' && snapshot.lastInputResult.message !== snapshot.tutorialHint;
  this.meter.clear();
  const meterWidth = WORLD_WIDTH - 156;
  const fill = meterWidth * clamp(snapshot.darkEnergy / MAX_ENERGY, 0, 1);
  this.meter.roundRect(78, WORLD_HEIGHT - 136, meterWidth, 40, 8).fill({ color: 0x061120, alpha: 0.9 });
  this.meter.roundRect(78, WORLD_HEIGHT - 136, meterWidth, 40, 8).stroke({ color: 0x78f2ff, alpha: 0.35, width: 2 });
  this.meter.roundRect(86, WORLD_HEIGHT - 126, fill, 20, 6).fill({ color: snapshot.darkEnergy < 25 ? 0xff5d73 : 0x67f4ff, alpha: 0.96 });
  this.meter.roundRect(86, WORLD_HEIGHT - 172, 220, 28, 4).fill({ color: 0x03040a, alpha: 0.46 });
  this.meter.roundRect(0, 0, 0, 0);
  this.endText.visible = snapshot.phase === 'ended';
  if (snapshot.phase === 'ended') {
    const fix = snapshot.suggestedFixes[0];
    this.endText.text = snapshot.endReason === 'pulse-died'
      ? `PULSE LOST\nDead end at: ${nodeLabel(snapshot, snapshot.deadEndNodeId)}\n${fix ? `Try Linking this node\n`to ${nodeLabel(snapshot, fix.toId)}.` : 'Try adding one more outgoing link.'}`
      : `${snapshot.stabilized ? 'SECTOR STABILIZED' : 'GALAXY COLLAPSED'}\n${snapshot.score} • ${formatTime(snapshot.timeMs)}\nEnergy nodes hit ${snapshot.chainAnalysis.reachableEnergyNodes}/${snapshot.chainAnalysis.totalEnergyNodes}\nSEED ${snapshot.seed}`;
  } else {
    this.endText.text = '';
  }
  this.debugText.visible = debug;
  if (debug) {
    this.debugText.text = [
      `phase: ${snapshot.phase}`,
      `tutorial: ${snapshot.tutorialStep}`,
      `selected: ${input.selectedNodeId ?? '—'} nearest: ${input.nearestNodeId ?? '—'}`,
      `links: ${snapshot.linksUsed}/${snapshot.linkBudget} pulses: ${snapshot.pulses.length}`,
      `chain: ${snapshot.lastChainNodeIds.join('>')} || '—'`,
      `analysis: ${snapshot.chainAnalysis.quality}`,
      `last: ${snapshot.lastInputResult.message}`,
      `hash: ${snapshot.stepHash}`
    ].join('\n');
  }
}

private drawCurve(
  graphics: Graphics,
  points: readonly WorldPoint[],
  options: { glowColor: number; coreColor: number; alpha: number; width: number }
): void {
  if (points.length < 2) {
    return;
  }
  drawSmooth(graphics, points);
  graphics.stroke({ color: options.glowColor, alpha: options.alpha * 0.28, width: options.width * 3.4 });
  drawSmooth(graphics, points);
  graphics.stroke({ color: options.coreColor, alpha: options.alpha, width: options.width });
}

private nodeText(id: number): Text {
  let text = this.nodeTexts.get(id);
  if (!text) {
    text = new Text({
      text: '',
      style: new TextStyle({

```

```

        align: 'center',
        fill: '#061120',
        fontFamily: 'Inter, system-ui, sans-serif',
        fontSize: id >= 1000 ? 18 : 26,
        fontWeight: '900',
        stroke: { color: 'ffffff', width: id >= 1000 ? 2 : 3 }
    });
    text.anchor.set(0.5);
    this.nodeTextLayer.addChild(text);
    this.nodeTexts.set(id, text);
}
return text;
}

private drawFlowDots(
    graphics: Graphics,
    from: PulseNode,
    to: PulseNode,
    timeMs: number,
    temporary: boolean,
    alpha: number
): void {
    const path = curvedLinkPath(from, to, temporary ? -0.16 : 0.12);
    for (let index = 0; index < 3; index += 1) {
        const t = ((timeMs * 0.00018 + index / 3) % 1 + 1) % 1;
        const point = pointOnQuadratic(path[0], path[1], path[2], t);
        graphics.circle(point.x, point.y, temporary ? 6 : 4).fill({ color: temporary ? 0xffffff : 0x9fe7ff, alpha: alpha * 0.66 });
    }
}

function drawSmooth(graphics: Graphics, points: readonly WorldPoint[]): void {
    graphics.moveTo(points[0].x, points[0].y);
    for (let index = 1; index < points.length - 1; index += 1) {
        const point = points[index];
        const next = points[index + 1];
        graphics.quadraticCurveTo(point.x, point.y, (point.x + next.x) / 2, (point.y + next.y) / 2);
    }
    const last = points[points.length - 1];
    graphics.lineTo(last.x, last.y);
}

function findNode(nodes: readonly PulseNode[], id: number): PulseNode | undefined {
    return nodes.find((node) => node.id === id);
}

function pulsePoint(snapshot: PulseSnapshot, fromId: number, toId: number | undefined, progress: number): WorldPoint | undefined {
    const from = findNode(snapshot.nodes, fromId);
    if (!from) {
        return undefined;
    }
    if (toId === undefined) {
        return from;
    }
    const to = findNode(snapshot.nodes, toId);
    if (!to) {
        return from;
    }
    const path = curvedLinkPath(from, to, 0.12);
    return pointOnQuadratic(path[0], path[1], path[2], progress);
}

function iconForNode(node: PulseNode): string {
    if (node.type === 'source') {
        return 'S';
    }
    if (node.type === 'energy') {
        return '+';
    }
    if (node.type === 'delay') {
        return 'II';
    }
    if (node.type === 'splitter') {
        return 'Y';
    }
    return 'o';
}

function pointAlongPolyline(points: readonly WorldPoint[], t: number): WorldPoint {
    if (points.length === 0) {
        return { x: 0, y: 0 };
    }
    if (points.length === 1) {
        return points[0];
    }
    const lengths: number[] = [];
    let total = 0;
    for (let index = 1; index < points.length; index += 1) {
        const length = Math.hypot(points[index].x - points[index - 1].x, points[index].y - points[index - 1].y);
        lengths.push(length);
        total += length;
    }
    let target = clamp(t, 0, 1) * total;
    for (let index = 1; index < points.length; index += 1) {
        const length = lengths[index - 1];
        if (target <= length) {
            const local = length <= 0 ? 0 : target / length;
            return {
                x: points[index - 1].x + (points[index].x - points[index - 1].x) * local,
                y: points[index - 1].y + (points[index].y - points[index - 1].y) * local
            };
        }
        target -= length;
    }
    return points[points.length - 1];
}

function nodeLabel(snapshot: PulseSnapshot, nodeId: number | undefined): string {
    const node = nodeId === undefined ? undefined : findNode(snapshot.nodes, nodeId);
    if (!node) {
        return 'UNKNOWN NODE';
    }
    const label = node.type === 'energy' ? 'ENERGY' : node.label;
    return `${label} ${node.id}`;
}

```

src/game/pulse/PulseSimulation.ts

```

import { INITIAL_ENERGY, MAX_ENERGY } from '../constants';
import { clamp } from '../math';
import { quantizeGesturePath, type GesturePathPoint, type WorldPoint } from '../gestures';
import { distancePointToSegment, nearestTwoNodesToPath, nodesCrossedByPath } from './PulseGeometry';
import { generatePulseLevel } from './PulseLevelGenerator';
import type {
    BuildInput,
    ChainAnalysis,
    HorizonLens,
    LiveInput,
    NodeTapAction,

```

```

PulseGamePhase,
PulseInputResult,
PulseLevel,
PulseLink,
PulseNode,
PulseReplayPayload,
PulseResult,
PulseSnapshot,
PulseState,
SuggestedFix,
TutorialStep
} from './PulseTypes';

const NORMAL_PULSE_SPEED = 310;
const TUTORIAL_PULSE_SPEED = 220;
const LINK_TRAVERSAL_SCORE = 10;
const ENERGY_NODE_SCORE = 100;
const DELAY_NODE_SCORE = 25;
const SPLITTER_NODE_SCORE = 75;
const LENS_DURATION_MS = 1500;
const TUTORIAL_LENS_DURATION_MS = 2100;
const LENS_MAX_CHARGES = 2;
const ENERGY_DRAIN_PER_SECOND = 0.38;
const STABILIZE_SCORE = 50;
const DELAY_MS = [360, 700, 1080] as const;

export interface PulseSimulationOptions {
  seed: string;
  startedAt: number;
}

export class PulseSimulation {
  readonly startedAt: number;
  private seedValue: string;
  private level: PulseLevel;
  private phase: PulseGamePhase = 'build';
  private timeMs = 0;
  private score = 0;
  private darkEnergy = INITIAL_ENERGY;
  private multiplier = 1;
  private maxMultiplier = 1;
  private chainLength = 0;
  private loopsCompleted = 0;
  private nextLinkId = 1;
  private nextPulseId = 1;
  private nextLensId = 1;
  private endReason: PulseSnapshot['endReason'];
  private deadEndNodeId: number | undefined;
  private collapsed = false;
  private stabilized = false;
  private buildInputs: BuildInput[] = [];
  private liveInputs: LiveInput[] = [];
  private links: PulseLink[] = [];
  private pulses: PulseState[] = [];
  private lenses: HorizonLens[] = [];
  private lastInputResult: PulseInputResult = { ok: true, kind: 'none', message: 'CONNECT TWO NODES' };
  private selectedNodeId: number | undefined;
  private stepHash = '00000000';
  private lensCharges = LENS_MAX_CHARGES;
  private tutorialActive = false;
  private tutorialStep: TutorialStep = 'skipped';
  private lastChainNodeIds: number[] = [];
  private lastTapAction: NodeTapAction = 'none';
  private energyNodesHit = new Set<number>();
  private slowDrainMs = 0;

  constructor(options: PulseSimulationOptions) {
    this.seedValue = options.seed;
    this.startedAt = options.startedAt;
    this.level = generatePulseLevel(options.seed);
    if (options.seed === 'tutorial-001') {
      this.startTutorial();
    }
    this.updateHash();
  }

  get seed(): string {
    return this.seedValue;
  }

  reset(seed = this.seed): void {
    this.seedValue = seed;
    this.level = generatePulseLevel(seed);
    this.phase = 'build';
    this.timeMs = 0;
    this.score = 0;
    this.darkEnergy = INITIAL_ENERGY;
    this.multiplier = 1;
    this.maxMultiplier = 1;
    this.chainLength = 0;
    this.loopsCompleted = 0;
    this.nextLinkId = 1;
    this.nextPulseId = 1;
    this.nextLensId = 1;
    this.endReason = undefined;
    this.deadEndNodeId = undefined;
    this.collapsed = false;
    this.stabilized = false;
    this.buildInputs = [];
    this.liveInputs = [];
    this.links = [];
    this.pulses = [];
    this.lenses = [];
    this.selectedNodeId = undefined;
    this.lensCharges = LENS_MAX_CHARGES;
    this.lastChainNodeIds = [];
    this.lastTapAction = 'none';
    this.energyNodesHit = new Set();
    this.slowDrainMs = 0;
    this.tutorialActive = seed === 'tutorial-001';
    this.tutorialStep = this.tutorialActive ? 'swipe-chain' : 'skipped';
    this.lastInputResult = { ok: true, kind: 'none', message: this.tutorialActive ? 'SWIPE THROUGH THESE NODES' : 'DRAW A CHAIN' };
    this.updateHash();
  }

  step(dtMs: number): void {
    if (this.phase === 'ended') {
      this.timeMs += dtMs;
      this.updateTimedVisuals(dtMs);
      this.updateHash();
      return;
    }

    this.timeMs += dtMs;
    this.updateTimedVisuals(dtMs);

    if (this.phase === 'pulse') {
      const tutorialGrace = this.tutorialActive && this.timeMs < 10000;
      const drainScale = this.slowDrainMs > 0 || tutorialGrace ? 0.35 : 1;
      this.darkEnergy = clamp(this.darkEnergy - (dtMs / 1000) * ENERGY_DRAIN_PER_SECOND * drainScale, 0, MAX_ENERGY);
      this.slowDrainMs = Math.max(0, this.slowDrainMs - dtMs);
      this.updatePulses(dtMs);
      this.expireTemporaryLinks(dtMs);
    }
  }

```

```

    if (this.darkEnergy <= 0) {
      this.endRun('collapsed');
    } else if (this.score >= this.level.targetScore || this.timeMs >= this.level.targetSurvivalMs) {
      this.endRun('stabilized');
    } else if (this.pulses.every((pulse) => !pulse.alive) && this.timeMs > 500) {
      this.darkEnergy = clamp(this.darkEnergy - 0.9, 0, MAX_ENERGY);
      if (this.darkEnergy <= 0) {
        this.endRun('collapsed');
      } else {
        this.endRun('pulse-died');
      }
    }
  }
}
this.updateHash();
}

selectNode(nodeId: number | undefined): PulseInputResult {
  this.selectedNodeId = nodeId;
  this.lastInputResult = nodeId
  ? { ok: true, kind: 'select', message: 'NODE SELECTED', fromId: nodeId }
  : { ok: true, kind: 'none', message: 'SELECT A NODE' };
  return this.lastInputResult;
}

addLink(fromId: number, toId: number, record = true): PulseInputResult {
  const validation = this.validateLink(fromId, toId, false);
  if (!validation.ok) {
    this.lastInputResult = validation;
    return validation;
  }
  const link: PulseLink = {
    id: this.nextLinkId,
    fromId,
    toId,
    temporary: false,
    ageMs: 0,
    expiresMs: 0,
    flashMs: 420
  };
  this.nextLinkId += 1;
  this.links.push(link);
  if (record) {
    this.buildInputs.push({ t: Math.round(this.timeMs), kind: 'link', fromId, toId });
  }
  this.selectedNodeId = undefined;
  this.lastInputResult = { ok: true, kind: 'link', message: 'GRAVITATIONAL LINK', fromId, toId };
  this.updateHash();
  return this.lastInputResult;
}

applyChainSwipe(points: readonly GesturePathPoint[], record = true): PulseInputResult {
  if (this.phase !== 'build') {
    this.lastInputResult = { ok: false, kind: 'invalid', message: 'CHAIN ONLY DURING BUILD' };
    return this.lastInputResult;
  }
  const path = quantizeGesturePath(points, 24);
  const crossed = nodesCrossedByPath(this.level.nodes, path, 78);
  const nodeIds = crossed.map((node) => node.id);
  this.lastChainNodeIds = nodeIds;
  if (nodeIds.length < 2) {
    this.lastInputResult = { ok: false, kind: 'chainSwipe', message: 'NO NODES CROSSED', nodeIds };
    this.advanceTutorial('chain-miss');
    return this.lastInputResult;
  }

  let created = 0;
  const createdNodes: number[] = [nodeIds[0]];
  for (let index = 1; index < nodeIds.length; index += 1) {
    const fromId = nodeIds[index - 1];
    const toId = nodeIds[index];
    if (fromId === toId) {
      continue;
    }
    const result = this.addLink(fromId, toId, false);
    if (!result.ok) {
      if (result.message === 'LINK EXISTS') {
        createdNodes.push(toId);
        continue;
      }
      if (result.message === 'LINK LIMIT') {
        break;
      }
    }
    continue;
  }
  created += 1;
  createdNodes.push(toId);
}

if (created > 0 && record) {
  this.buildInputs.push({ t: Math.round(this.timeMs), kind: 'chainSwipe', nodeIds: createdNodes, path });
}
this.selectedNodeId = undefined;
this.lastInputResult = created > 0
  ? { ok: true, kind: 'chainSwipe', message: `CHAIN CREATED ${created} LINKS`, nodeIds: createdNodes }
  : { ok: false, kind: 'chainSwipe', message: 'CHAIN BLOCKED', nodeIds };
this.advanceTutorial('chain');
this.updateHash();
return this.lastInputResult;
}

tapNode(nodeId: number, record = true): PulseInputResult {
  const node = this.nodeById(nodeId);
  if (!node) {
    this.lastInputResult = { ok: false, kind: 'invalid', message: 'NO NODE', nodeId };
    return this.lastInputResult;
  }
  if (this.phase === 'pulse') {
    return this.stabilizeNode(nodeId, record);
  }

  let action: NodeTapAction = 'select';
  if (node.type === 'energy') {
    node.primed = !node.primed;
    node.activationMs = 520;
    action = 'prime';
    this.lastInputResult = { ok: true, kind: 'nodeTap', message: node.primed ? 'ENERGY PRIMED' : 'ENERGY UNPRIMED', nodeId };
  } else if (node.type === 'delay') {
    node.delayLevel = ((node.delayLevel + 1) % 3) as 0 | 1 | 2;
    node.activationMs = 520;
    action = 'delay';
    this.lastInputResult = { ok: true, kind: 'nodeTap', message: `DELAY ${node.delayLevel + 1}`, nodeId };
  } else if (node.type === 'splitter') {
    node.splitterPriority = (node.splitterPriority + 1) % 3;
    node.activationMs = 520;
    action = 'splitter';
    this.lastInputResult = { ok: true, kind: 'nodeTap', message: 'SPLITTER AIMED', nodeId };
    this.advanceTutorial('splitter');
  } else {
    this.selectedNodeId = nodeId;
    this.lastInputResult = { ok: true, kind: 'select', message: `SELECTED ${node.label}`, nodeId };
  }
  this.lastTapAction = action;
  if (record) {

```

```

    this.buildInputs.push({ t: Math.round(this.timeMs), kind: 'nodeTap', nodeId, action });
  }
  this.updateHash();
  return this.lastInputResult;
}

stabilizeNode(nodeId: number, record = true): PulseInputResult {
  const node = this.nodeById(nodeId);
  if (!node) {
    this.lastInputResult = { ok: false, kind: 'invalid', message: 'NO NODE', nodeId };
    return this.lastInputResult;
  }
  const arrival = this.arrivalReadiness(nodeId);
  let rating: 'perfect' | 'stabilized' | 'early' | 'late' = 'early';
  let success = false;
  if (arrival === 'late') {
    rating = 'late';
  } else if (arrival === 'perfect') {
    rating = 'perfect';
    success = true;
  } else if (arrival === 'soon') {
    rating = 'stabilized';
    success = true;
  }

  if (success) {
    node.stabilizedMs = 760;
    node.activationMs = 620;
    this.addScore(rating === 'perfect' ? STABILIZE_SCORE + 25 : STABILIZE_SCORE, 2.2);
    this.slowDrainMs = 1500;
    this.lastInputResult = {
      ok: true,
      kind: 'stabilize',
      message: rating === 'perfect' ? 'PERFECT TAP +75' : 'STABILIZED +50',
      nodeId,
      scoreDelta: rating === 'perfect' ? 75 : 50,
      energyDelta: 2
    };
  } else {
    this.lastInputResult = { ok: false, kind: 'stabilize', message: rating === 'late' ? 'LATE' : 'EARLY', nodeId };
  }
  this.lastTapAction = 'stabilize';
  if (record) {
    this.liveInputs.push({ t: Math.round(this.timeMs), kind: 'stabilize', nodeId, rating, success });
  }
  this.advanceTutorial('stabilize');
  this.updateHash();
  return this.lastInputResult;
}

removeLinkNear(point: WorldPoint): PulseInputResult {
  if (this.phase !== 'build') {
    this.lastInputResult = { ok: false, kind: 'invalid', message: 'LINKS LOCKED' };
    return this.lastInputResult;
  }
  let bestIndex = -1;
  let bestDistance = 54;
  for (let index = 0; index < this.links.length; index += 1) {
    const link = this.links[index];
    if (link.temporary) {
      continue;
    }
    const from = this.nodeById(link.fromId);
    const to = this.nodeById(link.toId);
    if (!from || !to) {
      continue;
    }
    const distance = distancePointToSegment(point, from, to);
    if (distance < bestDistance) {
      bestDistance = distance;
      bestIndex = index;
    }
  }
  if (bestIndex === -1) {
    this.lastInputResult = { ok: false, kind: 'invalid', message: 'NO LINK' };
    return this.lastInputResult;
  }
  this.links.splice(bestIndex, 1);
  this.lastInputResult = { ok: true, kind: 'clear', message: 'LINK REMOVED' };
  this.updateHash();
  return this.lastInputResult;
}

undo(): PulseInputResult {
  const index = [...this.links].map((link, linkIndex) => ({ link, linkIndex })).reverse().find((entry) => !entry.link.temporary);
  if (!index) {
    this.lastInputResult = { ok: false, kind: 'invalid', message: 'NO LINK TO UNDO' };
    return this.lastInputResult;
  }
  this.links.splice(index.linkIndex, 1);
  this.buildInputs.push({ t: Math.round(this.timeMs), kind: 'undo' });
  this.lastInputResult = { ok: true, kind: 'undo', message: 'LINK UNDONE' };
  this.updateHash();
  return this.lastInputResult;
}

clearLinks(): PulseInputResult {
  this.links = this.links.filter((link) => !link.temporary);
  this.buildInputs.push({ t: Math.round(this.timeMs), kind: 'clear' });
  this.selectedNodeId = undefined;
  this.lastInputResult = { ok: true, kind: 'clear', message: 'LINKS CLEARED' };
  this.updateHash();
  return this.lastInputResult;
}

playPulse(record = true): PulseInputResult {
  if (this.phase !== 'build') {
    this.lastInputResult = { ok: false, kind: 'invalid', message: 'PULSE ALREADY RUNNING' };
    return this.lastInputResult;
  }
  const outgoing = this.outgoingLinks(this.level.sourceId);
  if (outgoing.length === 0) {
    this.lastInputResult = { ok: false, kind: 'invalid', message: 'CONNECT SOURCE FIRST' };
    return this.lastInputResult;
  }
  this.phase = 'pulse';
  this.timeMs = 0;
  this.emitFromNode(this.level.sourceId, undefined, 0, []);
  if (record) {
    this.buildInputs.push({ t: 0, kind: 'play' });
  }
  this.lastInputResult = { ok: true, kind: 'play', message: 'STABILIZING PULSE' };
  this.advanceTutorial('play');
  this.updateHash();
  return this.lastInputResult;
}

applyLens(points: readonly GesturePathPoint[]): PulseInputResult {
  const path = quantizeGesturePath(points, 24);
  const lensPath = path.map((point) => ({ x: point.x, y: point.y }));
  if (this.phase !== 'pulse') {
    this.lastInputResult = { ok: false, kind: 'invalid', message: 'LENS ONLY DURING PLAYBACK' };
    return this.lastInputResult;
  }

```



```

}
if (this.lensCharges <= 0) {
  this.lastInputResult = { ok: false, kind: 'lens', message: 'LENS RECHARGING' };
  return this.lastInputResult;
}

const anchors = nearestTwoNodesToPath(this.level.nodes, lensPath, 132);
if (!anchors) {
  this.createLens(lensPath, false);
  this.liveInputs.push({ t: Math.round(this.timeMs), kind: 'lens', path, success: false });
  this.lastInputResult = { ok: false, kind: 'lens', message: 'NO ANCHOR - SWIPE BETWEEN TWO NODES' };
  return this.lastInputResult;
}

const [from, to] = anchors;
const validation = this.validateLink(from.id, to.id, true);
if (!validation.ok) {
  this.createLens(lensPath, false);
  this.liveInputs.push({ t: Math.round(this.timeMs), kind: 'lens', path, fromId: from.id, toId: to.id, success: false });
  this.lastInputResult = { ok: false, kind: 'lens', message: 'NO ANCHOR - SWIPE BETWEEN TWO NODES', fromId: from.id, toId: to.id };
  return this.lastInputResult;
}

this.links.push({
  id: this.nextLinkId,
  fromId: from.id,
  toId: to.id,
  temporary: true,
  ageMs: 0,
  expiresMs: this.tutorialActive ? TUTORIAL_LENS_DURATION_MS : LENS_DURATION_MS,
  flashMs: 520
});
this.nextLinkId += 1;
this.lensCharges = Math.max(0, this.lensCharges - 1);
this.createLens(lensPath, true, from.id, to.id);
this.liveInputs.push({ t: Math.round(this.timeMs), kind: 'lens', path, fromId: from.id, toId: to.id, success: true });
this.lastInputResult = { ok: true, kind: 'lens', message: 'HORIZON LENS - BRIDGE CREATED', fromId: from.id, toId: to.id };
this.advanceTutorial('lens');
this.updateHash();
return this.lastInputResult;
}

forceBuildPhase(): void {
  this.phase = 'build';
  this.pulses = [];
  this.endReason = undefined;
  this.collapsed = false;
  this.stabilized = false;
  this.darkEnergy = Math.max(this.darkEnergy, 45);
  this.updateHash();
}

forcePulsePhase(): void {
  if (this.phase === 'build') {
    void this.playPulse(false);
  }
}

forceCollapse(): void {
  this.endRun('collapsed');
}

startTutorial(): void {
  this.seedValue = 'tutorial-001';
  this.level = generatePulseLevel(this.seedValue);
  this.tutorialActive = true;
  this.tutorialStep = 'swipe-chain';
  this.phase = 'build';
  this.timeMs = 0;
  this.score = 0;
  this.multiplier = 1;
  this.maxMultiplier = 1;
  this.chainLength = 0;
  this.loopsCompleted = 0;
  this.nextLinkId = 1;
  this.nextPulseId = 1;
  this.nextLensId = 1;
  this.buildInputs = [];
  this.liveInputs = [];
  this.links = [];
  this.pulses = [];
  this.lenses = [];
  this.endReason = undefined;
  this.deadEndNodeId = undefined;
  this.collapsed = false;
  this.stabilized = false;
  this.selectedNodeId = undefined;
  this.darkEnergy = INITIAL_ENERGY;
  this.lensCharges = LENS_MAX_CHARGES;
  this.lastChainNodeIds = [];
  this.lastTapAction = 'none';
  this.energyNodesHit = new Set();
  this.slowDrainMs = 0;
  this.lastInputResult = { ok: true, kind: 'none', message: 'SWIPE THROUGH THESE NODES', nodeIds: [1, 2, 3] };
  this.updateHash();
}

skipTutorial(): void {
  this.tutorialActive = false;
  this.tutorialStep = 'skipped';
  this.lastInputResult = { ok: true, kind: 'none', message: 'DRAW A CHAIN' };
  this.updateHash();
}

getTutorialStep(): TutorialStep {
  return this.tutorialStep;
}

analyzeChain(): ChainAnalysis {
  return this.computeChainAnalysis();
}

 getSuggestedFixes(): readonly SuggestedFix[] {
  return this.computeSuggestedFixes();
}

primeNode(id: number): PulseInputResult {
  const node = this.nodeById(id);
  if (!node || node.type !== 'energy') {
    this.lastInputResult = { ok: false, kind: 'invalid', message: 'NOT ENERGY', nodeId: id };
    return this.lastInputResult;
  }
  node.prime = true;
  node.activationMs = 520;
  this.lastTapAction = 'prime';
  this.lastInputResult = { ok: true, kind: 'nodeTap', message: 'ENERGY PRIMED', nodeId: id };
  this.updateHash();
  return this.lastInputResult;
}

cycleNode(id: number): PulseInputResult {
  return this.tapNode(id);
}

```

```

fixChain(): PulseInputResult {
  this.forceBuildPhase();
  this.lastInputResult = { ok: true, kind: 'fix', message: 'FIX THE DEAD END', nodeId: this.deadEndNodeId };
  return this.lastInputResult;
}

getNodes(): readonly PulseNode[] {
  return this.level.nodes;
}

getLinks(): readonly PulseLink[] {
  return this.links;
}

getPulses(): readonly PulseState[] {
  return this.pulses;
}

getLastInputResult(): PulseInputResult {
  return { ...this.lastInputResult };
}

getReplayPayload(): PulseReplayPayload {
  return {
    version: 1,
    mode: 'pulse-chain',
    seed: this.seed,
    startedAt: this.startedAt,
    buildInputs: this.buildInputs.map(input =>
      input.kind === 'chainSwipe'
        ? { ...input, nodeIds: [...input.nodeIds], path: input.path.map(point => ({ ...point })) }
        : { ...input }
    ),
    liveInputs: this.liveInputs.map(input =>
      input.kind === 'lens'
        ? { ...input, path: input.path.map(point => ({ ...point })) }
        : { ...input }
    ),
    result: this.getResult(),
    stepHash: this.stepHash
  };
}

getSnapshot(): PulseSnapshot {
  return {
    mode: 'pulse-chain',
    seed: this.seed,
    phase: this.phase,
    timeMs: Math.round(this.timeMs),
    score: this.score,
    darkEnergy: this.darkEnergy,
    collapseMeter: this.darkEnergy,
    multiplier: this.multiplier,
    maxMultiplier: this.maxMultiplier,
    chainLength: this.chainLength,
    loopsCompleted: this.loopsCompleted,
    linkBudget: this.level.linkBudget,
    linksUsed: this.links.filter(link => !link.temporary).length,
    lensCharges: this.lensCharges,
    targetScore: this.level.targetScore,
    targetSurvivalMs: this.level.targetSurvivalMs,
    ended: this.phase === 'ended',
    endReason: this.endReason,
    collapsed: this.collapsed,
    stabilized: this.stabilized,
    nodes: this.level.nodes,
    links: this.links,
    pulses: this.pulses,
    lenses: this.lenses,
    selectedNodeId: this.selectedNodeId,
    tutorialHint: this.tutorialHint(),
    tutorialActive: this.tutorialActive,
    tutorialStep: this.tutorialStep,
    tutorialHighlightNodeIds: this.tutorialHighlightNodeIds(),
    tutorialGhostPath: this.tutorialGhostPath(),
    chainAnalysis: this.computeChainAnalysis(),
    suggestedFixes: this.computeSuggestedFixes(),
    lastChainNodeIds: this.lastChainNodeIds,
    lastTapAction: this.lastTapAction,
    deadEndNodeId: this.deadEndNodeId,
    lastInputResult: this.lastInputResult,
    stepHash: this.stepHash
  };
}

private validateLink(fromId: number, toId: number, temporary: boolean): PulseInputResult {
  if (this.phase !== 'build' && !temporary) {
    return { ok: false, kind: 'invalid', message: 'LINKS LOCKED' };
  }
  const from = this.nodeById(fromId);
  const to = this.nodeById(toId);
  if (!from || !to) {
    return { ok: false, kind: 'invalid', message: 'NO ANCHOR', fromId, toId };
  }
  if (fromId === toId) {
    return { ok: false, kind: 'invalid', message: 'NO SELF LINK', fromId, toId };
  }
  if (this.links.some(link => link.fromId === fromId && link.toId === toId)) {
    return { ok: false, kind: 'invalid', message: 'LINK EXISTS', fromId, toId };
  }
  if (!temporary && this.links.filter(link => !link.temporary).length >= this.level.linkBudget) {
    return { ok: false, kind: 'invalid', message: 'LINK LIMIT', fromId, toId };
  }
  const maxOutgoing = from.type === 'splitter' ? 3 : from.type === 'source' ? 2 : 1;
  if (this.outgoingLinks(fromId).filter(link => !link.temporary || temporary).length >= maxOutgoing) {
    return { ok: false, kind: 'invalid', message: 'NODE OUTPUT FULL', fromId, toId };
  }
  return { ok: true, kind: 'link', message: 'VALID', fromId, toId };
}

private updatePulses(dtMs: number): void {
  for (const pulse of this.pulses) {
    if (!pulse.alive) {
      continue;
    }
    pulse.ageMs += dtMs;
    if (pulse.delayMs > 0) {
      pulse.delayMs = Math.max(0, pulse.delayMs - dtMs);
      continue;
    }
    if (pulse.nextNodeId === undefined) {
      this.continuePulse(pulse);
      continue;
    }
    const from = this.nodeById(pulse.currentNodeId);
    const to = this.nodeById(pulse.nextNodeId);
    if (!from || !to) {
      this.killPulse(pulse);
      continue;
    }
    const length = Math.max(1, Math.hypot(to.x - from.x, to.y - from.y));
    pulse.progress += (pulse.speed * (dtMs / 1000)) / length;
  }
}

```

```

    if (pulse.progress >= 1) {
      this.arriveAtNode(pulse, to.id);
    }
  }
  this.pulses = this.pulses.filter((pulse) => pulse.alive);
}

private arriveAtNode(pulse: PulseState, nodeId: number): void {
  const node = this.nodeById(nodeId);
  if (!node) {
    this.killPulse(pulse);
    return;
  }
  pulse.previousNodeId = pulse.currentNodeId;
  pulse.currentNodeId = nodeId;
  pulse.nextNodeId = undefined;
  pulse.progress = 0;
  pulse.comboChainLength += 1;
  pulse.visitedNodeIds.push(nodeId);
  this.chainLength = Math.max(this.chainLength, pulse.comboChainLength);
  this.multiplier = multiplierForChain(pulse.comboChainLength);
  this.maxMultiplier = Math.max(this.maxMultiplier, this.multiplier);
  node.activationMs = 520;
  this.addScore(LINK_TRAVERSAL_SCORE, 0.18);

  const firstRepeatedIndex = pulse.visitedNodeIds.indexOf(nodeId);
  if (firstRepeatedIndex >= 0 && firstRepeatedIndex < pulse.visitedNodeIds.length - 1) {
    const loopLength = pulse.visitedNodeIds.length - 1 - firstRepeatedIndex;
    if (loopLength >= 4) {
      this.loopsCompleted += 1;
      this.addScore(140 * this.loopsCompleted, 1.2);
      this.multiplier = Math.max(this.multiplier, Math.min(12, this.multiplier + this.loopsCompleted));
      this.maxMultiplier = Math.max(this.maxMultiplier, this.multiplier);
      this.lensCharges = Math.min(LENS_MAX_CHARGES, this.lensCharges + 1);
    }
  }

  if (node.type === 'energy') {
    const fresh = node.scoreCooldownMs <= 0;
    const primedBonus = node.primed ? 80 : 0;
    this.addScore((fresh ? ENERGY_NODE_SCORE : 25) + primedBonus, fresh ? 6.2 + primedBonus / 40 : 1.4);
    node.primed = false;
    node.scoreCooldownMs = fresh ? 2600 : node.scoreCooldownMs;
    this.energyNodesHit.add(node.id);
    this.lensCharges = Math.min(LENS_MAX_CHARGES, this.lensCharges + 1);
  } else if (node.type === 'delay') {
    this.addScore(DELAY_NODE_SCORE, 1.1);
    pulse.delayMs = DELAY_MS[node.delayLevel];
    return;
  } else if (node.type === 'splitter') {
    this.addScore(SPLITTER_NODE_SCORE, 1.8);
  }

  this.continuePulse(pulse);
}

private continuePulse(pulse: PulseState): void {
  const outgoing = this.outgoingLinks(pulse.currentNodeId);
  if (outgoing.length === 0) {
    this.killPulse(pulse);
    return;
  }

  const currentNode = this.nodeById(pulse.currentNodeId);
  const orderedOutgoing = currentNode?.type === 'splitter'
    ? [...outgoing].sort((a, b) => splitterOrder(a, b, currentNode.splitterPriority))
    : outgoing;

  if (currentNode?.type === 'splitter' && orderedOutgoing.length > 1) {
    for (const link of orderedOutgoing) {
      this.spawnPulse(pulse.currentNodeId, link.toId, pulse.previousNodeId, pulse.comboChainLength, pulse.visitedNodeIds);
    }
    pulse.alive = false;
    return;
  }

  const preferred = orderedOutgoing.find((link) => link.toId !== pulse.previousNodeId) ?? orderedOutgoing[0];
  pulse.nextNodeId = preferred.toId;
  pulse.progress = 0;
}

private emitFromNode(nodeId: number, previousNodeId: number | undefined, combo: number, visited: number[]): void {
  const outgoing = this.outgoingLinks(nodeId);
  if (outgoing.length === 0) {
    return;
  }
  for (const link of outgoing) {
    this.spawnPulse(nodeId, link.toId, previousNodeId, combo, visited.length > 0 ? visited : [nodeId]);
  }
}

private spawnPulse(
  currentNodeId: number,
  nextNodeId: number,
  previousNodeId: number | undefined,
  comboChainLength: number,
  visitedNodeIds: number[]
): void {
  this.pulses.push({
    id: this.nextPulseId,
    currentNodeId,
    previousNodeId,
    nextNodeId,
    progress: 0,
    speed: this.tutorialActive ? TUTORIAL_PULSE_SPEED : NORMAL_PULSE_SPEED,
    ageMs: 0,
    energy: 1,
    comboChainLength,
    delayMs: 0,
    alive: true,
    visitedNodeIds: [...visitedNodeIds]
  });
  this.nextPulseId += 1;
}

private killPulse(pulse: PulseState): void {
  pulse.alive = false;
  this.deadEndNodeId = pulse.currentNodeId;
  this.darkEnergy = clamp(this.darkEnergy - 5.6, 0, MAX_ENERGY);
  this.multiplier = 1;
  this.lastInputResult = { ok: false, kind: 'invalid', message: 'DEAD END' };
}

private addScore(base: number, energyGain: number): void {
  this.score += Math.round(base * this.multiplier);
  this.darkEnergy = clamp(this.darkEnergy + energyGain, 0, MAX_ENERGY);
}

private outgoingLinks(nodeId: number): PulseLink[] {
  return this.links.filter((link) => link.fromId === nodeId);
}

private nodeById(nodeId: number): PulseNode | undefined {

```

```

    return this.level.nodes.find((node) => node.id === nodeId);
}

private updateTimedVisuals(dtMs: number): void {
    for (const node of this.level.nodes) {
        node.activationMs = Math.max(0, node.activationMs - dtMs);
        node.scoreCooldownMs = Math.max(0, node.scoreCooldownMs - dtMs);
        node.stabilizedMs = Math.max(0, node.stabilizedMs - dtMs);
    }
    for (const link of this.links) {
        link.flashMs = Math.max(0, link.flashMs - dtMs);
    }
    for (let index = this.lenses.length - 1; index >= 0; index -= 1) {
        const lens = this.lenses[index];
        lens.ageMs += dtMs;
        if (lens.ageMs >= lens.durationMs) {
            this.lenses.splice(index, 1);
        }
    }
}

private expireTemporaryLinks(dtMs: number): void {
    for (let index = this.links.length - 1; index >= 0; index -= 1) {
        const link = this.links[index];
        if (!link.temporary) {
            continue;
        }
        link.ageMs += dtMs;
        if (link.ageMs >= link.expiresMs) {
            this.links.splice(index, 1);
        }
    }
}

private createLens(path: WorldPoint[], success: boolean, fromId?: number, toId?: number): void {
    this.lenses.push({
        id: this.nextLensId,
        path,
        fromId,
        toId,
        ageMs: 0,
        durationMs: LENS_DURATION_MS,
        success,
        message: success ? 'BRIDGE CREATED' : 'NO ANCHOR'
    });
    this.nextLensId += 1;
}

private endRun(reason: Exclude<PulseSnapshot['endReason'], undefined>): void {
    if (this.phase === 'ended') {
        return;
    }
    this.phase = 'ended';
    this.endReason = reason;
    this.collapsed = reason === 'collapsed';
    this.stabilized = reason === 'stabilized';
    if (this.stabilized) {
        this.addScore(350 + (this.level.linkBudget - this.links.filter((link) => !link.temporary).length) * 80, 0);
    }
    this.pulses = [];
    this.updateHash();
}

private getResult(): PulseResult {
    return {
        score: this.score,
        survivalMs: Math.round(this.timeMs),
        maxMultiplier: this.maxMultiplier,
        loopsCompleted: this.loopsCompleted,
        linksUsed: this.links.filter((link) => !link.temporary).length,
        bestChainLength: this.chainLength,
        energyNodesHit: this.energyNodesHit.size,
        stabilized: this.stabilized,
        collapsed: this.collapsed
    };
}

private tutorialHint(): string {
    if (this.tutorialActive) {
        if (this.tutorialStep === 'swipe-chain') {
            return 'SWIPE THROUGH THESE NODES';
        }
        if (this.tutorialStep === 'tap-splitter') {
            return 'TAP THE SPLITTER TO AIM IT';
        }
        if (this.tutorialStep === 'press-play') {
            return 'PRESS PLAY';
        }
        if (this.tutorialStep === 'stabilize') {
            return 'TAP THE NEXT NODE TO STABILIZE';
        }
        if (this.tutorialStep === 'lens') {
            return 'SWIPE BETWEEN NODES TO CREATE A HORIZON LENS';
        }
        if (this.tutorialStep === 'loops') {
            return 'BUILD LOOPS TO DELAY COLLAPSE';
        }
    }
    if (this.phase === 'build') {
        if (this.links.filter((link) => !link.temporary).length === 0) {
            return 'SWIPE THROUGH NODES TO DRAW A CHAIN';
        }
        return 'PRESS PLAY';
    }
    if (this.phase === 'pulse' && this.liveInputs.length === 0) {
        return 'SWIPE TO CREATE A HORIZON LENS';
    }
    return this.phase === 'ended' ? (this.stabilized ? 'SECTOR STABILIZED' : 'GALAXY COLLAPSED') : 'WATCH THE PULSE';
}

private advanceTutorial(event: 'chain' | 'chain-miss' | 'splitter' | 'play' | 'stabilize' | 'lens'): void {
    if (!this.tutorialActive) {
        return;
    }
    if (this.tutorialStep === 'swipe-chain' && event === 'chain' && this.links.some((link) => link.fromId === 1 && link.toId === 2) && this.links.some((link) => link.fromId === 2 && link.toId === 1)) {
        this.tutorialStep = 'tap-splitter';
        this.lastInputResult = { ...this.lastInputResult, message: 'CHAIN CREATED. TAP THE SPLITTER' };
    } else if (this.tutorialStep === 'tap-splitter' && event === 'splitter') {
        this.tutorialStep = 'press-play';
    } else if (this.tutorialStep === 'press-play' && event === 'play') {
        this.tutorialStep = 'stabilize';
    } else if (this.tutorialStep === 'stabilize' && event === 'stabilize') {
        this.tutorialStep = 'lens';
    } else if (this.tutorialStep === 'lens' && event === 'lens') {
        this.tutorialStep = 'loops';
    }
}

private tutorialHighlightNodeIds(): number[] {
    if (!this.tutorialActive) {
        return [];
    }
}

```

```

    if (this.tutorialStep === 'swipe-chain') {
      return [1, 2, 3];
    }
    if (this.tutorialStep === 'tap-splitter') {
      return [4];
    }
    if (this.tutorialStep === 'stabilize') {
      return [3, 4];
    }
    if (this.tutorialStep === 'lens') {
      return [3, 4];
    }
    return [];
  }

  private tutorialGhostPath(): WorldPoint[] {
    const ids = this.tutorialHighlightNodeIds();
    if (ids.length < 2) {
      return [];
    }
    return ids
      .map((id) => this.nodeById(id))
      .filter((node): node is PulseNode => node !== undefined)
      .map((node) => ({ x: node.x, y: node.y }));
  }

  private computeChainAnalysis(): ChainAnalysis {
    const totalEnergyNodes = this.level.nodes.filter((node) => node.type === 'energy').length;
    const reachable = this.reachableFromSource();
    const reachableEnergyNodes = this.level.nodes.filter((node) => node.type === 'energy' && reachable.has(node.id)).length;
    const deadEndNodeIds = [...reachable].filter((nodeId) => nodeId !== this.level.sourceId && this.outgoingLinks(nodeId).filter((link) => !link.temporary).length === 0);
    const hasLoop = this.hasReachableLoop();
    const linksUsed = this.links.filter((link) => !link.temporary).length;
    let quality: ChainAnalysis['quality'] = 'Draw a chain';
    if (linksUsed > 0) {
      quality = 'Good start';
    }
    if (deadEndNodeIds.length > 0) {
      quality = 'Dead end detected';
    }
    } else if (hasLoop && reachableEnergyNodes >= 2) {
      quality = 'Great loop';
    }
    } else if (hasLoop) {
      quality = 'Loop possible';
    }
    } else if (reachableEnergyNodes < Math.min(2, totalEnergyNodes)) {
      quality = 'Hit more Energy nodes';
    }
  }
  return { reachableEnergyNodes, totalEnergyNodes, deadEndNodeIds, hasLoop, linksUsed, quality };
}

  private computeSuggestedFixes(): SuggestedFix[] {
    const analysis = this.computeChainAnalysis();
    const fromId = this.deadEndNodeIds[0];
    const from = fromId ? this.nodeById(fromId) : undefined;
    if (!from) {
      return [];
    }
    return this.level.nodes
      .filter((node) => node.id !== from.id && !this.links.some((link) => link.fromId === from.id && link.toId === node.id))
      .sort((a, b) => Math.hypot(a.x - from.x, a.y - from.y) - Math.hypot(b.x - from.x, b.y - from.y))
      .slice(0, 2)
      .map((node) => ({ fromId: from.id, toId: node.id, message: `Try linking ${from.label} to ${node.label}` }));
  }

  private reachableFromSource(): Set<number> {
    const seen = new Set<number>();
    const queue = [this.level.sourceId];
    while (queue.length > 0) {
      const nodeId = queue.shift();
      if (nodeId === undefined || seen.has(nodeId)) {
        continue;
      }
      seen.add(nodeId);
      for (const link of this.outgoingLinks(nodeId).filter((candidate) => !candidate.temporary)) {
        queue.push(link.toId);
      }
    }
    return seen;
  }

  private hasReachableLoop(): boolean {
    const reachable = this.reachableFromSource();
    const visiting = new Set<number>();
    const visited = new Set<number>();
    const visit = (nodeId: number): boolean => {
      if (!reachable.has(nodeId)) {
        return false;
      }
      if (visiting.has(nodeId)) {
        return true;
      }
      if (visited.has(nodeId)) {
        return false;
      }
      visiting.add(nodeId);
      for (const link of this.outgoingLinks(nodeId).filter((candidate) => !candidate.temporary)) {
        if (visit(link.toId)) {
          return true;
        }
      }
      visiting.delete(nodeId);
      visited.add(nodeId);
      return false;
    };
    return visit(this.level.sourceId);
  }

  private arrivalReadiness(nodeId: number): 'perfect' | 'soon' | 'early' | 'late' {
    for (const pulse of this.pulses) {
      if (pulse.nextNodeId === nodeId) {
        if (pulse.progress >= 0.78) {
          return 'perfect';
        }
        if (pulse.progress >= 0.42) {
          return 'soon';
        }
        return 'early';
      }
      if (pulse.currentNodeId === nodeId) {
        return 'late';
      }
    }
    return 'early';
  }

  private updateHash(): void {
    const data = [
      this.phase,
      Math.round(this.timeMs),
      this.score,
      Math.round(this.darkEnergy * 10),
      this.multiplier,
      this.tutorialStep,
      this.level.nodes.map((node) => `${node.id}:${node.primed ? 1 : 0}:${node.delayLevel}:${node.splitterPriority}`).join('|'),
    ];
  }

```

```

    this.links.map((link) => `${link.fromId}>${link.toId}:${link.temporary ? Math.round(link.expiresMs - link.ageMs) : 0}`).join('|'),
    this.pulses.map((pulse) => `${pulse.currentNodeId}>${pulse.nextNodeId ?? 0}:${Math.round(pulse.progress * 1000)}:${Math.round(pulse.delayMs)}').join('|')
  ].join(';');
  let hash = 2166136261;
  for (let index = 0; index < data.length; index += 1) {
    hash ^= data.charCodeAt(index);
    hash = Math.imul(hash, 16777619);
  }
  this.stepHash = (hash >>> 0).toString(16).padStart(8, '0');
}

function multiplierForChain(chainLength: number): number {
  if (chainLength >= 15) {
    return 8;
  }
  if (chainLength >= 10) {
    return 5;
  }
  if (chainLength >= 7) {
    return 3;
  }
  if (chainLength >= 4) {
    return 2;
  }
  return 1;
}

function splitterOrder(a: PulseLink, b: PulseLink, priority: number): number {
  const aValue = (a.toId + priority * 7) % 13;
  const bValue = (b.toId + priority * 7) % 13;
  return aValue - bValue;
}

```

src/game/pulse/PulseTypes.ts

```

import type { WorldPoint } from '../gestures';

export type PulseGamePhase = 'build' | 'pulse' | 'ended';
export type PulseEndReason = 'collapsed' | 'stabilized' | 'pulse-died' | 'manual';
export type PulseNodeType = 'source' | 'conduit' | 'energy' | 'delay' | 'splitter';
export type TutorialStep = 'swipe-chain' | 'tap-splitter' | 'press-play' | 'stabilize' | 'lens' | 'loops' | 'complete' | 'skipped';
export type NodeTapAction = 'select' | 'prime' | 'delay' | 'splitter' | 'stabilize' | 'none';

export interface PulseNode extends WorldPoint {
  id: number;
  type: PulseNodeType;
  radius: number;
  ring: number;
  label: string;
  activationMs: number;
  scoreCooldownMs: number;
  primed: boolean;
  delayLevel: 0 | 1 | 2;
  splitterPriority: number;
  stabilizedMs: number;
}

export interface PulseLink {
  id: number;
  fromId: number;
  toId: number;
  temporary: boolean;
  ageMs: number;
  expiresMs: number;
  flashMs: number;
}

export interface PulseState {
  id: number;
  currentNodeId: number;
  previousNodeId?: number;
  nextNodeId?: number;
  progress: number;
  speed: number;
  ageMs: number;
  energy: number;
  comboChainLength: number;
  delayMs: number;
  alive: boolean;
  visitedNodeIds: number[];
}

export interface HorizonLens {
  id: number;
  path: WorldPoint[];
  fromId?: number;
  toId?: number;
  ageMs: number;
  durationMs: number;
  success: boolean;
  message: 'HORIZON LENS' | 'BRIDGE CREATED' | 'NO ANCHOR';
}

export interface ChainAnalysis {
  reachableEnergyNodes: number;
  totalEnergyNodes: number;
  deadEndNodeIds: number[];
  hasLoop: boolean;
  linksUsed: number;
  quality: 'Draw a chain' | 'Good start' | 'Hit more Energy nodes' | 'Dead end detected' | 'Loop possible' | 'Great loop';
}

export interface SuggestedFix {
  fromId: number;
  toId: number;
  message: string;
}

export interface PulseLevel {
  seed: string;
  nodes: PulseNode[];
  sourceId: number;
  linkBudget: number;
  targetScore: number;
  targetSurvivalMs: number;
}

export type BuildInput =
  | { t: number; kind: 'link'; fromId: number; toId: number }
  | { t: number; kind: 'chainSwipe'; nodeIds: number[]; path: { x: number; y: number; t: number }[] }
  | { t: number; kind: 'nodeTap'; nodeId: number; action: NodeTapAction }
  | { t: number; kind: 'undo' }
  | { t: number; kind: 'clear' }
  | { t: number; kind: 'play' };

export type LiveInput =
  | {
    t: number;
    kind: 'lens';
  }

```

```

    path: { x: number; y: number; t: number }[];
    fromId?: number;
    toId?: number;
    success: boolean;
  }
}

| {
  t: number;
  kind: 'stabilize';
  nodeId: number;
  rating: 'perfect' | 'stabilized' | 'early' | 'late';
  success: boolean;
};
});

export interface PulseResult {
  score: number;
  survivalMs: number;
  maxMultiplier: number;
  loopsCompleted: number;
  linksUsed: number;
  bestChainLength: number;
  energyNodesHit: number;
  stabilized: boolean;
  collapsed: boolean;
}

export interface PulseReplayPayload {
  version: 1;
  mode: 'pulse-chain';
  seed: string;
  startedAt: number;
  buildInputs: BuildInput[];
  liveInputs: LiveInput[];
  result: PulseResult;
  stepHash: string;
}

export interface PulseInputResult {
  ok: boolean;
  kind: 'select' | 'link' | 'chainSwipe' | 'nodeTap' | 'stabilize' | 'undo' | 'clear' | 'play' | 'lens' | 'fix' | 'invalid' | 'none';
  message: string;
  fromId?: number;
  toId?: number;
  nodeId?: number;
  nodeIds?: number[];
  scoreDelta?: number;
  energyDelta?: number;
}

export interface PulseSnapshot {
  mode: 'pulse-chain';
  seed: string;
  phase: PulseGamePhase;
  timeMs: number;
  score: number;
  darkEnergy: number;
  collapseMeter: number;
  multiplier: number;
  maxMultiplier: number;
  chainLength: number;
  loopsCompleted: number;
  linkBudget: number;
  linksUsed: number;
  lensCharges: number;
  targetScore: number;
  targetSurvivalMs: number;
  ended: boolean;
  endReason?: PulseEndReason;
  collapsed: boolean;
  stabilized: boolean;
  nodes: readonly PulseNode[];
  links: readonly PulseLink[];
  pulses: readonly PulseState[];
  lenses: readonly HorizonLens[];
  selectedNodeId?: number;
  tutorialHint: string;
  tutorialActive: boolean;
  tutorialStep: TutorialStep;
  tutorialHighlightNodeIds: readonly number[];
  tutorialGhostPath: readonly WorldPoint[];
  chainAnalysis: ChainAnalysis;
  suggestedFixes: readonly SuggestedFix[];
  lastChainNodeIds: readonly number[];
  lastTapAction: NodeTapAction;
  deadEndNodeId?: number;
  lastInputResult: PulseInputResult;
  stepHash: string;
}

```

src/main.ts

```

import './styles.css';
import { EventHorizonGame } from './game/EventHorizonGame';
import { PulseMode } from './game/pulse/PulseMode';

interface EventHorizonRuntime {
  start: () => Promise<void>;
  restart: () => void;
  setPaused: (paused: boolean) => void;
  setInputDebug: (enabled: boolean) => void;
  exportPoster: () => Promise<string>;
  forceEnd: () => void;
  getSnapshot: () => unknown;
  getReplayPayload: () => unknown;
  destroy?: () => void;
}

const root = document.querySelector<HTMLDivElement>('#game-root');
const restartButton = document.querySelector<HTMLButtonElement>('#restart-button');
const shareButton = document.querySelector<HTMLButtonElement>('#share-button');
const helpButton = document.querySelector<HTMLButtonElement>('#help-button');
const helpOverlay = document.querySelector<HTMLDivElement>('#help-overlay');
const helpPlayButton = document.querySelector<HTMLButtonElement>('#help-play-button');
const helpSkipButton = document.querySelector<HTMLButtonElement>('#help-skip-button');
const posterLink = document.querySelector<HTMLAnchorElement>('#poster-link');
const pulseControls = document.querySelector<HTMLDivElement>('#pulse-controls');
const pulseUndoButton = document.querySelector<HTMLButtonElement>('#pulse-undo-button');
const pulseClearButton = document.querySelector<HTMLButtonElement>('#pulse-clear-button');
const pulsePlayButton = document.querySelector<HTMLButtonElement>('#pulse-play-button');

if (
  !root ||
  !restartButton ||
  !shareButton ||
  !helpButton ||
  !helpOverlay ||
  !helpPlayButton ||
  !helpSkipButton ||
  !posterLink ||
  !pulseControls ||
  !pulseUndoButton ||

```

```

!pulseClearButton ||
!pulsePlayButton
) {
  throw new Error('Event Horizon shell is missing required DOM nodes.');
```

```

}

const params = new URLSearchParams(window.location.search);
const mode = params.get('mode') === 'legacy' ? 'legacy' : 'pulse-chain';
const debugInput = params.get('debugInput') === '1';
const seed = params.get('seed') ?? 'tutorial-001';
```

```

const game: EventHorizonRuntime =
  mode === 'legacy'
    ? new EventHorizonGame(root, {
        seed: 'eh-2026-05-29-alpha',
        startedAt: 1780051200000
      })
    : new PulseMode(root, {
        seed,
        startedAt: Date.now()
      });
```

```

await game.start();
game.setInputDebug(debugInput);
pulseControls.hidden = mode === 'legacy';
let pulsePaused = false;
```

```

const helpKey = mode === 'legacy' ? 'eventHorizon.helpSeen' : 'eventHorizon.iteration04HelpSeen';
```

```

const hasSeenHelp = (): boolean => {
  try {
    return localStorage.getItem(helpKey) === '1';
  } catch {
    return false;
  }
};
```

```

const markHelpSeen = (): void => {
  try {
    localStorage.setItem(helpKey, '1');
  } catch {
    // localStorage can be unavailable in locked-down browser modes.
  }
};
```

```

const openHelp = (): void => {
  helpOverlay.hidden = false;
  helpPlayButton.textContent = mode === 'legacy' ? 'PLAY' : 'START TUTORIAL';
  helpSkipButton.hidden = mode === 'legacy';
  game.setPaused(true);
};
```

```

const closeHelp = (startTutorial: boolean): void => {
  helpOverlay.hidden = true;
  markHelpSeen();
  if (game instanceof PulseMode) {
    if (startTutorial) {
      game.startTutorial();
    } else {
      game.skipTutorial();
    }
  }
  game.setPaused(false);
  updatePulseControls();
};
```

```

if (!hasSeenHelp()) {
  openHelp();
}
```

```

restartButton.addEventListener('click', () => {
  posterLink.removeAttribute('href');
  pulsePaused = false;
  game.restart();
  updatePulseControls();
});
```

```

helpButton.addEventListener('click', openHelp);
helpPlayButton.addEventListener('click', () => closeHelp(true));
helpSkipButton.addEventListener('click', () => closeHelp(false));
```

```

shareButton.addEventListener('click', async () => {
  const poster = await game.exportPoster();
  posterLink.href = poster;
});
```

```

if (game instanceof PulseMode) {
  pulseUndoButton.addEventListener('click', () => {
    const snapshot = game.getSnapshot();
    if (snapshot.phase === 'build') {
      game.undo();
    } else if (snapshot.phase === 'pulse') {
      pulsePaused = !pulsePaused;
      game.setPaused(pulsePaused);
    } else if (snapshot.endReason === 'pulse-died') {
      pulsePaused = false;
      game.fixChain();
    } else {
      pulsePaused = false;
      game.restart();
    }
    updatePulseControls();
  });
  pulseClearButton.addEventListener('click', () => {
    const snapshot = game.getSnapshot();
    if (snapshot.phase === 'build') {
      game.clearLinks();
    } else {
      pulsePaused = false;
      game.restart();
    }
    updatePulseControls();
  });
  pulsePlayButton.addEventListener('click', () => {
    if (game.getSnapshot().phase === 'build') {
      game.playPulse();
      pulsePaused = false;
      updatePulseControls();
    } else if (game.getSnapshot().phase === 'ended') {
      const nextSeed = `seed-${Date.now().toString(36)}`;
      window.location.href = `${window.location.pathname}?seed=${encodeURIComponent(nextSeed)}`;
    }
  });
  window.setInterval(updatePulseControls, 250);
  updatePulseControls();
} else {
  pulseControls.hidden = true;
}
```

```

function updatePulseControls(): void {
  if (!(game instanceof PulseMode)) {
    return;
  }
}
```



```

const controls = pulseControls;
const undoButton = pulseUndoButton;
const clearButton = pulseClearButton;
const playButton = pulsePlayButton;
if (!controls || !undoButton || !clearButton || !playButton) {
  return;
}
const snapshot = game.getSnapshot();
controls.dataset.phase = snapshot.phase;
if (snapshot.phase === 'build') {
  undoButton.hidden = false;
  clearButton.hidden = false;
  playButton.hidden = false;
  undoButton.textContent = 'Undo';
  clearButton.textContent = 'Clear';
  playButton.textContent = 'Play';
  playButton.disabled = snapshot.linksUsed === 0;
  return;
}
if (snapshot.phase === 'pulse') {
  undoButton.hidden = false;
  clearButton.hidden = false;
  playButton.hidden = true;
  undoButton.textContent = pulsePaused ? 'Resume' : 'Pause';
  clearButton.textContent = 'Restart';
  playButton.disabled = true;
  return;
}
undoButton.hidden = false;
clearButton.hidden = false;
playButton.hidden = false;
undoButton.textContent = snapshot.endReason === 'pulse-died' ? 'Fix Chain' : 'Replay';
clearButton.textContent = 'Replay';
playButton.textContent = 'New Seed';
playButton.disabled = false;
}

```

```

declare global {
  interface Window {
    __EVENT_HORIZON__?: {
      exportPoster: () => Promise<string>;
      forceEnd: () => void;
      getReplayPayload: () => unknown;
      getSnapshot: () => unknown;
      restart: () => void;
    };
    __EVENT_HORIZON_DEBUG__?: {
      addLink: (fromId: number, toId: number) => unknown;
      clearLinks: () => unknown;
      forceBuildPhase: () => void;
      forceCollapse: () => void;
      forceHelp: (open: boolean) => void;
      forcePulsePhase: () => void;
      getLastInputResult: () => unknown;
      getLinks: () => unknown;
      getMode: () => string;
      getNodes: () => unknown;
      getPulses: () => unknown;
      getReplayPayload: () => unknown;
      getSnapshot: () => unknown;
      playPulse: () => unknown;
      startTutorial: () => void;
      skipTutorial: () => void;
      getTutorialStep: () => unknown;
      simulateChainSwipe: (nodeIds: number[]) => unknown;
      analyzeChain: () => unknown;
      getSuggestedFixes: () => unknown;
      primeNode: (id: number) => unknown;
      cycleNode: (id: number) => unknown;
      stabilizeNode: (id: number) => unknown;
      setInputDebug: (enabled: boolean) => void;
      simulateLens: (points: { x: number; y: number }[]) => unknown;
    };
  }
}

```

```

window.__EVENT_HORIZON__ = {
  exportPoster: () => game.exportPoster(),
  forceEnd: () => game.forceEnd(),
  getReplayPayload: () => game.getReplayPayload(),
  getSnapshot: () => game.getSnapshot(),
  restart: () => game.restart()
};

```

```

window.__EVENT_HORIZON_DEBUG__ =
  game instanceof PulseMode
    ? {
        addLink: (fromId, toId) => game.addLink(fromId, toId),
        clearLinks: () => game.clearLinks(),
        forceBuildPhase: () => game.forceBuildPhase(),
        forceCollapse: () => game.forceCollapse(),
        forceHelp: (open) => {
          if (open) {
            openHelp();
          } else {
            closeHelp(false);
          }
        },
        forcePulsePhase: () => game.forcePulsePhase(),
        getLastInputResult: () => game.getLastInputResult(),
        getLinks: () => game.getLinks(),
        getMode: () => game.getMode(),
        getNodes: () => game.getNodes(),
        getPulses: () => game.getPulses(),
        getReplayPayload: () => game.getReplayPayload(),
        getSnapshot: () => game.getSnapshot(),
        playPulse: () => game.playPulse(),
        startTutorial: () => game.startTutorial(),
        skipTutorial: () => game.skipTutorial(),
        getTutorialStep: () => game.getTutorialStep(),
        simulateChainSwipe: (nodeIds) => game.simulateChainSwipe(nodeIds),
        analyzeChain: () => game.analyzeChain(),
        getSuggestedFixes: () => game.getSuggestedFixes(),
        primeNode: (id) => game.primeNode(id),
        cycleNode: (id) => game.cycleNode(id),
        stabilizeNode: (id) => game.stabilizeNode(id),
        setInputDebug: (enabled) => game.setInputDebug(enabled),
        simulateLens: (points) => game.simulateLens(points)
      }
    : {
        addLink: () => undefined,
        clearLinks: () => undefined,
        forceBuildPhase: () => undefined,
        forceCollapse: () => game.forceEnd(),
        forceHelp: (open) => {
          if (open) {
            openHelp();
          } else {
            closeHelp(false);
          }
        },
        forcePulsePhase: () => undefined,
        getLastInputResult: () => undefined,

```

```

getLinks: () => [],
getMode: () => 'legacy',
getNodes: () => [],
getPulses: () => [],
getReplayPayload: () => game.getReplayPayload(),
getSnapshot: () => game.getSnapshot(),
playPulse: () => undefined,
startTutorial: () => undefined,
skipTutorial: () => undefined,
getTutorialStep: () => undefined,
simulateChainSwipe: () => undefined,
analyzeChain: () => undefined,
getSuggestedFixes: () => [],
primeNode: () => undefined,
cycleNode: () => undefined,
stabilizeNode: () => undefined,
setInputDebug: (enabled) => game.setInputDebug(enabled),
simulateLens: () => undefined
};

```

src/styles.css

```

:root {
  color-scheme: dark;
  font-family:
    Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  background: #03040a;
  color: #f7fbff;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
  -webkit-font-smoothing: antialiased;
}

* {
  box-sizing: border-box;
  -webkit-tap-highlight-color: transparent;
}

html,
body,
#app,
#game-shell,
#game-root {
  width: 100%;
  height: 100%;
  margin: 0;
  overflow: hidden;
  overscroll-behavior: none;
  touch-action: none;
  user-select: none;
  -webkit-user-select: none;
  -webkit-touch-callout: none;
}

body {
  position: fixed;
  inset: 0;
  background:
    radial-gradient(circle at 50% 42%, rgba(22, 49, 89, 0.24), transparent 38%),
    linear-gradient(180deg, #070912 0%, #03040a 100%);
}

button,
a {
  -webkit-tap-highlight-color: transparent;
}

#game-shell {
  position: fixed;
  inset: 0;
  display: grid;
  place-items: center;
  background: #03040a;
}

#game-root {
  position: absolute;
  inset: 0;
  overflow: hidden;
}

#game-root canvas {
  display: block;
  width: 100%;
  height: 100%;
  touch-action: none;
  user-select: none;
  -webkit-user-select: none;
  -webkit-touch-callout: none;
}

#status-layer {
  position: fixed;
  top: max(12px, env(safe-area-inset-top));
  right: max(12px, env(safe-area-inset-right));
  z-index: 2;
  display: flex;
  gap: 8px;
}

#pulse-controls {
  position: fixed;
  right: max(14px, env(safe-area-inset-right));
  bottom: max(22px, env(safe-area-inset-bottom));
  left: max(14px, env(safe-area-inset-left));
  z-index: 3;
  display: grid;
  grid-template-columns: 1fr 1fr 1.45fr;
  gap: 10px;
  pointer-events: auto;
}

#pulse-controls[hidden] {
  display: none;
}

#pulse-controls[data-phase="pulse"],
#pulse-controls[data-phase="ended"] {
  grid-template-columns: repeat(3, 1fr);
}

#restart-button,
#share-button,
#help-button,
#poster-link,
#pulse-controls button {
  display: grid;
  place-items: center;
  border: 1px solid rgba(179, 226, 255, 0.26);
  border-radius: 8px;
}

```

```

background: rgba(7, 12, 25, 0.72);
color: #f7fbff;
font-size: 21px;
line-height: 1;
text-decoration: none;
backdrop-filter: blur(8px);
touch-action: manipulation;
}

#restart-button,
#share-button,
#help-button,
#poster-link {
width: 40px;
height: 40px;
}

#pulse-controls button {
min-height: 48px;
color: #f7fbff;
font-size: 14px;
font-weight: 900;
text-transform: uppercase;
}

#pulse-controls button:disabled {
opacity: 0.45;
}

#pulse-controls button[hidden] {
display: none;
}

#pulse-play-button {
background: linear-gradient(90deg, rgba(103, 244, 255, 0.9), rgba(210, 103, 255, 0.9));
color: #061120;
}

#poster-link:not([href]) {
display: none;
}

#help-overlay {
position: fixed;
inset: 0;
z-index: 5;
display: grid;
place-items: center;
padding: max(18px, env(safe-area-inset-top)) max(16px, env(safe-area-inset-right))
max(18px, env(safe-area-inset-bottom)) max(16px, env(safe-area-inset-left));
background: rgba(2, 4, 10, 0.82);
backdrop-filter: blur(12px);
touch-action: none;
}

#help-overlay[hidden] {
display: none;
}

#help-panel {
width: min(92vw, 440px);
max-height: min(92vh, 760px);
overflow: auto;
padding: 18px;
border: 1px solid rgba(139, 222, 255, 0.34);
border-radius: 8px;
background: rgba(7, 12, 25, 0.94);
box-shadow: 0 20px 80px rgba(0, 0, 0, 0.48);
color: #f7fbff;
}

#help-panel h1 {
margin: 10px 0 6px;
font-size: 25px;
letter-spacing: 0;
}

#help-panel h2 {
margin: 10px 0 3px;
color: #9ffcff;
font-size: 13px;
letter-spacing: 0.08em;
}

#help-panel p {
margin: 7px 0;
color: #cfefff;
font-size: 14px;
line-height: 1.36;
}

#help-panel ol {
margin: 14px 0;
padding-left: 22px;
color: #f2fbff;
font-size: 15px;
line-height: 1.45;
}

#help-panel li {
margin: 8px 0;
}

#help-play-button,
#help-skip-button {
width: 100%;
min-height: 52px;
margin-top: 12px;
border: 0;
border-radius: 8px;
background: linear-gradient(90deg, #67f4ff, #d267ff);
color: #061120;
font-size: 17px;
font-weight: 900;
letter-spacing: 0;
touch-action: manipulation;
}

#help-skip-button {
border: 1px solid rgba(179, 226, 255, 0.24);
background: rgba(7, 12, 25, 0.84);
color: #f7fbff;
}

.help-example {
position: relative;
height: 96px;
margin: 2px 0 8px;
overflow: hidden;
border-radius: 8px;
background:
radial-gradient(circle at 50% 44%, rgba(82, 28, 120, 0.28), transparent 34%),
radial-gradient(circle at 72% 58%, rgba(103, 244, 255, 0.14), transparent 18%),

```

```

    #03040a;
  }

  .help-hole,
  .help-orb,
  .help-tether,
  .help-swipe {
    position: absolute;
    display: block;
  }

  .help-hole {
    left: 44%;
    top: 38%;
    width: 48px;
    height: 48px;
    border: 5px solid rgba(103, 244, 255, 0.34);
    border-radius: 50%;
    background: #000;
    box-shadow: 0 0 34px rgba(103, 244, 255, 0.22);
  }

  .help-orb {
    left: 70%;
    top: 58%;
    width: 30px;
    height: 30px;
    border-radius: 50%;
    background: #d9fbff;
    box-shadow: 0 0 20px #67f4ff, 0 0 46px rgba(210, 103, 255, 0.7);
  }

  .help-tether {
    left: 52%;
    top: 54%;
    width: 105px;
    height: 4px;
    border-radius: 999px;
    background: #67f4ff;
    box-shadow: 0 0 18px #67f4ff;
    transform: rotate(18deg);
    transform-origin: left center;
  }

  .help-swipe {
    left: 58%;
    top: 42%;
    width: 104px;
    height: 42px;
    border-bottom: 7px solid #ffffff;
    border-left: 7px solid transparent;
    border-radius: 0 0 80px 80px;
    box-shadow: 0 12px 20px rgba(210, 103, 255, 0.5);
    transform: rotate(-22deg);
  }

  @media (min-width: 820px) {
    #game-root {
      inset: 0;
    }
  }

```

tests/e2e/playable.spec.ts

```

import { expect, test, type Page } from '@playwright/test';

const WORLD_WIDTH = 1080;
const WORLD_HEIGHT = 1920;

test('first visit opens updated help and tutorial', async ({ page }) => {
  await openGame(page);
  await expect(page.locator('#help-overlay')).toBeVisible();
  await expect(page.locator('#help-overlay')).toContainText('Build a chain. Then keep it alive.');
```

await expect(page.locator('#help-play-button')).toHaveText('START TUTORIAL');

});

test('tutorial Step 1 highlights nodes and swipe creates a chain', async ({ page }) => {
 await startTutorial(page);
 const snapshot = await getSnapshot(page);
 expect(snapshot.tutorialStep).toBe('swipe-chain');

expect(snapshot.tutorialHighlightNodeIds).toEqual([1, 2, 3]);

await swipeThroughNodes(page, [1, 2, 3]);

await page.waitForTimeout(160);

const after = await getSnapshot(page);

expect(after.linksUsed).toBe(2);

expect(after.tutorialStep).toBe('tap-splitter');

expect(after.lastChainNodeIds).toEqual([1, 2, 3]);

});

test('tapping splitter changes tutorial state and output priority', async ({ page }) => {
 await tutorialChainReady(page);
 const before = await nodeById(page, 4);
 await tapNode(page, 4);
 const after = await nodeById(page, 4);
 expect(after.splitterPriority).not.toBe(before.splitterPriority);
 expect((await getSnapshot(page)).tutorialStep).toBe('press-play');
});

test('pressing Play launches visible pulse and node tap stabilizes it', async ({ page }) => {
 await tutorialReadyToPlay(page);
 await page.locator('#pulse-play-button').click();
 await page.waitForFunction(() => (window.__EVENT_HORIZON__?.getSnapshot() as { phase?: string } | undefined)?.phase === 'pulse');

await page.waitForTimeout(2200);

await tapNode(page, 3);

await page.waitForTimeout(120);

const snapshot = await getSnapshot(page);

expect(snapshot.lastInputResult.kind).toBe('stabilize');

expect(snapshot.lastInputResult.ok).toBe(true);

expect(snapshot.tutorialStep).toBe('lens');

});

test('swiping Horizon Lens creates bridge and replay records grammar', async ({ page }) => {
 await tutorialPulseReadyForLens(page);
 await swipeThroughNodes(page, [3, 4]);
 await page.waitForTimeout(160);
 const snapshot = await getSnapshot(page);
 const replay = await page.evaluate(() => window.__EVENT_HORIZON__?.getReplayPayload()) as {
 buildInputs: Array<{ kind: string }>;
 liveInputs: Array<{ kind: string; success?: boolean }>;
 };
 expect(snapshot.lastInputResult.kind).toBe('lens');

expect(snapshot.lastInputResult.ok).toBe(true);

expect(snapshot.links.some((link) => link.temporary && link.fromId === 3 && link.toId === 4)).toBe(true);

expect(replay.buildInputs.some((input) => input.kind === 'chainSwipe')).toBe(true);

expect(replay.buildInputs.some((input) => input.kind === 'nodeTap')).toBe(true);

expect(replay.buildInputs.some((input) => input.kind === 'play')).toBe(true);

expect(replay.liveInputs.some((input) => input.kind === 'stabilize')).toBe(true);

expect(replay.liveInputs.some((input) => input.kind === 'lens')).toBe(true);

});

```

test('dead-end failure shows suggested fix and FIX_CHAIN returns to build', async ({ page }) => {
  await startTutorial(page);
  await page.evaluate(() => {
    window.__EVENT_HORIZON_DEBUG__?.skipTutorial();
    window.__EVENT_HORIZON_DEBUG__?.clearLinks();
    window.__EVENT_HORIZON_DEBUG__?.addLink(1, 2);
    window.__EVENT_HORIZON_DEBUG__?.playPulse();
  });
  await page.waitForFunction(() => (window.__EVENT_HORIZON__?.getSnapshot() as { phase?: string } | undefined)?.phase === 'ended', null, { timeout: 9000 });
  const ended = await getSnapshot(page);
  expect(ended.endReason).toBe('pulse-died');
  expect(ended.suggestedFixes.length).toBeGreaterThan(0);
  await page.locator('#pulse-undo-button').click();
  await page.waitForTimeout(120);
  expect((await getSnapshot(page)).phase).toBe('build');
});

async function openGame(page: Page): Promise<void> {
  await page.addInitScript(() => {
    window.localStorage.clear();
  });
  await page.goto('./?seed=tutorial-001&debugInput=1');
  await page.locator('canvas').waitFor({ state: 'visible' });
}

async function startTutorial(page: Page): Promise<void> {
  await openGame(page);
  await page.locator('#help-play-button').click();
  await expect(page.locator('#help-overlay')).toBeHidden();
}

async function tutorialChainReady(page: Page): Promise<void> {
  await startTutorial(page);
  await swipeThroughNodes(page, [1, 2, 3]);
  await page.waitForFunction(() => (window.__EVENT_HORIZON__?.getSnapshot() as { tutorialStep?: string } | undefined)?.tutorialStep === 'tap-splitter');
}

async function tutorialReadyToPlay(page: Page): Promise<void> {
  await tutorialChainReady(page);
  await tapNode(page, 4);
  await page.waitForFunction(() => (window.__EVENT_HORIZON__?.getSnapshot() as { tutorialStep?: string } | undefined)?.tutorialStep === 'press-play');
}

async function tutorialPulseReadyForLens(page: Page): Promise<void> {
  await tutorialReadyToPlay(page);
  await page.locator('#pulse-play-button').click();
  await page.waitForFunction(() => (window.__EVENT_HORIZON__?.getSnapshot() as { phase?: string } | undefined)?.phase === 'pulse');
  await page.waitForTimeout(2200);
  await tapNode(page, 3);
  await page.waitForFunction(() => (window.__EVENT_HORIZON__?.getSnapshot() as { tutorialStep?: string } | undefined)?.tutorialStep === 'lens');
}

async function getSnapshot(page: Page) {
  return (await page.evaluate(() => window.__EVENT_HORIZON__?.getSnapshot())) as {
    phase: string;
    tutorialStep: string;
    tutorialHighlightNodeIds: number[];
    linksUsed: number;
    lastChainNodeIds: number[];
    lastInputResult: { kind: string; ok: boolean; message: string };
    links: Array<{ fromId: number; toId: number; temporary: boolean }>;
    suggestedFixes: unknown[];
    endReason?: string;
  };
}

async function nodeById(page: Page, id: number) {
  const node = await page.evaluate((nodeId) => {
    const nodes = window.__EVENT_HORIZON_DEBUG__?.getNodes() as Array<{ id: number; x: number; y: number; splitterPriority: number }> | undefined;
    return nodes?.find((candidate) => candidate.id === nodeId);
  }, id);
  if (!node) {
    throw new Error('Missing node ${id}');
  }
  return node as { id: number; x: number; y: number; splitterPriority: number };
}

async function tapNode(page: Page, id: number): Promise<void> {
  const node = await nodeById(page, id);
  const point = await worldToScreen(page, node.x, node.y);
  await page.mouse.click(point.x, point.y);
}

async function swipeThroughNodes(page: Page, ids: number[]): Promise<void> {
  const points = [];
  for (const id of ids) {
    const node = await nodeById(page, id);
    points.push(await worldToScreen(page, node.x, node.y));
  }
  await page.mouse.move(points[0].x, points[0].y);
  await page.mouse.down();
  for (const point of points.slice(1)) {
    await page.mouse.move(point.x, point.y, { steps: 8 });
  }
  await page.mouse.up();
}

async function worldToScreen(page: Page, x: number, y: number): Promise<{ x: number; y: number }> {
  const box = await page.locator('canvas').boundingBox();
  if (!box) {
    throw new Error('Canvas box unavailable.');
```

tests/pulse-simulation.test.ts

```

import { describe, expect, it } from 'vitest';
import { generatePulseLevel } from '../src/game/pulse/PulseLevelGenerator';
import { PulseSimulation } from '../src/game/pulse/PulseSimulation';

const options = {
  seed: 'tutorial-001',
  startedAt: 1780185600000
};

describe('pulse-chain mode', () => {
  it('tutorial level is deterministic and hand-tuned', () => {
    const first = generatePulseLevel('tutorial-001');
    const second = generatePulseLevel('tutorial-001');
    expect(second).toEqual(first);
    expect(first.nodes.slice(0, 4).map((node) => node.type)).toEqual(['source', 'energy', 'delay', 'splitter']);
  });
});
```

```

it('seeded level generation is deterministic for normal seeds', () => {
  expect(generatePulseLevel('abc')).toEqual(generatePulseLevel('abc'));
  expect(generatePulseLevel('abc').nodes).not.toEqual(generatePulseLevel('xyz').nodes);
});

it('swipe crossing 3 nodes creates 2 links and records chainSwipe', () => {
  const sim = new PulseSimulation(options);
  const result = sim.applyChainSwipe(pathFor(sim, [1, 2, 3]));
  expect(result.ok).toBe(true);
  expect(sim.getLinks().map((link) => [link.fromId, link.toId])).toEqual([
    [1, 2],
    [2, 3]
  ]);
  expect(sim.getReplayPayload().buildInputs.some((input) => input.kind === 'chainSwipe')).toBe(true);
});

it('chain swipe ignores duplicate adjacent nodes and avoids self-links', () => {
  const sim = new PulseSimulation(options);
  sim.applyChainSwipe(pathFor(sim, [1, 2, 2, 3]));
  expect(sim.getLinks().map((link) => [link.fromId, link.toId])).toEqual([
    [1, 2],
    [2, 3]
  ]);
});

it('chain swipe respects link budget', () => {
  const sim = new PulseSimulation(options);
  sim.applyChainSwipe(pathFor(sim, [1, 2, 3, 4, 5, 6, 8, 11]));
  expect(sim.getSnapshot().linksUsed).toBeLessThanOrEqual(sim.getSnapshot().linkBudget);
});

it('chain analysis detects reachable energy nodes, dead ends, and loops', () => {
  const sim = new PulseSimulation(options);
  sim.applyChainSwipe(pathFor(sim, [1, 2, 3]));
  expect(sim.analyzeChain().toMatchObject({ reachableEnergyNodes: 1, deadEndNodeIds: [3], hasLoop: false }));
  sim.addLink(3, 4);
  sim.addLink(4, 7);
  sim.addLink(7, 9);
  sim.addLink(9, 2);
  const analysis = sim.analyzeChain();
  expect(analysis.hasLoop).toBe(true);
  expect(analysis.deadEndNodeIds).toHaveLength(0);
});

it('tapping Energy primes it, Delay cycles delay, and Splitter cycles priority', () => {
  const sim = new PulseSimulation(options);
  expect(sim.primeNode(2).ok).toBe(true);
  expect(sim.getNodes().find((node) => node.id === 2)?.primed).toBe(true);
  const delayBefore = sim.getNodes().find((node) => node.id === 3)?.delayLevel;
  sim.cycleNode(3);
  expect(sim.getNodes().find((node) => node.id === 3)?.delayLevel).not.toBe(delayBefore);
  const splitterBefore = sim.getNodes().find((node) => node.id === 4)?.splitterPriority;
  sim.cycleNode(4);
  expect(sim.getNodes().find((node) => node.id === 4)?.splitterPriority).not.toBe(splitterBefore);
});

it('pulse travels, delay pauses, splitter branches, and energy scores', () => {
  const sim = new PulseSimulation(options);
  sim.applyChainSwipe(pathFor(sim, [1, 2, 3, 4]));
  sim.addLink(4, 5);
  sim.addLink(4, 6);
  sim.playPulse();
  step(sim, 6500);
  const snapshot = sim.getSnapshot();
  expect(snapshot.score).toBeGreaterThan(100);
  expect(snapshot.maxMultiplier).toBeGreaterThanOrEqual(1);
});

it('pulse-phase tap stabilizes a node shortly before arrival', () => {
  const sim = new PulseSimulation(options);
  sim.applyChainSwipe(pathFor(sim, [1, 2, 3]));
  sim.playPulse();
  step(sim, 2200);
  const result = sim.stabilizeNode(3);
  expect(result.ok).toBe(true);
  expect(sim.getReplayPayload().liveInputs.some((input) => input.kind === 'stabilize' && input.nodeId === 3)).toBe(true);
});

it('Horizon Lens creates a temporary bridge', () => {
  const sim = new PulseSimulation(options);
  sim.applyChainSwipe(pathFor(sim, [1, 2, 3]));
  sim.playPulse();
  step(sim, 2600);
  const result = sim.applyLens(pathFor(sim, [3, 4]));
  expect(result.ok).toBe(true);
  expect(sim.getLinks().some((link) => link.temporal && link.fromId === 3 && link.toId === 4)).toBe(true);
  expect(sim.getReplayPayload().liveInputs.some((input) => input.kind === 'lens')).toBe(true);
});

it('dead end kills pulse and suggests fixes', () => {
  const sim = new PulseSimulation(options);
  sim.addLink(1, 2);
  sim.playPulse();
  step(sim, 6000);
  expect(sim.getSnapshot().phase).toBe('ended');
  expect(sim.getSnapshot().endReason).toBe('pulse-died');
  expect(sim.getSuggestedFixes().length).toBeGreaterThan(0);
});

it('replay with chainSwipe, taps, and lens reproduces result and stepHash', () => {
  const first = runScripted();
  const second = runScripted();
  expect(second.getReplayPayload().result).toEqual(first.getReplayPayload().result);
  expect(second.getReplayPayload().stepHash).toBe(first.getReplayPayload().stepHash);
});

function step(sim: PulseSimulation, ms: number): void {
  const frames = Math.ceil(ms / (1000 / 60));
  for (let index = 0; index < frames; index += 1) {
    sim.step(1000 / 60);
  }
}

function pathFor(sim: PulseSimulation, nodeIds: number[]) {
  return nodeIds.flatMap((nodeId, index) => {
    const node = sim.getNodes().find((candidate) => candidate.id === nodeId);
    if (!node) {
      throw new Error(`Missing node ${nodeId}`);
    }
    return [
      { x: node.x - 8, y: node.y - 8, t: index * 90 },
      { x: node.x, y: node.y, t: index * 90 + 35 },
      { x: node.x + 8, y: node.y + 8, t: index * 90 + 70 }
    ];
  });
}

function runScripted(): PulseSimulation {
  const sim = new PulseSimulation(options);
  sim.applyChainSwipe(pathFor(sim, [1, 2, 3]));
  sim.cycleNode(4);
  sim.playPulse();
}

```

```

step(sim, 2200);
sim.stabilizeNode(3);
step(sim, 300);
sim.applyLens(pathFor(sim, [3, 4]));
step(sim, 6500);
return sim;
}

```

tests/score-submit.test.ts

```

import { describe, expect, it } from 'vitest';
import scoreSubmit from '../netlify/functions/score-submit.mjs';

const replay = {
  version: 1,
  seed: 'eh-2026-05-29-alpha',
  startedAt: 1780051200000,
  survivalMs: 67421,
  score: 1280,
  energyCaptured: 87,
  maxStreak: 24,
  tapEvents: [],
  swipeEvents: [],
  phaseTransitions: []
};

const pulseReplay = {
  version: 1,
  mode: 'pulse-chain',
  seed: 'tutorial-001',
  startedAt: 1780185600000,
  buildInputs: [
    {
      t: 0,
      kind: 'chainSwipe',
      nodeIds: [1, 2, 3],
      path: [
        { x: 250, y: 1388, t: 0 },
        { x: 420, y: 1165, t: 90 },
        { x: 635, y: 1010, t: 180 }
      ]
    },
    { t: 220, kind: 'nodeTap', nodeId: 4, action: 'splitter' },
    { t: 520, kind: 'play' }
  ],
  liveInputs: [
    { t: 2200, kind: 'stabilize', nodeId: 3, rating: 'stabilized', success: true },
    {
      t: 1420,
      kind: 'lens',
      path: [
        { x: 835, y: 1215, t: 0 },
        { x: 700, y: 1410, t: 110 }
      ],
      fromId: 6,
      toId: 8,
      success: true
    }
  ],
  result: {
    score: 620,
    survivalMs: 12120,
    maxMultiplier: 2,
    loopsCompleted: 1,
    linksUsed: 5,
    bestChainLength: 5,
    energyNodesHit: 2,
    stabilized: false,
    collapsed: false
  },
  stepHash: '04ce9b1a'
};

describe('score-submit function', () => {
  it('accepts a valid replay payload', async () => {
    const response = await scoreSubmit(
      new Request('https://example.test/.netlify/functions/score-submit', {
        method: 'POST',
        headers: { 'content-type': 'application/json' },
        body: JSON.stringify(replay)
      })
    );
    expect(response.status).toBe(200);
    await expect(response.json()).resolves.toMatchObject({ ok: true, score: 1280 });
  });

  it('accepts a valid pulse-chain replay payload', async () => {
    const response = await scoreSubmit(
      new Request('https://example.test/.netlify/functions/score-submit', {
        method: 'POST',
        headers: { 'content-type': 'application/json' },
        body: JSON.stringify(pulseReplay)
      })
    );
    expect(response.status).toBe(200);
    await expect(response.json()).resolves.toMatchObject({ ok: true, score: 620, survivalMs: 12120 });
  });

  it('rejects malformed replay payloads', async () => {
    const response = await scoreSubmit(
      new Request('https://example.test/.netlify/functions/score-submit', {
        method: 'POST',
        headers: { 'content-type': 'application/json' },
        body: JSON.stringify({ ok: false })
      })
    );
    expect(response.status).toBe(422);
  });
});

```